Security of ZisK[1]

Héctor Masip-Ardevol
hector@zisk.technology
ZisK Core Team

October 30, 2026

Abstract

This document analyzes the security of ZisK's proof system, from the proof of a single chip to the statement about program execution that a final proof certifies. It is organized in four parts. Part I describes the proving system as implemented and bounds the knowledge soundness of every argument it uses. Part II analyzes the recursive aggregation layer: the bus argument that ties the AIR instances together is abstracted as an additive fingerprint of a weighted multiset of messages, and the per-node bookkeeping as accumulators over a commutative monoid; from any adversary producing an accepting final proof, an extractor obtains a bounded collection of locally satisfying, bus-balanced AIR witnesses. Part III states, and partly proves, that such a collection encodes a successful execution of the committed program. Part IV composes the three into the main theorem: for a fixed valid program, the final argument is knowledge-sound for ZisK's execution relation, with an explicit error bound. The idealizations made, the assumptions inherited from the primitives and the obligations left open are recorded in a ledger.

[1]This document describes ZisK v0.16.1 [48cf7cc – 2026-03-20].
Source code available at https://github.com/0xPolygonHermez/zisk.

Contents

1. Introduction **3**
1.1. What ZisK Proves . . . 3
2. Preliminaries **5**
2.1. Arithmetization . . . 5
2.2. Interfaces . . . 9
2.3. Security Definitions . . . 10
3. The ZisK Proof System **12**
3.1. Structure . . . 12
3.2. Accumulators . . . 13
3.3. Recursive Relations . . . 14
3.4. ZisK Instantiation . . . 17

Part I: The Proving System **19**

4. The ZisK Proving Protocol **19**
4.1. Reed–Solomon Codes . . . 19
4.2. Interactive Oracle Proofs . . . 21
4.3. Constraint Polynomials . . . 22
4.4. The Protocol . . . 24
4.5. Round-by-Round Soundness . . . 26
5. Non-Interactive Compilation and Extraction **30**
5.1. Merkle Commitments . . . 30
5.2. The Fiat–Shamir Transcript . . . 30
5.3. The BCS Transformation . . . 32
5.4. Binding Through the Global Challenge . . . 33
6. Knowledge Soundness of the Arguments **35**
6.1. Statement . . . 35
6.2. Concrete Parameters . . . 36

Part II: Recursion **38**

7. From the Root Check to Bus Balancing **38**
8. Descending the Aggregation Tree **41**
9. Main Theorem of the Recursion **45**
9.1. The Target Relation . . . 45
9.2. Correct-Witness Extractability . . . 47

(WIP) Part III: Semantics **50**

10. Executions and Words **50**
10.1. Transpiler Correctness . . . 50
10.2. Words and Limbs . . . 50
10.3. Where Limbs Are Well Formed . . . 51
10.4. Addresses, Steps and Wrap-Around . . . 51
11. Bus Semantics **52**
11.1. Pulls and Pushes . . . 52
11.2. Contracts . . . 52
11.3. The Channels of ZisK . . . 53
12. Chips **54**

12.1. Valid Programs 54
12.2. The Rom and the Fixed Tables 54
12.3. BinaryAdd 55
12.4. The Main Chip 55
12.5. The Memory AIRs 56
12.6. Obligations 57
12.7. The Order 57
13. Segmentation and Stitching **58**
13.1. Chains of Segments 58
13.2. Main and Memory Segments 58
13.3. The Global Step 59
14. Memory **59**
14.1. Accesses 59
14.2. The Memory of an Execution 60
15. Reconstructing the Execution **61**
(WIP) Part IV: Composition **62**
16. Main Theorem **62**
16.1. Execution Extractability 62
17. Ledger and Limitations **63**
References **65**

1. Introduction

Consider a party that needs the result of a computation but cannot, or does not want to, carry it out itself: a blockchain that must validate a block of transactions without every node re-executing it, a light client that follows a chain it cannot store, or a service that runs a computation on behalf of its users. The party delegates the computation to a *prover*, which runs the program and returns the result. Nothing guarantees that the prover is honest, so the result comes with a proof, and the party that checks it, the *verifier*, accepts the result only if the proof convinces it of the claim

"The program P, run on the input x, produces the output y."

A *zero-knowledge virtual machine* (zkVM) produces such proofs for arbitrary programs, written in ordinary programming languages and compiled to a standard instruction set, rather than for computations written by hand as constraints. ZisK [Zis26] is such a system, for programs compiled to RISC-V and translated into the ZisK instruction set. Its prover does not produce one monolithic proof of the entire execution. ZisK splits the work of the VM among a fixed family of *chips*, each in charge of one kind of operation (e.g., the main loop, memory or the arithmetic), and splits the execution of each chip into segments; every segment is proved by a succinct argument for its chip; and the proofs are aggregated by a tree of recursive verifiers whose root checks that the segments fit together. The verifier receives a single proof, the *final proof*, together with the public statement it attests to.

The security of such a system is not the security of its textbook components. Each layer departs from the construction whose analysis it would like to inherit: the interactive oracle proof (IOP) behind every argument is made non-interactive with a transcript that does not observe everything the verifier reads, the segment proofs share a bus key derived once for the whole execution, the recursive verifiers switch verification on and off depending on the shape of the tree, and the bus contributions of the segments are checked only once, summed, at the root. A gap in any of these places can make an accepting proof certify nothing, while every component remains sound on its own. This paper states precisely what an accepting final proof of ZisK certifies, and proves it for the protocol as deployed, under explicit assumptions on the primitives and on the chips, with some of the lemmas on the semantics stated and their proofs deferred.

1.1. What ZisK Proves.

The claim above cannot be what the final proof certifies as it stands. Delegation pays off only if checking the proof is much cheaper than running the program, and to check the claim the verifier would have to read P and x. Moreover, x is often the part of the computation that the prover wants to keep private. ZisK therefore proves a refinement of the claim, obtained by revisiting each of its three components and adding a fourth.

The program. ZisK proves executions of programs for the ZisK instruction set, a machine with registers and a byte-addressed memory, into which RISC-V programs are translated; a program starts from a fixed initial state. A program can be megabytes long, so the verifier does not receive P but a short *commitment* to it, a value computed from its code such that no one can find a different program with the same value. A verifier that knows which program it expects, for instance because its developer published the commitment, compares the commitment with that value, and the proof then speaks about that program and no other.

The input. The input x is supplied before the program starts. It may be large, such as a batch of transactions, or secret, such as a signing key, and in either case it should not be sent to the

verifier. The claim therefore does not fix x but quantifies over it: the proof says that *some* input leads to the output. Whatever the verifier must know about x has to be revealed by the program itself, for instance by outputting a hash of it.

The output. A program has no single output: when it halts, all of its memory is a candidate. ZisK lets the program *declare public outputs* while it runs, the values it chooses to reveal, and y is the sequence of these values.

The hints. To run efficiently, a program may request values from the prover while it runs, its *hints*, typically the result of a costly computation that the program then checks, such as a square root that it verifies by squaring. Hints do not appear in the original claim and come from an untrusted source, so the claim quantifies over them as it does over the input. Neither kind of data is trusted: the program must behave correctly whatever values it receives, and checking the hints is the responsibility of the program, not of ZisK.

The public statement. A proof is therefore checked against a short *public statement* $\boldsymbol{p}$ made of two parts: the public outputs y and the commitment to P. The input and the hints stay with the prover. What a proof claims is:

> *There exist an input x and a sequence of hints on which the committed program P, started from its initial state, runs according to the semantics of the ZisK instruction set until it halts, and declares exactly the public outputs y of $\boldsymbol{p}$.*

The whitepaper formalizes this claim as a relation $\mathcal{R}_{\text{exec}}$ whose witness is the whole execution: the program, the input, the hints and the sequence of machine states. The claim concerns the committed ZisK program; its connection to the original RISC-V binary rests on the correctness of the transpiler, which we assume.

Theorem 1.1 (Main Theorem, Informal). *For every efficient adversary that makes at most Q queries to the random oracles and outputs $\boldsymbol{p}$ with a final proof that the verifier accepts, an extractor outputs a witness of $\mathcal{R}_{\text{exec}}$ for $\boldsymbol{p}$, except with probability at most the sum of:*

- *The knowledge-soundness errors of the arguments proving the nodes of the aggregation tree, each about Q times the largest round-by-round error of its IOP.*
- *The soundness error of the bus argument at a key derived after the leaves are committed, for each of the Q queries.*
- *The extraction errors of the challenge accumulator and of the commitments.*

The proof of Theorem 1.1 retraces, in reverse, the way ZisK builds a proof: from the final proof it recovers a proof for every segment, from each of these proofs a valid trace of its segment, and from the traces an execution of the program (Figure 1).

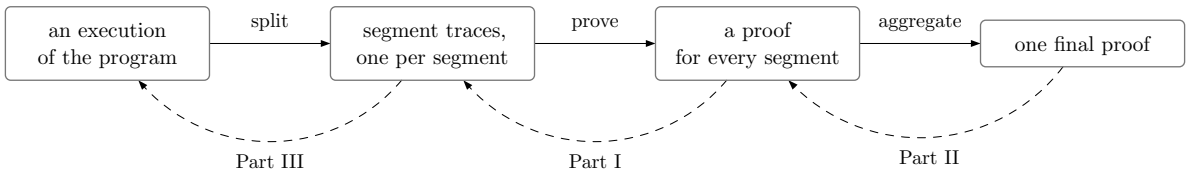


FIGURE 1. How ZisK proves an execution (solid arrows) and how the analysis recovers one from an accepted final proof (dashed arrows). Part IV composes the three steps.

Part I: The Proving System. Every chip and every recursive verifier of ZisK is proved with the same protocol: an IOP that checks the constraints of its chip, compiled into a succinct argument

with Merkle trees and the Fiat–Shamir transformation. The compilation is where ZisK departs from the textbook: the bus key and the first stage of every chip proof are bound through the global challenge rather than through the proof's own transcript. A prover that could choose its first-stage columns after the out-of-domain sample could satisfy the constraint check with any trace, so the binding lemma, which combines preimage awareness of the accumulator with extractability of the Merkle trees, is what makes the chip proofs sound.

Part II: Recursion. Each node of the aggregation tree carries a leaf counter, a homomorphic hash of the leaves' typed commitments, and the sum of their bus contributions; the root of the tree checks that the bus key was derived from the hash and that the contributions, with the global messages of the statement, sum to zero. The analysis descends the tree with one extraction game per layer, and a zero test at a key derived after the leaves are fixed turns the root check into balance of every bus.

Part III: Semantics. A balanced collection of locally satisfying witnesses of the ZisK chips encodes an execution: the memory argument reconstructs the memory, the continuation buses stitch the segments of each chip into one sequence, and the constraints of the Main chip and of the chips it delegates to implement one step of the instruction set. The decoding is deterministic and involves no cryptography.

Part IV: Composition. The last part composes the three steps into Theorem 16.2 and records, in a ledger, every assumption the theorem rests on and every obligation left open.

2. Preliminaries

This section fixes the notation and the definitions shared by the rest of the paper: the arithmetization layer (AIRs, interactions and buses), the interfaces of the cryptographic components (trace commitments, the multiset hash and key derivation) and the security notions they are required to satisfy. ZisK's concrete choices are collected in Section 3.4.

Notation and Parameters. Let $\mathbb{F}$ be a prime field of order p and $\mathbb{K}$ an extension field of $\mathbb{F}$. $\mathbb{Z}_p^n$ denotes the additive group of n-vectors of integers modulo p, the same set as $\mathbb{F}^n$ viewed as a $\mathbb{Z}$-module. For integers $a \le b$, $[a, b]$ and $[a, b)$ are the closed and half-open integer intervals and $[b] := [1, b]$. Field elements are identified with integers in $[0, p)$ when needed. λ is the security parameter; "PPT" means probabilistic polynomial time in λ, and $\mathsf{negl}(\lambda)$ and $\mathsf{poly}(\lambda)$ denote negligible and polynomially bounded functions. All algorithms have oracle access to the random oracles; Q bounds the total number of oracle queries an adversary makes.

The recursion is governed by two integer parameters: the *aggregation arity* $A \ge 2$, the number of children an aggregation node can take, and the *leaf-count exponent* $B \ge 1$: an aggregation tree may contain at most $2^B - 1$ leaves. We assume throughout that

$$A \cdot 2^B \le p, \quad 2^B < p, \quad 2^B \le \mathsf{poly}(\lambda) :$$

the first condition guarantees that sums of A range-checked counters never wrap around in $\mathbb{F}$, the second is the cardinality restriction required by the multiset hash, and the third makes the number of leaves, and hence the running time of the extractor, polynomial.

2.1. Arithmetization.

A ZisK proof covers many *instances* of a fixed family of chips. The constraints of an instance read two kinds of values: the *columns* of its trace, which take one value per row, and the *scalar values*, which are constant over the rows.

Definition 2.1 (AIR and Witness). Given integers $N \geq 1$, $W \geq 1$ and $V \geq 0$ and a finite set $O \subset \mathbb{Z}$ of *row offsets*, an *algebraic intermediate representation* (AIR) of *height* N is a finite set of *constraints*

$$\mathcal{C} \subset \mathbb{F}\Big[X_j^{(o)}, Y_l : j \in [W], o \in O, l \in [V]\Big].$$

The variables $X_j^{(o)}$ refer to the columns and the variables Y_l to the scalar values. A *witness* for $\mathcal{C}$ is a pair $w = (\boldsymbol{T}, \boldsymbol{v})$ with $\boldsymbol{T} \in \mathbb{F}^{N \times W}$, the *trace*, and $\boldsymbol{v} \in \mathbb{F}^V$, the *values*. At row $r \in [0, N)$ the variable $X_j^{(o)}$ takes the value $T_j(r + o)$, with row indices taken modulo N, and the variable Y_l the value v_l. The witness *satisfies* $\mathcal{C}$ if every constraint polynomial evaluates to zero at every row.

Offsets may be negative: $X_j^{(o)}$ with $o < 0$ reads row $r - |o|$. Since the rows are taken modulo N, the constraints are enforced on a cyclic domain, and shifting every constraint by $\min O$ rows gives an equivalent AIR with offsets in $[0, \max O - \min O]$.

The columns and the values of a witness are of several kinds, which differ in who chooses them, who sees them and how many instances share them. The description of an AIR partitions both.

Definition 2.2 (Column Classes). The description of an AIR $\mathcal{C}$ with W columns and height N partitions its column indices as $[W] = J^{\mathrm{fix}} \sqcup J^{\mathrm{cach}} \sqcup J^{\mathrm{wit}}$:

1. The *fixed columns* J^{fix}, whose values are prescribed as part of the description of $\mathcal{C}$ and hence known to both the prover and the verifier.
2. The *cached columns* J^{cach}, whose values are computed once and reused across different proofs. Their values are not part of the description; instead, a commitment to them (Definition 2.11) travels in the publics, at the coordinates designated by a *cached-commitment binding* $\psi : [m] \to [k]$.
3. The *witness columns* J^{wit}, computed afresh for every proof and only known in full to the prover.

The *prescribed data* of $\mathcal{C}$ is the family $\boldsymbol{d} = \big(d_j\big)_{j \in J^{\mathrm{fix}}}$ of fixed columns $d_j \in \mathbb{F}^N$. For a trace $\boldsymbol{T}$ and a set of columns J we write $\boldsymbol{T}|_J$ for the submatrix of the columns in J.

Definition 2.3 (Value Classes). The description of an AIR $\mathcal{C}$ with V values partitions its value indices as $[V] = I^{\mathrm{pub}} \sqcup I^{\mathrm{pv}} \sqcup I^{\mathrm{air}}$, with $\big|I^{\mathrm{pub}}\big| = k$ and $|I^{\mathrm{pv}}| = k'$ for every AIR of a proof. For values $\boldsymbol{v}$ and a set of indices I we write $\boldsymbol{v}|_I$ for the subvector of the coordinates in I.

1. The *publics* $\boldsymbol{p}_{\mathrm{ext}} := \boldsymbol{v}|_{I^{\mathrm{pub}}} \in \mathcal{P}_{\mathrm{ext}} := \mathbb{F}^k$, shared by all instances and seen by the external verifier.
2. The *proof values* $\boldsymbol{p}_{\mathrm{pv}} := \boldsymbol{v}|_{I^{\mathrm{pv}}} \in \mathcal{P}_{\mathrm{pv}} := \mathbb{F}^{k'}$, chosen by the prover once per proof and shared by all instances, but not seen by the external verifier.
3. The *AIR values* $\boldsymbol{a} := \boldsymbol{v}|_{I^{\mathrm{air}}}$ of an instance, chosen by the prover for that instance.

The publics and the proof values form the *public statement*

$$\boldsymbol{p} = \big(\boldsymbol{p}_{\mathrm{ext}}, \boldsymbol{p}_{\mathrm{pv}}\big) \in \mathcal{P} := \mathcal{P}_{\mathrm{ext}} \times \mathcal{P}_{\mathrm{pv}}.$$

Every AIR that takes part in a proof reads the whole public statement, but the external verifier sees only $\boldsymbol{p}_{\mathrm{ext}}$: the proof values are hidden by the Final Verifier (Definition 3.9).

The constraints of an AIR therefore read, besides its trace, only its own AIR values and the public statement: they are satisfied or not under a public statement $\boldsymbol{p}$. They are also *local*: a constraint relates a few rows of a single trace, those within its window of offsets O. Many of the facts that a zkVM must establish are not of this kind. When the main loop delegates an addition to an arithmetic unit, the result it uses must be the one that the unit computed; and a load must return

the value of the last store to the same address, which may have been executed in another segment and recorded by another AIR. ZisK expresses such relations with *buses*. Every row of an AIR may put messages, tuples of field elements, on a bus, each with a *multiplicity*: positive when the row proves the message and negative when it assumes it (Section 11). The relation holds when, on every bus, the multiplicities of each message add up to zero over all the AIR instances of a proof, that is, when the bus is *balanced* (Definition 2.10). An *interaction* declares, for one AIR, the message that each of its rows puts on a bus and with which multiplicity.

Definition 2.4 (Interaction). An *interaction* of an AIR $\mathcal{C}$ on the bus $b \in \mathbb{F}$ is a tuple $\iota = (b, m_\iota, \boldsymbol{e}_\iota)$, where the *multiplicity* m_ι and the *message* components $e_{\iota,1}, ..., e_{\iota,\ell_b}$ are polynomials over $\mathbb{F}$ in the variables of $\mathcal{C}$. Given a witness w and a row r, we write $m_\iota(w, r) \in \mathbb{F}$ and $\boldsymbol{e}_\iota(w, r) \in \mathbb{F}^{\ell_b}$ for their evaluations at row r. The set of bus identifiers in use is $\mathcal{B}$; every interaction on bus b has the common message length ℓ_b, and $\ell \coloneqq \max_{b \in \mathcal{B}} \ell_b$.

Definition 2.5 (Chip). A *chip* is a pair $(\mathcal{C}, \mathcal{I})$ of an AIR $\mathcal{C}$ and a finite set $\mathcal{I}$ of interactions of $\mathcal{C}$.

A chip is the unit into which ZisK splits the work of the VM: the main loop, each memory region, each arithmetic unit and each precompile is a chip. Throughout, we fix a positive integer M and an indexed family of chips

$$\mathfrak{A} = \{(\mathcal{C}_\tau, \mathcal{I}_\tau)\}_{\tau \in [M]},$$

where τ is the *type* of the chip and $\mathcal{C}_\tau$ has height N_τ and parameters W_τ, O_τ and V_τ. We write $\mathcal{W}_\tau \coloneqq \mathbb{F}^{N_\tau \times W_\tau} \times \mathbb{F}^{V_\tau}$ for the witness space of type τ.

Each chip carries its column and value classes and its prescribed data; we index them by the type, as in J_τ^{fix}, I_τ^{air}, $\boldsymbol{d}_\tau$ and ψ_τ.

Definition 2.6 (Local Satisfaction). For $\tau \in [M]$, $\boldsymbol{p} = (\boldsymbol{p}_{\text{ext}}, \boldsymbol{p}_{\text{pv}}) \in \mathcal{P}$ and $w = (\boldsymbol{T}, \boldsymbol{v}) \in \mathcal{W}_\tau$, the predicate $\text{Local}_\tau(\boldsymbol{p}, w)$ holds if and only if:

1. w satisfies $\mathcal{C}_\tau$.
2. The values carry the public statement: $\boldsymbol{v}|_{I_\tau^{\text{pub}}} = \boldsymbol{p}_{\text{ext}}$ and $\boldsymbol{v}|_{I_\tau^{\text{pv}}} = \boldsymbol{p}_{\text{pv}}$.
3. $\boldsymbol{T}$ agrees with the prescribed data on the fixed columns: $T_j(r) = d_{\tau,j}(r)$ for every $j \in J_\tau^{\text{fix}}$ and $r \in [0, N_\tau)$.
4. The cached columns open the commitment carried in $\boldsymbol{p}_{\text{ext}}$:

$$\mathsf{Commit}_{\text{cach}}\left(\boldsymbol{T}|_{J_\tau^{\text{cach}}}\right) = \left(p_{\psi_\tau(1)}, ..., p_{\psi_\tau(m)}\right),$$

   where p_i is the i-th coordinate of $\boldsymbol{p}_{\text{ext}}$.

Fixed columns are part of the description of $\mathcal{C}_\tau$. Cached columns are not: they are computed once per program, committed with $\mathsf{Commit}_{\text{cach}}$, and the commitment travels in the publics, where the description of $\mathcal{C}_\tau$ designates its coordinates. The program being executed is therefore bound through $\boldsymbol{p}$ rather than hard-wired into the AIRs: the program ROM lives in the cached columns of the Rom chip, and whoever accepts a final proof for $\boldsymbol{p}_{\text{ext}}$ must check that the cached commitments carried in $\boldsymbol{p}_{\text{ext}}$ are those of the intended program.

Definition 2.7 (Message Multiset of a Witness). Let $\mathcal{V} \coloneqq \left\{(b, v) : b \in \mathcal{B}, v \in \mathbb{F}^{\ell_b}\right\}$ be the set of *bus messages*. An $\mathbb{F}$-*weighted multiset* of messages is a finitely supported map $D : \mathcal{V} \to \mathbb{F}$; we write $|D|$ for the size of its support and add such maps pointwise. For $\tau \in [M]$ and $w \in \mathcal{W}_\tau$, the *message multiset* of w is

$$\mathrm{Msg}_\tau(w)(b,v) := \sum_{\iota \in \mathcal{I}_\tau, b_\iota = b} \sum_{r=0}^{N_\tau - 1} m_\iota(w,r) \cdot \mathbf{1}[\boldsymbol{e}_\iota(w,r) = v].$$

Its support has at most $N_\tau \cdot |\mathcal{I}_\tau|$ elements.

A row that proves a message, with positive multiplicity, and a row of the same witness that assumes it, with negative multiplicity, cancel in $\mathrm{Msg}_\tau(w)$: the multiset records the net flow of every message out of the witness.

Definition 2.8 (Bus Fingerprint). A *bus fingerprint* $\mathcal{F}$ consists of a finite *key space* $\mathcal{X}$, an abelian group $G_\mathcal{F}$ of *values*, a class $\mathcal{D}$ of *admissible* nonzero $\mathbb{F}$-weighted multisets of messages, and, for every key $\chi \in \mathcal{X}$, a map $\mathcal{F}_\chi$ from $\mathbb{F}$-weighted multisets of messages to $G_\mathcal{F} \cup \{\perp\}$, such that:

1. (*Additivity*) $\mathcal{F}_\chi(0) = 0$, and $\mathcal{F}_\chi(D + D') = \mathcal{F}_\chi(D) + \mathcal{F}_\chi(D')$ whenever the right-hand side is defined;
2. (*Soundness*) For every Λ and every adversary $\mathcal{A}$ that outputs a multiset before the key is drawn uniformly from $\mathcal{X}$, the *fingerprint advantage*:

   $$\mathsf{Adv}^{\mathsf{fp}}_{(\mathcal{F},\Lambda)}(\mathcal{A}) := \Pr\big[D \leftarrow \mathcal{A}; \chi \leftarrow \mathcal{X} : D \in \mathcal{D} \wedge |D| \le \Lambda \wedge \mathcal{F}_\chi(D) = 0\big]$$

   is small. The fingerprint is *statistically sound* with error $\varepsilon_\mathcal{F}$ if $\mathsf{Adv}^{\mathsf{fp}}_{(\mathcal{F},\Lambda)}(\mathcal{A}) \le \varepsilon_\mathcal{F}(\Lambda)$ for every adversary, with $\varepsilon_\mathcal{F}$ non-decreasing, and *computationally sound* if the advantage is negligible for every PPT adversary.

The *bus contribution* of $w \in \mathcal{W}_\tau$ under the key χ is

$$\mathrm{BusCont}_\chi(\mathcal{I}_\tau, w) := \mathcal{F}_\chi(\mathrm{Msg}_\tau(w)) \in G_\mathcal{F} \cup \{\perp\}.$$

A fingerprint is an additive summary of a multiset that vanishes on an admissible nonzero multiset only for few keys, or only for keys no efficient adversary can anticipate.

Definition 2.9 (Typed Witness Collection). A *typed witness collection* is a finite list

$$L = \big((\tau_1, w_1), ..., \big(\tau_{|L|}, w_{|L|}\big)\big), \quad \tau_i \in [M], \quad w_i \in \mathcal{W}_{\tau_i}.$$

Types and witnesses may repeat, and each entry counts separately; the order of the list carries no meaning. The *number of bus terms* of L is $\Lambda(L) := \sum_{i=1}^{|L|} N_{\tau_i} \cdot \big|\mathcal{I}_{\tau_i}\big|$, and we set $\Lambda_{\max} := (2^B - 1) \cdot \max_{\tau \in [M]} N_\tau |\mathcal{I}_\tau| + g$, so that $\Lambda(L) + g \le \Lambda_{\max}$ whenever $|L| < 2^B$, where g bounds the global messages introduced below.

Some messages are not emitted by any witness: they belong to the system as a whole and are determined by the public statement. In ZisK these *global messages* open and close the continuation chains, initialize the registers and inject the public outputs into the buses. We model them as a fixed function $\mathcal{G}$ that maps every public statement $\boldsymbol{p} \in \mathcal{P}$ to an $\mathbb{F}$-weighted multiset of messages $\mathcal{G}(\boldsymbol{p})$, with $|\mathcal{G}(\boldsymbol{p})| \le g$ for a constant g.

Definition 2.10 (Balance of a Witness Collection). The *signed multiplicity* of a typed witness collection L is the $\mathbb{F}$-weighted multiset

$$\mathcal{M}_L := \sum_{i=1}^{|L|} \mathrm{Msg}_{\tau_i}(w_i),$$

that is,

$$\mathcal{M}_L(b,v) = \sum_{i=1}^{|L|} \sum_{\iota \in \mathcal{I}_{\tau_i}, b_\iota = b} \sum_{r=0}^{N_{\tau_i}-1} m_\iota(w_i, r) \cdot \mathbf{1}[\boldsymbol{e}_\iota(w_i, r) = v].$$

The collection is *balanced for* $\boldsymbol{p} \in \mathcal{P}$, written $\mathrm{Balanced}_{\boldsymbol{p}}(L)$, if $\mathcal{M}_L + \mathcal{G}(\boldsymbol{p}) = 0$, i.e. $\mathcal{M}_L(b,v) + \mathcal{G}(\boldsymbol{p})(b,v) = 0$ for every bus $b \in \mathcal{B}$ and every message $v \in \mathbb{F}^{\ell_b}$. Note that $|\mathcal{M}_L + \mathcal{G}(\boldsymbol{p})| \le \Lambda(L) + g$.

Note that multiplicities are field elements: $\mathrm{Balanced}_{\boldsymbol{p}}(L)$ does not by itself assert that every message is sent and received the same number of times as integers, since p copies of a message cancel. We return to this point in Remark 7.2.

2.2. **Interfaces.**

A *non-interactive argument system* $\Pi = (\mathbf{P}, \mathbf{V})$ for a relation $\mathcal{R}$ consists of a *prover* $\mathbf{P}$, which on input a statement $\mathbb{x}$ and a witness $\mathbb{w}$ outputs a proof $\pi \leftarrow \mathbf{P}(\mathbb{x}, \mathbb{w})$, and a deterministic *verifier* $\mathbf{V}$, together with a verification key vk_Π produced at setup, where $\mathbf{V}(\mathsf{vk}_\Pi, \mathbb{x}, \pi) \in \{0,1\}$ and the value 1 means that the proof π is accepted for the statement $\mathbb{x}$. We write $\mathbf{V}_X(\mathbb{x}, \pi) \coloneqq \mathbf{V}(\mathsf{vk}_X, \mathbb{x}, \pi)$ for the verifier of the system attesting to the relation $\mathcal{R}_X$, and keep the explicit form $\mathbf{V}(\mathsf{vk}, \mathbb{x}, \pi)$ where the key is a variable. The relations of Section 3.3 refer to argument systems only through their verifiers, so the prover is never named again, and to the following fixed public maps only through their signatures. Their security properties are defined in Section 2.3, and ZisK's choices are given in Section 3.4. Two of the maps are commitments, in the following sense.

Definition 2.11 (Deterministic Commitment Scheme). A *deterministic commitment scheme* Com consists of a set of *messages*, a finite set of *commitments*, and a polynomial-time computable function Commit from the former to the latter. It is *binding* if for every PPT adversary $\mathcal{A}$

$$\mathsf{Adv}^{\mathsf{bind}}_{\mathsf{Com}}(\mathcal{A}) \coloneqq \Pr\big[(m, m') \leftarrow \mathcal{A}\big(1^\lambda\big) : m \neq m' \wedge \mathsf{Commit}(m) = \mathsf{Commit}(m')\big] \le \mathsf{negl}(\lambda).$$

Since Commit is deterministic, binding is collision resistance of Commit. When the messages are vectors or matrices over $\mathbb{F}$, the scheme is a *vector commitment* if it also offers openings of individual positions.

The following interface elements are used throughout:

- **Carriers.** A finite key space $\mathcal{X}$ and an abelian group $G_{\mathcal{F}}$ of fingerprint values (Definition 2.8), and a finite abelian *accumulator group* $G_{\mathcal{H}}$.
- **Trace commitment.** A deterministic commitment scheme Com whose messages are the first-stage parts of witnesses, with commitment space $\{0,1\}^*$: for $w \in \bigcup_{\tau \in [M]} \mathcal{W}_\tau$, the commitment $\mathrm{com} = \mathsf{Commit}(w)$ covers the witness columns and the first-stage AIR values of w, and binding and extraction (Definition 2.13) refer to this part. The argument system for a chip takes the key $\chi \in \mathcal{X}$ as a public input, so the witness is key-independent.
- **Cached-column commitment.** A deterministic commitment scheme $\mathsf{Com}_{\mathrm{cach}}$ with message space $\bigcup_{\tau \in [M]} \mathbb{F}^{N_\tau \times |J^{\mathrm{cach}}_\tau|}$ and commitment space $\mathbb{F}^m$. Its output for the cached columns of a type is not part of a verification key but of the public statement (Definition 2.2).
- **Hash-to-group and multiset hash.** $h : [M] \times \{0,1\}^* \to G_{\mathcal{H}}$ maps a *typed commitment* (τ, com) to a group element, and

  $$\mathcal{H}(\mathcal{S}) \coloneqq \sum_{x \in \operatorname{supp} \mathcal{S}} \mathrm{mult}_{\mathcal{S}}(x) \cdot h(x) \in G_{\mathcal{H}}$$

  is the induced multiset hash on finite multisets $\mathcal{S}$ of typed commitments, where $\mathrm{mult}_{\mathcal{S}}(x)$ is the multiplicity of x in $\mathcal{S}$. $\mathcal{H}$ is additively homomorphic over multiset union.

- **Key derivation.** $\mathsf{DeriveKey} : \mathcal{P} \times G_{\mathcal{H}} \to \mathcal{X}$, a fixed public function of the public statement and the accumulated value.

2.3. **Security Definitions.**

Definition 2.12 (Knowledge Soundness). A non-interactive argument system Π for a relation $\mathcal{R}$ is *knowledge-sound* if there exists a PPT algorithm $\mathcal{E}$, the *extractor*, such that for every PPT adversary $\mathcal{A}$:

$$\mathsf{Adv}^{\mathsf{ks}}_{\Pi}(\mathcal{A}) := \Pr\big[(\mathbb{x}, \pi) \leftarrow \mathcal{A}\big(1^{\lambda}\big); \mathbb{w} \leftarrow \mathcal{E}(\mathbb{x}, \pi) : \mathbf{V}(\mathsf{vk}_{\Pi}, \mathbb{x}, \pi) = 1 \wedge (\mathbb{x}; \mathbb{w}) \notin \mathcal{R}\big] \leq \mathsf{negl}(\lambda),$$

the probability taken over the coins of $\mathcal{A}$ and $\mathcal{E}$ and the random oracles, to which all algorithms have access. We assume that the running time of $\mathcal{E}(\mathbb{x}, \pi)$ is at most $\mathsf{Time}\big(\mathcal{A}\big(1^{\lambda}\big)\big) + \mathsf{poly}(\lambda)$ for every $\mathcal{A}$ and every $(\mathbb{x}, \pi)$ in the support of $\mathcal{A}$'s output: the extractor incurs an additive polynomial overhead over one evaluation of $\mathcal{A}$. Besides $(\mathbb{x}, \pi)$, the extractor reads the list of $\mathcal{A}$'s random-oracle queries and answers, which we leave implicit in the notation. We call $\mathsf{Adv}^{\mathsf{ks}}_{\Pi}(\mathcal{A})$ the *knowledge-soundness error* of $\mathcal{A}$ against Π.

Remark (Straight-Line Extraction). *The extractor of Definition 2.12 is straight-line: it receives the statement, the proof and the adversary's oracle queries, and does not rewind the adversary. For succinct proofs this is not achievable in the plain model, and in the random-oracle model it holds for one argument but does not compose across a recursion (Remark 2.1). The definition is a proxy that makes the reduction structure explicit; a concrete instantiation replaces it by extractability in the random-oracle model with the overhead discussed there.*

Definition 2.13 (Straight-Line Extractable Commitment). A deterministic commitment scheme Com (Definition 2.11) is *straight-line extractable* if there is a PPT algorithm $\mathcal{E}_{\mathsf{Com}}$ that, given a transcript $\mathcal{T}$ of random-oracle queries and answers and a value com, outputs a message or $\perp$, such that for every PPT adversary $\mathcal{A}$ that at some point during its execution announces a value com and at the end outputs a message m,

$$\mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{A}) := \Pr\big[\mathsf{Commit}(m) = \mathrm{com} \wedge \mathcal{E}_{\mathsf{Com}}(\mathcal{T}_{\leq t}, \mathrm{com}) \neq m\big] \leq \mathsf{negl}(\lambda),$$

where t is the time of the announcement and $\mathcal{T}_{\leq t}$ the transcript up to it.

For a Merkle tree over a random oracle the extractor recomputes the leaves from the tree of queried nodes below the root; it fails only if the adversary later opens the root to a leaf it had not queried, which requires guessing an oracle output or finding a collision. Straight-line extractability implies binding: if $\mathcal{A}$ outputs $m \neq m'$ with $\mathsf{Commit}(m) = \mathsf{Commit}(m') = \mathrm{com}$, the extractor's output on com differs from at least one of them, so the adversary $\mathcal{D}$ that runs $\mathcal{A}$, announces com and returns one of m, m' chosen at random fails extraction with at least half of $\mathcal{A}$'s success probability: $\mathsf{Adv}^{\mathsf{bind}}_{\mathsf{Com}}(\mathcal{A}) \leq 2 \cdot \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D})$.

Definition 2.14 (Preimage Awareness of the Accumulator). The multiset hash $\mathcal{H}$ is *preimage aware on multisets of cardinality below* 2^B [DRS09] if there is a PPT algorithm $\mathcal{E}_{\mathcal{H}}$ that, given a transcript $\mathcal{T}$ of random-oracle queries and answers and a value $y \in G_{\mathcal{H}}$, outputs a multiset of typed commitments or $\perp$, such that for every PPT adversary $\mathcal{A}$ that at some point during its execution announces a value y and at the end outputs a multiset $\mathcal{S}$,

$$\mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{A}) := \Pr\big[|\mathcal{S}| < 2^B \wedge \mathcal{H}(\mathcal{S}) = y \wedge \mathcal{E}_{\mathcal{H}}(\mathcal{T}_{\leq t}, y) \neq \mathcal{S}\big] \leq \mathsf{negl}(\lambda),$$

where t is the time of the announcement, $\mathcal{T}_{\leq t}$ the transcript up to it, and $|\mathcal{S}|$ the cardinality of $\mathcal{S}$, counted with multiplicity.

The definition mirrors Definition 2.13, with the accumulator in the role of the commitment: what is announced is a value of $G_{\mathcal{H}}$, and what is extracted is the small multiset the adversary later opens it to. The cardinality restriction is necessary for ZisK's accumulator group: $\mathbb{Z}_p^n$ has exponent p, so adding p copies of any element to a multiset leaves $\mathcal{H}$ unchanged, and $2^B < p$ rules this out. Preimage awareness implies collision resistance on such multisets, since two distinct small multisets with the same hash would both have to equal the extractor's output.

For LtHash, collision resistance on bounded multisets reduces to the short-integer-solution problem [Ajt96, Zis25]; the reduction requires the inner digest of h to be collision resistant on its own, which a single field element is not, so we assume the digest has at least 2λ bits (Section 17). Preimage awareness, however, is not known to follow from any standard assumption for an algebraic accumulator, and assuming it is the second idealization of Remark 2.1. It is nevertheless the property the analysis needs, and collision resistance alone would not do: an adversary may fix a target y by querying $\mathsf{DeriveKey}$ at it and only afterwards obtain the h-values of the elements it assembles into a multiset hashing to y, choosing which fresh values to combine after seeing them. Whether this succeeds is a preimage question, computational rather than statistical, and it is exactly what the extractor of Definition 2.14 rules out.

Definition 2.15 (Random-Oracle Model). The hash-to-group function h, the key derivation $\mathsf{DeriveKey}$, the Fiat–Shamir transcript of every argument and the compression functions of the Merkle trees of Com and $\mathsf{Com}_{\mathrm{cach}}$ are independent random oracles. An adversary makes at most Q queries in total; every algorithm, including verifiers and extractors, has oracle access.

The definitions above, together with the random-oracle model, are the setting of the whole analysis. Two of the properties it relies on are idealizations, which we make explicit.

Remark 2.1 (Idealized Proof Model). *The analysis is idealized in two respects.*

First, Definition 2.12 postulates a straight-line extractor that runs in essentially the time of the adversary. Because the argument systems are hash-based, the natural model is the random-oracle model, in which such extractors exist for a single argument [BCS16]. Composing straight-line extraction across the layers of a recursion tree, however, requires every argument to remain knowledge-sound relative to the same oracle, and such relativized succinct arguments do not exist in general [BCG24]. In our analysis we idealize over this obstruction and assume that straight-line extraction composes.

Second, the bus argument is sound only if the multiset of typed commitments that the accumulator $\boldsymbol{c}$ stands for is fixed before the random oracle answers at $\boldsymbol{c}$. For the trace commitments this is the straight-line extractability of Merkle trees in the random-oracle model (Definition 2.13). For the accumulator we assume the analogous property, preimage awareness (Definition 2.14): a straight-line extractor recovers, from the transcript at the time a value is announced, the multiset the adversary later opens it to. For the algebraic accumulator $\mathcal{H}$ this is an idealization. Distinct small multisets with equal hash exist in abundance; only the hardness of finding them protects the scheme; and recovering a multiset from its accumulator is a subset-sum problem, so no straight-line extractor is known; only collision resistance follows from standard assumptions. Remark 7.1 discusses the rigorous alternative.

Consequently, while the analysis produces explicit advantage bounds and reduction running times (Theorem 9.5), the two idealizations mean that these should be read as statements about the reduction structure (the recursion topology is sound and the relations compose) rather than as a concrete security level.

3. The ZisK Proof System

This section describes the system that the rest of the paper analyzes: the aggregation tree into which a ZisK proof is organized, the three accumulators that its nodes carry, the relation attested by each kind of node, and the concrete choices ZisK makes for the components of Section 2. Part I proves knowledge soundness for the argument systems that attest to these relations, Part II shows what an accepting proof of the whole tree implies, and Part III gives the result its meaning in terms of program execution.

3.1. Structure.

A ZisK proof is a tree. Its leaves are the AIR instances of an execution: for each type $\tau \in [M]$ the prover produces some number of witnesses in $\mathcal{W}_\tau$, one per segment of that type, and proves each with an argument system for the AIR of the type. Above every AIR proof sits a *leaf verifier*, which verifies the AIR proof, possibly through a compressor (below), and re-exposes its statement in a common format. Above the leaf verifiers, *aggregation nodes* have A child slots each, where A is the aggregation arity. A slot holds either a child proof or a *null child*, a placeholder that is not verified and contributes nothing, and at least two slots of every aggregation node hold proofs. The tree need not be balanced, because a node is proved as soon as enough children are available. At the top, a single *Final Verifier* verifies the root of the aggregation tree and performs the global checks; its statement, the external statement $\boldsymbol{p}_{\text{ext}}$, is the only thing an external verifier sees. Figure 2 shows an example with three leaves. The leaf verifiers, the aggregation nodes and the Final Verifier are the *recursive verifiers*: each is itself an AIR, whose constraints encode a verifier, and is proved with the same argument system as the chips (Section 4).

ZisK may also insert a *compressor* above an AIR proof: a recursive verifier that verifies that proof and re-exposes its statement unchanged, so that the statement is re-proved at a different rate or with a smaller verifier (Definition 3.4); ZisK does so for the AIR proofs of some chips, between them and their leaf verifiers. A compressor changes no statement, so the analysis treats a compressed proof as a proof of the argument it compresses, at the cost of one more term in the knowledge-soundness error (Lemma 6.3).

Three quantities travel up the tree and are combined at every aggregation node: the number of leaves, an accumulation of the leaves' commitments, and an accumulation of the leaves' bus contributions. They are the *accumulators* of Section 3.2; the counter is range-checked at every node, and the Final Verifier checks one condition on each.

The order of operations matters for the bus argument. Bus contributions are fingerprints under a key χ, and the key must be fixed before any leaf is proved yet must not be under the prover's control. ZisK derives it from the leaves themselves: the prover commits to every witness, accumulates the typed commitments with the multiset hash $\mathcal{H}$, derives χ from the public statement and the accumulated value with $\mathsf{DeriveKey}$, and only then proves the leaves with χ as a public input. The tree recomputes the accumulation as it aggregates, and the Final Verifier checks that the key the leaves used is the one derived from what was actually aggregated. Because the shape of the tree is not known when the key is derived, the accumulation must not depend on it; this is why the accumulators below are commutative monoids.

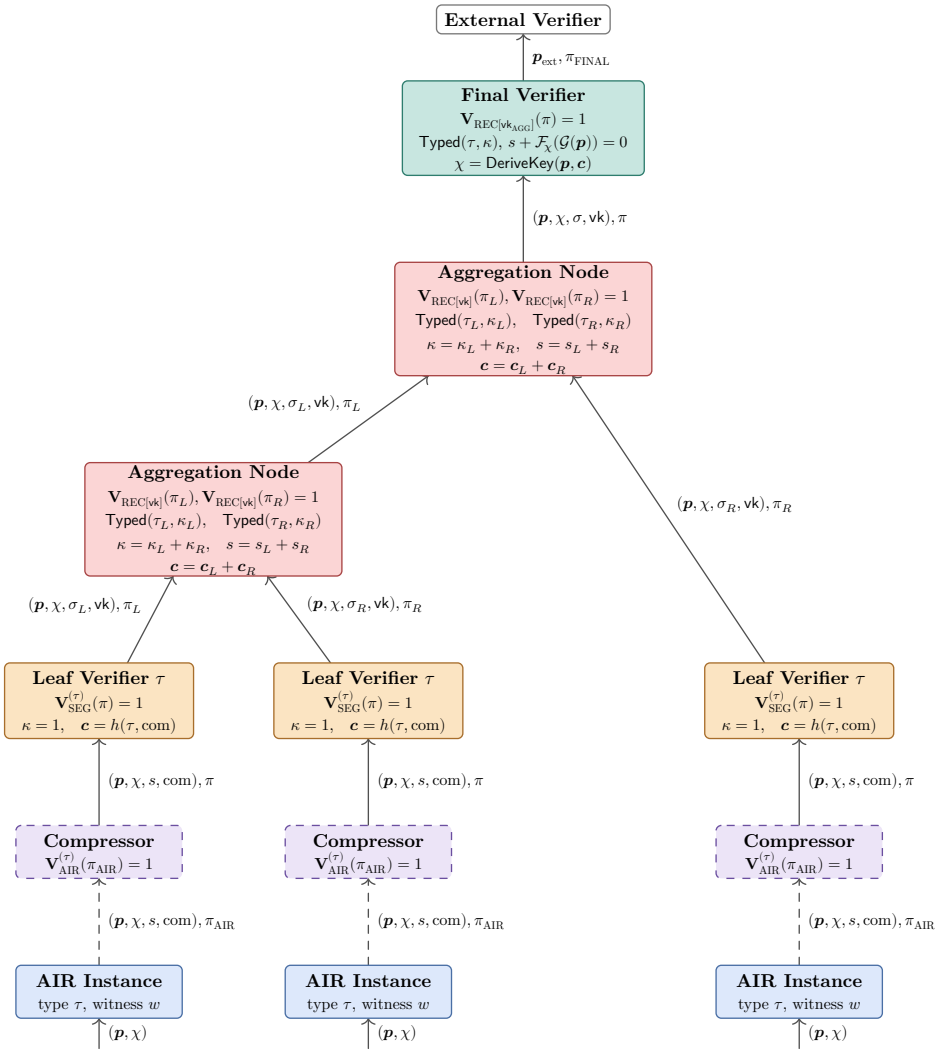


FIGURE 2. An aggregation tree with three AIR instances. Each arrow carries the statement and the proof a node produces, which the node above takes as witness: an AIR instance of type τ proves $(\boldsymbol{p}, \chi, s, \text{com})$; a leaf verifier re-exposes it as $(\boldsymbol{p}, \chi, \sigma, \mathsf{vk})$ with $\sigma = (\tau, 1, \boldsymbol{c}, s)$; an aggregation node adds its children's accumulators, dispatches on their types and declares type $M + 1$; the Final Verifier fixes $\mathsf{vk} = \mathsf{vk}_{\text{AGG}}$, checks the root predicates and exposes only $\boldsymbol{p}_{\text{ext}}$. The pair $(\boldsymbol{p}, \chi)$ is the public input of every AIR instance. The AIR proofs of some types are compressed: an optional compressor (dashed, purple) verifies the proof and re-exposes the same statement, and the leaf verifier checks the compressor's proof instead. Aggregation nodes are drawn with two children; in general they have A slots, the unused ones holding null children.

3.2. **Accumulators.**

Definition 3.1 (Accumulator). An *accumulator* over the aggregation tree consists of a commutative monoid of *values*, a *leaf contribution* assigning a value to every leaf, a *child check*, a predicate on values that every aggregation node evaluates on the value of each of its children, and a *root check*, a predicate on values that the Final Verifier evaluates on the value of the root of the aggregation tree. The value of a leaf is its contribution, and the value of an aggregation node is the sum of the values of its non-null children. By associativity and commutativity, the value of any node is the sum of the contributions of the leaves below it, whatever the shape of the subtree.

The recursion carries three accumulators. Their values at a node, together with the node's type, form the *recursive statement*

$$\sigma = (\tau, \kappa, \boldsymbol{c}, s),$$

where the type $\tau \in [M+1]$ is the type of the verified chip for a leaf verifier and the marker $M+1$ for an aggregation node. We write $\sigma.\tau$, $\sigma.\kappa$, $\sigma.\boldsymbol{c}$ and $\sigma.s$ for the components of a recursive statement.

Accumulator	**Monoid**	**Leaf contribution**	**Child check**	**Root check**
leaf counter κ	$(\mathbb{F}, +)$	1	$\kappa_X \in [1, 2^B)$	$\kappa \in [1, 2^B)$
challenge accumulator $\boldsymbol{c}$	$(G_{\mathcal{H}}, +)$	$h(\tau, \mathrm{com})$	—	$\chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c})$
bus partial sum s	$(G_{\mathcal{F}}, +)$	$\mathcal{F}_\chi(\mathrm{Msg}_\tau(w))$	—	$s + \mathcal{F}_\chi(\mathcal{G}(\boldsymbol{p})) = 0$

TABLE 1. The three accumulators of the recursion.

The counter is added in $\mathbb{F}$. Its child check is the following predicate on a node's type and counter, which every aggregation node applies to each non-null child and the Final Verifier applies to the root.

Definition 3.2 (Typing Predicate). For a type τ and a counter κ,

$$\mathsf{Typed}(\tau, \kappa) := (\tau \in [M] \wedge \kappa = 1) \vee (\tau = M + 1 \wedge 2 \le \kappa < 2^B).$$

The typing predicate places every counter in $[1, 2^B)$, and $A \cdot 2^B \le p$ then rules out wrap-around, so the field sums are integer sums and no node has 2^B or more leaves below it. The other two accumulators are checked only at the root. The value of the challenge accumulator at a node is $\mathcal{H}$ of the multiset of typed commitments below it, by the homomorphism of $\mathcal{H}$; the value of the bus partial sum is $\mathcal{F}_\chi$ of the summed message multisets below it, by the additivity of $\mathcal{F}$, so at the root it equals $\mathcal{F}_\chi(\mathcal{M}_L)$ for the collection L of all leaves. The Final Verifier adds the contribution $\mathcal{F}_\chi(\mathcal{G}(\boldsymbol{p}))$ of the global messages, which it computes itself from the public statement and the key, so its root check $s + \mathcal{F}_\chi(\mathcal{G}(\boldsymbol{p})) = 0$ is a fingerprint test on $\mathcal{M}_L + \mathcal{G}(\boldsymbol{p})$.

An update rule that is not associative and commutative would make the root value depend on the shape of the tree, which is not known when the key is derived (Section 3.1); a hash-tree accumulation $\boldsymbol{c} = H(\boldsymbol{c}_1, \boldsymbol{c}_2, \boldsymbol{c}_3)$, for instance, would force the prover to fix the tree before deriving χ. Writing every update as the monoid sum therefore loses no generality.

3.3. Recursive Relations.

We write a relation as $\mathcal{R} = \{(\mathbb{x}; \mathbb{w}) \mid \varphi(\mathbb{x}, \mathbb{w})\}$ with public statement $\mathbb{x}$, witness $\mathbb{w}$ and *defining predicate* φ; in the definitions below, φ is the conjunction of the clauses displayed to the right of the bar. Every statement of a recursive verifier below the Final Verifier carries the public statement $\boldsymbol{p} \in \mathcal{P}$, a key $\chi \in \mathcal{X}$, a recursive statement σ and a verification key vk. We proceed from the leaves to the root, naming after each relation the argument system that proves it, since the relation above uses its verifier.

Definition 3.3 (AIR Relation). For $\tau \in [M]$,

$$\mathcal{R}^{(\tau)}_{\mathrm{AIR}} = \left\{ (\boldsymbol{p}, \chi, s, \mathrm{com}); w \;\middle|\; \begin{array}{ll} & \mathrm{Local}_\tau(\boldsymbol{p}, w) \\ \wedge & s = \mathrm{BusCont}_\chi(\mathcal{I}_\tau, w) \\ \wedge & \mathsf{Commit}(w) = \mathrm{com} \end{array} \right\}$$

with $\boldsymbol{p} \in \mathcal{P}$, $\chi \in \mathcal{X}$, $s \in G_{\mathcal{F}}$ and $w = (\boldsymbol{T}, \boldsymbol{v}) \in \mathcal{W}_\tau$.

The relation is in *commit-and-prove* form: the trace commitment is part of the statement. A proof for $\mathcal{C}_\tau$ already contains this commitment, so $\mathbf{V}^{(\tau)}_{\mathrm{AIR}}$ simply reads it off the proof and compares it with the statement component; the point of exposing it is that the leaf verifier can then bind the challenge contribution to a value that is a function of the statement and witness rather than of an object that occurs in neither.

Let $\Pi^{(\tau)}_{\mathrm{AIR}}$ be an argument system for $\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$, one per type, with verifier $\mathbf{V}^{(\tau)}_{\mathrm{AIR}}$ and key $\mathsf{vk}^{(\tau)}_{\mathrm{AIR}}$.

A compressor (Section 3.1) re-proves the statement of another argument.

Definition 3.4 (Compressor Relation). For an argument Π_X with verifier $\mathbf{V}_X$ and key vk_X,

$$\mathcal{R}^{(X)}_{\mathrm{COMP}} := \{(\mathbb{x}; \pi) \mid \mathbf{V}(\mathsf{vk}_X, \mathbb{x}, \pi) = 1\}.$$

Let $\Pi^{(X)}_{\mathrm{COMP}}$ be an argument system for $\mathcal{R}^{(X)}_{\mathrm{COMP}}$; a proof of $\Pi^{(X)}_{\mathrm{COMP}}$ for $\mathbb{x}$ stands in for a proof of Π_X for $\mathbb{x}$. The setup fixes the set $T_{\mathrm{COMP}} \subseteq [M]$ of the types whose AIR proofs are compressed, and for $\tau \in T_{\mathrm{COMP}}$ we write $\Pi^{(\tau)}_{\mathrm{COMP}}$, with verifier $\mathbf{V}^{(\tau)}_{\mathrm{COMP}}$ and key $\mathsf{vk}^{(\tau)}_{\mathrm{COMP}}$, for the compressor of $\Pi^{(\tau)}_{\mathrm{AIR}}$, which hard-codes $\mathsf{vk}^{(\tau)}_{\mathrm{AIR}}$. A leaf verifier checks whichever of the two proofs its type produces.

Definition 3.5 (Segment Verifier). For $\tau \in [M]$, the *segment verifier* of type τ and its key are

$$\left(\mathbf{V}^{(\tau)}_{\mathrm{SEG}}, \mathsf{vk}^{(\tau)}_{\mathrm{SEG}}\right) := \begin{cases} \left(\mathbf{V}^{(\tau)}_{\mathrm{COMP}}, \mathsf{vk}^{(\tau)}_{\mathrm{COMP}}\right) & \text{if } \tau \in T_{\mathrm{COMP}}, \\ \left(\mathbf{V}^{(\tau)}_{\mathrm{AIR}}, \mathsf{vk}^{(\tau)}_{\mathrm{AIR}}\right) & \text{otherwise,} \end{cases}$$

and we write $\mathbf{V}^{(\tau)}_{\mathrm{SEG}}(\mathbb{x}, \pi)$ for $\mathbf{V}^{(\tau)}_{\mathrm{SEG}}\left(\mathsf{vk}^{(\tau)}_{\mathrm{SEG}}, \mathbb{x}, \pi\right)$. In both cases its statements are those of $\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$.

Definition 3.6 (Leaf-Verifier Relation). For $\tau \in [M]$,

$$\mathcal{R}^{(\tau)}_{\mathrm{LEAF}} = \left\{ (\boldsymbol{p}, \chi, \sigma, \mathsf{vk}); (\mathrm{com}, \pi) \;\middle|\; \begin{array}{ll} & \sigma = (\tau, 1, \boldsymbol{c}, s) \\ \wedge & \mathbf{V}^{(\tau)}_{\mathrm{SEG}}((\boldsymbol{p}, \chi, s, \mathrm{com}), \pi) = 1 \\ \wedge & \boldsymbol{c} = h(\tau, \mathrm{com}) \end{array} \right\}$$

The key vk carried in the statement is the propagated aggregation key, which is not used here.

In the terms of Section 3.2, a leaf verifier of type τ contributes 1, $h(\tau, \mathrm{com})$ and the bus contribution s certified by the AIR proof. Let $\Pi^{(\tau)}_{\mathrm{LEAF}}$ be an argument system for $\mathcal{R}^{(\tau)}_{\mathrm{LEAF}}$, with verifier $\mathbf{V}^{(\tau)}_{\mathrm{LEAF}}$ and key $\mathsf{vk}^{(\tau)}_{\mathrm{LEAF}}$; it hard-codes $\mathsf{vk}^{(\tau)}_{\mathrm{SEG}}$.

An aggregation node must verify children of every kind, leaf verifiers of any type and other aggregation nodes. It hard-codes the M leaf-verifier keys and selects among them by the child's type; for an aggregation child it uses a key that it does not hard-code but reads from its own statement. The reserved type 0 marks a null child.

Definition 3.7 (Recursive Dispatch). For a type $\tau \in [M+1]$ and a key vk, the *dispatched verifier* is

$$\mathbf{V}^{(\tau)}_{\mathrm{REC}}[\mathsf{vk}](\mathbb{x}, \pi) := \begin{cases} \mathbf{V}^{(\tau)}_{\mathrm{LEAF}}(\mathbb{x}, \pi) & \text{if } \tau \in [M], \\ \mathbf{V}(\mathsf{vk}, \mathbb{x}, \pi) & \text{if } \tau = M+1, \end{cases}$$

and $\mathbf{V}^{(\tau)}_{\mathrm{REC}}[\mathsf{vk}]$ rejects everything if $\tau \notin [M+1]$; in particular a null child ($\tau = 0$) never passes the dispatched verifier.

The dispatch is a function of the key vk carried in the statement.

Definition 3.8 (Aggregation Relation). Write $\varkappa_X := (\boldsymbol{p}, \chi, \sigma_X, \mathsf{vk})$ for the statement of the child in slot $X \in [A]$, $\sigma_X = (\tau_X, \kappa_X, \boldsymbol{c}_X, s_X)$ for its recursive statement, and $N := \{X \in [A] : \tau_X \neq 0\}$ for the set of non-null slots. Then

$$\mathcal{R}_{\mathrm{AGG}} = \left\{ (\boldsymbol{p}, \chi, \sigma, \mathsf{vk}); ((\sigma_X, \pi_X))_{X\in[A]} \;\middle|\; \begin{array}{ll} & \sigma = (M+1, \kappa, \boldsymbol{c}, s) \text{ and } |N| \geq 2 \\ \wedge & \mathsf{Typed}(\tau_X, \kappa_X) \text{ for } X \in N \\ \wedge & \mathbf{V}_{\mathrm{REC}}^{(\tau_X)}[\mathsf{vk}](\varkappa_X, \pi_X) = 1 \text{ for } X \in N \\ \wedge & \kappa = \sum_{X\in N} \kappa_X \text{ and } \boldsymbol{c} = \sum_{X\in N} \boldsymbol{c}_X \text{ and } s = \sum_{X\in N} s_X \end{array} \right\}$$

Null slots carry no constraint: their proofs are not verified and their recursive statements do not enter the sums.

Two invariants of this relation are used repeatedly. First, the *propagation invariant*: every non-null child is verified on a statement that carries the parent's own $(\boldsymbol{p}, \chi, \mathsf{vk})$, so every node of a valid tree carries the root's public statement, fingerprint key and verification key; in particular an aggregation child is verified under $\mathbf{V}(\mathsf{vk}, \cdot, \cdot)$ for the very vk the parent declares. Since the cached-column commitments travel in $\boldsymbol{p}$ (Definition 2.2), the binding of $\mathsf{Com}_{\mathrm{cach}}$ then makes the cached columns of all leaves of one type equal. Second, the *counter invariant*: Typed places the counters of the non-null children in $[1, 2^B)$, there are at least two and at most A of them, and $A \cdot 2^B \leq p$ makes $\kappa = \sum_{X\in N} \kappa_X$ an identity of integers; hence every non-null child has $\kappa_X \leq \kappa - 1$.

Let Π_{AGG} be an argument system for $\mathcal{R}_{\mathrm{AGG}}$, with verifier $\mathbf{V}_{\mathrm{AGG}}$ and key $\mathsf{vk}_{\mathrm{AGG}}$. There is no circularity: $\mathcal{R}_{\mathrm{AGG}}$ is one ordinary relation over statements that contain an arbitrary string vk, the key $\mathsf{vk}_{\mathrm{AGG}}$ of the argument system for that relation is produced afterwards at setup, and the Final relation below fixes $\mathsf{vk} := \mathsf{vk}_{\mathrm{AGG}}$, at which point $\mathbf{V}_{\mathrm{REC}}^{(M+1)}[\mathsf{vk}_{\mathrm{AGG}}] = \mathbf{V}_{\mathrm{AGG}}$.

Definition 3.9 (Final Relation). Fix the aggregation key $\mathsf{vk}_{\mathrm{AGG}}$ as a parameter. Then, with $\boldsymbol{p} := (\boldsymbol{p}_{\mathrm{ext}}, \boldsymbol{p}_{\mathrm{pv}})$,

$$\mathcal{R}_{\mathrm{FINAL}} = \left\{ \boldsymbol{p}_{\mathrm{ext}}; (\boldsymbol{p}_{\mathrm{pv}}, \chi, \sigma, \pi) \;\middle|\; \begin{array}{ll} & \sigma = (\tau, \kappa, \boldsymbol{c}, s) \text{ and } \mathsf{Typed}(\tau, \kappa) \\ \wedge & \mathbf{V}_{\mathrm{REC}}^{(\tau)}[\mathsf{vk}_{\mathrm{AGG}}]((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}_{\mathrm{AGG}}), \pi) = 1 \\ \wedge & s + \mathcal{F}_{\chi}(\mathcal{G}(\boldsymbol{p})) = 0 \\ \wedge & \chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c}) \end{array} \right\}$$

Only the external statement $\boldsymbol{p}_{\mathrm{ext}}$ survives at the root: the proof values, the fingerprint key, the recursive statement and the child proof are witnesses. The typing predicate excludes a null child at the root, which would otherwise leave $\boldsymbol{c}$ and s unconstrained. Dispatching on τ lets the Final Verifier accept a leaf-verifier proof directly when the execution fits in a single AIR instance.

Let Π_{FINAL} be an argument system for $\mathcal{R}_{\mathrm{FINAL}}$, with verifier $\mathbf{V}_{\mathrm{FINAL}}$ and key $\mathsf{vk}_{\mathrm{FINAL}}$; this is the verifier run by the external party. The verification keys involved play different roles. Table 2 records them.

Key	Selected by	Hard-coded in
$\mathsf{vk}^{(\tau)}_{\mathrm{AIR}}$	the type τ of the compressor	$\Pi^{(\tau)}_{\mathrm{COMP}}$, for $\tau \in T_{\mathrm{COMP}}$
$\mathsf{vk}^{(\tau)}_{\mathrm{SEG}}$	the type τ of the leaf verifier	$\Pi^{(\tau)}_{\mathrm{LEAF}}$
$\mathsf{vk}^{(\tau)}_{\mathrm{LEAF}}$	the child type $\tau_X \in [M]$ read from the child statement	Π_{AGG} (all M keys)
$\mathsf{vk}_{\mathrm{AGG}}$	the child type $\tau_X = M+1$; the key itself arrives as the public vk	Π_{FINAL} only
$\mathsf{vk}_{\mathrm{FINAL}}$	—	the external verifier

TABLE 2. Verification-key ledger. The type τ selects which hard-coded key an aggregation node uses for a child; the propagated public vk is the aggregation node's own key, which no aggregation node can hard-code and which the Final Verifier fixes.

An execution with n_{leaves} leaves has between $\lceil (n_{\mathrm{leaves}}-1)/(A-1) \rceil$ and $n_{\mathrm{leaves}}-1$ aggregation nodes, since each has between two and A non-null children; with a single leaf it has none, and the Final Verifier verifies the leaf verifier directly. The whitepaper lists every proof-system call of a ZisK proof, with the number of its instances, the AIRs of T_{COMP} and the parameters of every argument [Zis26].

3.4. **ZisK Instantiation.**

The security analysis is stated for the following choices, which fix the carriers and the maps of Section 2.2. Four of them are built from one *underlying hash function*, and the analysis depends on that function only through the random-oracle model of Definition 2.15, never through its internals. ZisK currently instantiates it with Poseidon2 [GKS23], whose arithmetization-friendly design makes the recursive verifiers cheap; a general-purpose hash such as BLAKE3 [O'C+20] or SHA-2 [Nat15] is an equally valid instantiation, and switching among them changes none of the statements below. The only requirements are those the analysis names where it uses them: collision resistance of the digest, with at least 2λ bits (Section 17), and the random-oracle idealization.

Fields. $\mathbb{F}$ is the Goldilocks field, of order $p = 2^{64} - 2^{32} + 1$, and $\mathbb{K} = \mathbb{F}[X]/(X^3 - X - 1)$ its cubic extension, with $|\mathbb{K}| = p^3 \approx 2^{192}$.

Public statement. In ZisK, $k = 68$ and $k' = 8$. The external statement $\boldsymbol{p}_{\mathrm{ext}}$ consists of the program commitment $\mathrm{root}_{\mathrm{ROM}}$, the root of the Rom chip's cached columns, at the $m = 4$ coordinates that ψ designates (Definition 2.2), and of the 32 public outputs, each as two 32-bit limbs; the proof values $\boldsymbol{p}_{\mathrm{pv}}$ are boolean flags that switch on the genesis anchors of the optional chips. The program commitment is bound by Definition 2.6 and by DeriveKey, which absorbs it; the public outputs are bound by the global messages $\mathcal{G}(\boldsymbol{p})$, which only a `pubout` of the Main chip balances. The input, the hints and the intermediate states of the execution are not part of $\boldsymbol{p}$. The external verifier also holds a context that is not part of $\boldsymbol{p}$, the key $\mathsf{vk}_{\mathrm{FINAL}}$, which fixes the fixed columns of every chip and every key hard-coded in the recursive verifiers (Table 2). To check a claim about a given program, it compares $\mathrm{root}_{\mathrm{ROM}}$ with the commitment of that program.

Bus fingerprint. The key space is $\mathcal{X} = \mathbb{K}^2$, a key being a *challenge pair* $\chi = (\alpha, \gamma)$, and the values lie in $G_{\mathcal{F}} = \mathbb{K}$. The fingerprint is LogUp [Hab22]: for $\chi = (\alpha, \gamma)$ the message (b, v) is *compressed* to $\hat{e}_\gamma(b, v) := b + \sum_{j=1}^{\ell_b} \gamma^j v_j \in \mathbb{K}$ and

$$\mathcal{F}^{\mathrm{LogUp}}_{\chi}(D) := \sum_{(b,v) \in \mathrm{supp}\ D} \frac{D(b,v)}{\alpha + \hat{e}_\gamma(b,v)},$$

with $\mathcal{F}_\chi^{\mathrm{LogUp}}(D) := \perp$ if some denominator vanishes. It admits every nonzero multiset, and Lemma 7.1 shows that it is statistically sound with $\varepsilon_{\mathcal{F}}(\Lambda) = \left(\ell \cdot \binom{\Lambda}{2} + \max(\Lambda - 1, 0)\right)/|\mathbb{K}|$. The AIR argument does not evaluate this closed form. It constrains a running sum over the rows of the trace, adding $m_\iota(w,r)/\big(\alpha + \hat{e}_\gamma(b_\iota, \boldsymbol{e}_\iota(w,r))\big)$ at every row through an explicitly constrained inverse, and exposes the final value s. Grouping the row terms by message shows that the exposed value equals $\mathcal{F}_\chi^{\mathrm{LogUp}}(\mathrm{Msg}_\tau(w))$ whenever every row denominator is nonzero. At a pole the constraints do not determine s: a row of zero multiplicity whose denominator vanishes satisfies its fraction constraint for every value of the fraction column. The poles of a witness are at most $(N_\tau + 1)|\mathcal{I}_\tau|$ values of α for each γ (Remark 4.1), fixed by the witness, and the key is derived after the commitment to the witness; the probability of hitting one is charged in Lemma 5.8.

Multiset hash. The accumulator group is $G_{\mathcal{H}} = \mathbb{Z}_p^n$ with $n = 368$, and h and $\mathcal{H}$ are LtHash [Zis25, Cla+03]: h absorbs the typed commitment into a sponge of the underlying hash function and expands it, by 22 further permutations, into a pseudorandom vector of $\mathbb{Z}_p^n$ (the 23 states of width 16). The type enters h as the root of the type's fixed columns, which identifies the type provided that no two types have the same fixed columns, and $\mathcal{H}$ adds these vectors with multiplicity. Its collision resistance on multisets of cardinality below 2^B rests on the hardness of the short integer solution problem [Ajt96]; the analysis assumes the stronger preimage awareness (Definition 2.14), which is the second idealization of Remark 2.1. ZisK takes $B = 20$, so that a proof has fewer than 2^{20} leaves, and aggregation nodes with $A = 3$ slots; the cardinality bound is what the leaf counter enforces, and $A \cdot 2^B \leq p$ holds with room to spare.

Key derivation. DeriveKey hashes the public statement $\boldsymbol{p}$ —the external statement and the proof values— and the accumulated vector with the underlying hash function into a *global challenge* in $\mathbb{K}$, from which a fresh sponge squeezes the pair (α, γ). The analysis treats the pair as a single random-oracle output (Definition 2.15), which suppresses a term $(Q+1)/|\mathbb{K}|$ for the global challenge coinciding with an input already queried to the sponge that squeezes (α, γ).

Trace commitment. Com is a Merkle tree over the underlying hash function, so it is straight-line extractable in the random-oracle model (Definition 2.13).

Cached-column commitment. $\mathsf{Com}_{\mathrm{cach}}$ is likewise a Merkle tree over the underlying hash function. The program ROM is the cached column group of the Rom chip, and its root is one of the publics, so a proof is bound to a program through $\boldsymbol{p}$. The leaf commitment com covers the witness columns and the AIR values of the first stage, not the cached columns; this is sound because DeriveKey absorbs $\boldsymbol{p}$, and with it the cached-column commitment, so the cached columns are fixed before the key χ is derived, exactly as the committed witnesses are.

Frequent-operation caches. The caches of precomputed operations are tables that prove, on the Operation Bus, every operation they list. Their columns are fixed, generated at setup independently of the program, so they are bound by the verification key like every fixed column (Table 2). That every row of the table is a correct operation is a property of the setup, which this analysis takes as an obligation (Section 17).

Part I: The Proving System

4. The ZisK Proving Protocol

Every AIR of ZisK is proved with one proving system: a univariate STARK [Ben+18a] in the DEEP-ALI style, with FRI [Ben+18b] as low-degree test, Merkle trees as vector commitments, and the BCS transformation [BCS16] to make it non-interactive. This section describes the full deployed protocol, the prover and verifier used by ZisK, so that the soundness analysis refers to the actual protocol rather than to an idealized one.

We describe the protocol as an interactive oracle proof, in which the prover's committed functions are oracles and the challenges are uniformly random, and prove its round-by-round knowledge soundness. Its compilation into a non-interactive argument, with ZisK's Merkle trees and Fiat–Shamir transcript, is the subject of Section 5.

Notation. Throughout, $\mathbb{F}$ is the Goldilocks field, of order $p = 2^{64} - 2^{32} + 1$, and $\mathbb{K} = \mathbb{F}[X]/(X^3 - X - 1)$ its cubic extension. We identify $\mathbb{F}$ with the constant polynomials of $\mathbb{K}$ and write elements of $\mathbb{K}$ as $c = c_0 + c_1 X + c_2 X^2$ with $(c_0, c_1, c_2) \in \mathbb{F}^3$. Since 2^{32} divides $p - 1$, for every $b \le 32$ the group $\mathbb{F}^\times$ has a unique subgroup H_b of order 2^b. We fix generators ω_b of H_b with $\omega_{b+1}^2 = \omega_b$, and the *coset shift* $\sigma := 7$, a generator of $\mathbb{F}^\times$. For an integer $d \ge 0$, we write $\mathbb{F}^{<d}[X]$ and $\mathbb{K}^{<d}[X]$ for the sets of polynomials over $\mathbb{F}$ and over $\mathbb{K}$ of degree less than d.

An AIR $\mathcal{C}$ is proved at a trace height $N = 2^n$ with a *blowup* 2^k, $k \ge 1$, the *rate* being $\rho := 2^{-k}$. We write $n_e := n + k$, and define

$$H := H_n = \{\omega^r : r \in [0, N)\}, \quad \omega := \omega_n, \qquad D := \sigma H_{n_e} = \left\{x_i := \sigma \omega_{n_e}^i : i \in [0, 2^{n_e})\right\}.$$

H is the *trace domain*: row r of the trace corresponds to the point ω^r. D is the *evaluation domain*; it is disjoint from H because $\sigma \notin H_{n_e}$. For a column $T_j : [0, N) \to \mathbb{K}$ of a trace, its *interpolant* $\hat{T}_j$ is the unique polynomial in $\mathbb{K}^{<N}[X]$ with $\hat{T}_j(\omega^r) = T_j(r)$ for every r. For an integer o, the polynomial $\hat{T}_j(\omega^o X)$ interpolates the column shifted by o rows, $r \mapsto T_j(r + o \bmod N)$. The *vanishing polynomial* of H is $Z_H(X) := X^N - 1$, and the *first-row selector* $L_1 \in \mathbb{F}^{<N}[X]$ is the polynomial with $L_1(1) = 1$ and $L_1(\omega^r) = 0$ for $r \in (0, N)$.

4.1. Reed–Solomon Codes.

This section collects the facts about Reed–Solomon codes on which the analysis of the protocol rests: list decoding up to the Johnson bound, mutual correlated agreement, and folding.

Let $\mathcal{L} \subset \mathbb{F}^\times$ be a coset of a subgroup of order a power of two. For a degree bound d, the *Reed–Solomon code* $\mathsf{RS}[\mathbb{K}, \mathcal{L}, d]$ is the set of functions $\mathcal{L} \to \mathbb{K}$ that are evaluations of a polynomial in $\mathbb{K}^{<d}[X]$, that is:

$$\mathsf{RS}[\mathbb{K}, \mathcal{L}, d] := \left\{f : \mathcal{L} \to \mathbb{K} : \exists p \in \mathbb{K}^{<d}[X], f(x) = p(x)\ \forall x \in \mathcal{L}\right\}$$

and $d/|\mathcal{L}|$ is its rate. For a function $f : \mathcal{L} \to \mathbb{K}$ and a set $S \subset \mathcal{L}$, $f[S]$ is the restriction of f to S, so that $f[S] = g[S]$ means that f and g agree on S; and for an integer m, $\mathcal{L}^m := \{x^m : x \in \mathcal{L}\}$ and $S^m := \{x^m : x \in S\}$. For functions $f, g : \mathcal{L} \to \mathbb{K}$, $\Delta(f, g)$ is the fraction of points of $\mathcal{L}$ where they differ, and for a set $\mathcal{S}$ of functions, $\Delta(f, \mathcal{S}) := \min_{g \in \mathcal{S}} \Delta(f, g)$.

For a code $\mathcal{C}$ of functions $\mathcal{L} \to \mathbb{K}$ and $t \ge 1$, the *t-interleaved code* $\mathcal{C}^t$ is the set of t-tuples $(u_1, ..., u_t)$ of codewords, viewed as functions $\mathcal{L} \to \mathbb{K}^t$, $x \mapsto (u_1(x), ..., u_t(x))$. Two tuples agree at a point only if all their coordinates agree there, so a tuple of functions $(f_1, ..., f_t)$ is δ-close to $\mathcal{C}^t$ if and only if there is a set $S \subset \mathcal{L}$ with $|S| \ge (1 - \delta)|\mathcal{L}|$ on which every f_j agrees with a codeword: a *common* agreement set. The analysis decodes the committed columns in an interleaved code, because they are opened together, one row per query.

Definition 4.1 (List Decoding). For a code $\mathcal{C} \subset \Sigma^{\mathcal{L}}$, a function $f : \mathcal{L} \to \Sigma$ and $\delta \in [0,1]$, let

$$\Lambda(\mathcal{C}, f, \delta) \coloneqq \{u \in \mathcal{C} : \Delta(f, u) \leq \delta\}.$$

The code is (δ, ℓ)-*list decodable* if $|\Lambda(\mathcal{C}, f, \delta)| \leq \ell$ for every f. Every code of minimum relative distance $\delta_{\mathcal{C}}$ is $(\delta, 1)$-list decodable for $\delta < \delta_{\mathcal{C}}/2$, the *unique-decoding* regime.

The minimum relative distance of $\mathsf{RS}[\mathbb{K}, \mathcal{L}, d]$ is $1 - (d-1)/|\mathcal{L}|$, and up to the Johnson bound its lists remain small, also when the code is interleaved [Arn+25].

Theorem 4.2 (Johnson Bound). *For every $\eta \in (0, 1 - \sqrt{\rho})$ and every $t \geq 1$, the code $\mathsf{RS}[\mathbb{K}, \mathcal{L}, d]^t$ is $(1 - \sqrt{\rho} - \eta, 1/(2\eta\sqrt{\rho}))$-list decodable.*

A random linear combination of functions that are far from a code is, with high probability, itself far from the code.

Definition 4.3 (Proximity Generator, Mutual Correlated Agreement). A *proximity generator* is a randomized algorithm Gen that, on input a length $m \geq 1$, outputs coefficients $(r_1, ..., r_m) \in \mathbb{K}^m$ for a linear combination of m functions. Let $\mathcal{C} \coloneqq \mathsf{RS}[\mathbb{K}, \mathcal{L}, d]$. The generator Gen has *mutual correlated agreement* for $\mathcal{C}$ with error $\varepsilon^{\mathrm{mca}}$ if for every $f_1, ..., f_m : \mathcal{L} \to \mathbb{K}$ and every $\theta \in [0,1]$,

$$\Pr_{(r_1,...,r_m) \leftarrow \mathsf{Gen}(m)} \left[\begin{array}{c} |S| \geq (1-\theta) \cdot |\mathcal{L}| \\ \exists S \subset \mathcal{L} \text{ s.t. } \wedge\, \exists u \in \mathcal{C}, u[S] = \sum_{j \in [m]} r_j \cdot f_j[S] \\ \wedge\, \exists i \in [m], \forall u' \in \mathcal{C}, u'[S] \neq f_i[S] \end{array} \right] \leq \varepsilon^{\mathrm{mca}}(\mathcal{C}, m, \theta).$$

We call $(r_1, ..., r_m)$ *bad* for $(f_1, ..., f_m)$ at θ if such a set S exists.

The usual correlated agreement guarantees that, for good coefficients, the combined functions agree with codewords on *some* common set as large as the agreement set of their combination. Mutual correlated agreement guarantees it on *every* such set, and it is this form that lets the analysis of FRI track where agreement holds. ZisK combines functions with powers of a single challenge.

Lemma 4.4 ([Ben+25]). *Let $\mathcal{C} \coloneqq \mathsf{RS}[\mathbb{K}, \mathcal{L}, d]$ have rate ρ. The* powers generator*, which samples $r \leftarrow \mathbb{K}$ and outputs*

$$\mathsf{Powers}(m; r) \coloneqq (1, r, ..., r^{m-1}) \in \mathbb{K}^m,$$

has, for every $m \geq 1$, mutual correlated agreement for $\mathcal{C}$ with error

$$\varepsilon^{\mathrm{mca}}(\mathcal{C}, m, \theta) \coloneqq \begin{cases} \frac{m-1}{|\mathbb{K}|} \cdot \left(\frac{2(m_J + 1/2)^5 + 3(m_J + 1/2) \cdot \theta \cdot \rho}{3\rho^{3/2}} \cdot |\mathcal{L}| + \frac{m_J + 1/2}{\sqrt{\rho}} \right), & \text{if } \theta < 1 - \sqrt{\rho}, \\ 1, & \text{otherwise} \end{cases}$$

where $m_J \coloneqq \max\left(\left\lceil \frac{\sqrt{\rho}}{(1 - \sqrt{\rho} - \theta)} \right\rceil, 3 \right)$. We call r bad *for $(f_1, ..., f_m)$ at θ if $\mathsf{Powers}(m; r)$ is.*

Folding. FRI folds a function along the blocks of a power map.

Definition 4.5 (Fold). Let m be an integer dividing $|\mathcal{L}|$. For $y \in \mathcal{L}^m$, the *block* of y is $\mathsf{Block}(\mathcal{L}, m, y) \coloneqq \{x \in \mathcal{L} : x^m = y\}$, a set of m points; we write $\mathsf{Block}(y)$ when $\mathcal{L}$ and m are clear. For $f : \mathcal{L} \to \mathbb{K}$ and $y \in \mathcal{L}^m$, let $\hat{p}_{f,y} \in \mathbb{K}^{<m}[X]$ be the polynomial with $\hat{p}_{f,y}(x) = f(x)$ for every $x \in \mathsf{Block}(y)$. For $\alpha \in \mathbb{K}$, the *fold* of f is

$$\mathsf{Fold}(f, m, \alpha) : \mathcal{L}^m \to \mathbb{K}, \quad \mathsf{Fold}(f, m, \alpha)(y) \coloneqq \hat{p}_{f,y}(\alpha).$$

Writing $\hat{p}_{f,y}(X) = \sum_{i \in [m]} f_i(y) X^{i-1}$ defines the *coefficient functions* $f_1, ..., f_m : \mathcal{L}^m \to \mathbb{K}$ of f, and then

$$\mathsf{Fold}(f, m, \alpha) = \sum_{i \in [m]} \alpha^{i-1} \cdot f_i,$$

the combination of the coefficient functions by $\mathsf{Powers}(m; \alpha)$.

Lemma 4.6 ([GMW25]). *Let m divide d and $|\mathcal{L}|$, and let $\mathcal{C} := \mathsf{RS}[\mathbb{K}, \mathcal{L}, d]$ and $\mathcal{C}' := \mathsf{RS}[\mathbb{K}, \mathcal{L}^m, d/m]$. Then:*

1. *If $f \in \mathcal{C}$, then $\mathsf{Fold}(f, m, \alpha) \in \mathcal{C}'$ for every $\alpha \in \mathbb{K}$.*
2. *If Powers has mutual correlated agreement for $\mathcal{C}'$ with error $\varepsilon^{\mathrm{mca}}$, then for every $f : \mathcal{L} \to \mathbb{K}$ and every $\theta \in [0, 1]$,*

$$\Pr_{r \leftarrow \mathbb{K}} \left[\begin{array}{c} |S^m| \geq (1-\theta) \cdot |\mathcal{L}^m| \\ \exists S \subset \mathcal{L} \text{ s.t. } \wedge\, \exists u' \in \mathcal{C}', \mathsf{Fold}(f, m, r)[S^m] = u'[S^m] \\ \wedge\, \forall u \in \mathcal{C}, u[S] \neq f[S] \end{array} \right] \leq \varepsilon^{\mathrm{mca}}(\mathcal{C}', m, \theta).$$

We call r bad *for f at θ if such a set S exists.*

4.2. **Interactive Oracle Proofs.**

Interactive oracle proofs [BCS16] combine interactive proofs with probabilistically checkable proofs: in every round the prover sends a message that the verifier may read only at the positions of its choice. We follow [Arn+25] and define them as a special case of interactive oracle proofs of proximity, for *public-coin* protocols, in which the verifier's messages are uniformly random and independent of what it has seen. For a ternary relation $\mathcal{R} = \{(\mathbb{x}, \mathbb{y}, \mathbb{w})\}$, the *instance* $\mathbb{x}$ is read in full by the verifier, the *input* $\mathbb{y}$ only as an oracle, and the *witness* $\mathbb{w}$ is known to the prover; $L(\mathcal{R}) := \{(\mathbb{x}, \mathbb{y}) : \exists \mathbb{w}, (\mathbb{x}, \mathbb{y}, \mathbb{w}) \in \mathcal{R}\}$ is the language of $\mathcal{R}$.

Definition 4.7 (Interactive Oracle Proof of Proximity). A *k-round public-coin IOP of proximity* (IOPP) for a ternary relation $\mathcal{R}$ is a pair $(\mathbf{P}, \mathbf{V})$ of an interactive algorithm $\mathbf{P}$, the prover, and an interactive oracle algorithm $\mathbf{V}$, the verifier. The prover receives $(\mathbb{x}, \mathbb{y}, \mathbb{w})$; the verifier receives $\mathbb{x}$ and oracle access to $\mathbb{y}$. In round $i \in [k]$ the prover sends a message π_i, and the verifier replies with a uniformly random message α_i. A message π_i consists of *oracles*, functions that the verifier may query at points of its choice, and of plain values, which it reads in full. After round k the verifier queries $\mathbb{y}$ and the oracles, its queries being part of α_k, and outputs a bit $\mathbf{V}^{\mathbb{y}, \pi_1, \ldots, \pi_k}(\mathbb{x}, \alpha_1, \ldots, \alpha_k)$. The IOPP has *perfect completeness* if, for every $(\mathbb{x}, \mathbb{y}, \mathbb{w}) \in \mathcal{R}$,

$$\Pr_{\alpha_1, \ldots, \alpha_k} \left[\mathbf{V}^{\mathbb{y}, \pi_1, \ldots, \pi_k}(\mathbb{x}, \alpha_1, \ldots, \alpha_k) = 1 \;\middle|\; \begin{array}{c} \pi_1 \leftarrow \mathbf{P}(\mathbb{x}, \mathbb{y}, \mathbb{w}) \\ \vdots \\ \pi_k \leftarrow \mathbf{P}(\mathbb{x}, \mathbb{y}, \mathbb{w}, \alpha_1, \ldots, \alpha_{k-1}) \end{array} \right] = 1.$$

An *interactive oracle proof* (IOP) is an IOPP whose input is empty, $\mathbb{y} = \bot$, for a relation $\mathcal{R} = \{(\mathbb{x}, \bot, \mathbb{w})\}$; we then omit $\bot$ and write $(\mathbb{x}; \mathbb{w}) \in \mathcal{R}$, $L(\mathcal{R}) = \{\mathbb{x} : \exists \mathbb{w}, (\mathbb{x}; \mathbb{w}) \in \mathcal{R}\}$ and $\mathbf{V}^{\pi_1, \ldots, \pi_k}(\mathbb{x}, \alpha_1, \ldots, \alpha_k)$.

A *transcript* $\mathsf{tr} = (\pi_1, \alpha_1, \ldots, \pi_i, \alpha_i)$ is the sequence of the messages exchanged up to some round, and $\mathsf{tr} \parallel \alpha$ its extension by a message. Soundness is measured round by round [Can+19], through a function that assigns a state to each partial transcript. In the knowledge version that we use [Blo+23, Arn+25], state 1 means that a witness can be extracted from the transcript, and a transcript in state 0 is flipped to state 1 by the next challenge only with small probability, unless it already determines a witness.

Definition 4.8 (State Function). Let $(\mathbf{P}, \mathbf{V})$ be a k-round IOPP for $\mathcal{R}$. A *state function* is a function $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}) \in \{0, 1\}$ with the following properties.

1. *Empty transcript.* $\mathsf{State}(\mathbb{x}, \mathbb{y}, \varnothing) = 0$ for every $(\mathbb{x}, \mathbb{y})$.

2. *Prover moves.* If tr ends with a verifier message, or is empty, then $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr} \parallel \pi) = \mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr})$ for every prover message π.
3. *Full transcript.* If tr is a full transcript and $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}) = 0$, then $\mathbf{V}$ rejects in the interaction described by tr, for every choice of its queries.
4. *Default.* If $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}) = 1$, then $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}') = 1$ for every extension tr' of tr.

The query phase of the verifier is folded into the last round: its queries are part of α_k, and $\mathbf{V}$'s decision is a function of the full transcript and of the answers. The default property, from [GMW25], says that a prover who gets lucky in one round has won, whatever happens later; it makes every prefix of a transcript in state 0 also be in state 0.

Definition 4.9 (Round-by-Round Knowledge Soundness). A k-round IOPP $(\mathbf{P}, \mathbf{V})$ for $\mathcal{R}$ has *round-by-round knowledge soundness* with errors $(\varepsilon_1, ..., \varepsilon_k)$ and extraction time et if it has a state function and a deterministic algorithm E, the *extractor*, running in time et, such that for every $(\mathbb{x}, \mathbb{y})$, every $i \in [k]$ and every partial transcript $\mathsf{tr} = (\pi_1, \alpha_1, ..., \alpha_{i-1}, \pi_i)$ with $\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr}) = 0$,

$$\Pr_{\alpha_i}[\mathsf{State}(\mathbb{x}, \mathbb{y}, \mathsf{tr} \parallel \alpha_i) = 1] > \varepsilon_i \quad \Longrightarrow \quad (\mathbb{x}, \mathbb{y}, E(\mathbb{x}, \mathbb{y}, \mathsf{tr})) \in \mathcal{R}.$$

The empty transcript is in state 0 even when $(\mathbb{x}, \mathbb{y}) \in L(\mathcal{R})$: the state records whether a witness can be extracted from the transcript, not whether one exists. For $(\mathbb{x}, \mathbb{y}) \notin L(\mathcal{R})$ the extractor cannot succeed, so every round flips the state with probability at most ε_i. A prover that convinces the verifier ends in state 1, by the full-transcript property, so some round flipped the state, and a union bound gives soundness error $\sum_i \varepsilon_i$; the compiled argument pays the maximum $\max_i \varepsilon_i$ instead, per query to the random oracle (Section 5.3). For an IOP we omit $\mathbb{y} = \perp$ and write $\mathsf{State}(\mathbb{x}, \mathsf{tr})$ and $E(\mathbb{x}, \mathsf{tr})$.

4.3. Constraint Polynomials.

The columns of a witness $w = (\boldsymbol{T}, \boldsymbol{v})$ follow the classes of Definition 2.2. The fixed columns are given by the setup, and the cached columns by their commitment in the publics. The proving system computes the witness columns in two *stages*. The *first-stage* columns are those of the AIR, over $\mathbb{F}$ (Definition 2.1), and do not depend on any challenge. The *second-stage* columns are the bus columns of Section 4.3.1, which depend on the challenges (α, γ) sampled after the first stage is committed. The second stage is over $\mathbb{K}$: its columns, its AIR values $\boldsymbol{a}^{(2)}$ and the *airgroup value* s of the instance, its bus contribution, take values in $\mathbb{K}$, and its constraints are polynomials over $\mathbb{K}$ whose coefficients include the challenges. We call the AIR with both stages the *expanded AIR*. Its values are the public statement $\boldsymbol{p} = (\boldsymbol{p}_{\text{ext}}, \boldsymbol{p}_{\text{pv}})$ of Definition 2.3, the AIR values $\boldsymbol{a} = (\boldsymbol{a}^{(1)}, \boldsymbol{a}^{(2)})$, split by stage, and s.

Definition 4.10 (Constraint Polynomial). Let $C \in \mathbb{K}\left[\left(X_j^{(o)}\right)_{j\in[W],o\in O}, (Y_l)_{l\in[V]}\right]$ be a constraint, let $\hat{T}_1, ..., \hat{T}_W \in \mathbb{K}^{<N}[X]$ be the interpolants of the columns of a trace $\boldsymbol{T}$, and let $\boldsymbol{v} \in \mathbb{K}^V$ be values. The *constraint polynomial* of C is

$$\hat{C}(X) \coloneqq C\left(\left(\hat{T}_j(\omega^o X)\right)_{j\in[W],o\in O}, \boldsymbol{v}\right) \in \mathbb{K}[X].$$

At $X = \omega^r$ the constraint polynomial evaluates C at row r, so the witness $(\boldsymbol{T}, \boldsymbol{v})$ satisfies C at every row if and only if $\hat{C}$ vanishes on H, that is, if and only if Z_H divides $\hat{C}$. Moreover $\deg\left(\hat{C}\right) \leq \deg(C) \cdot (N-1)$, where $\deg(C)$ is the total degree of C in the column variables. The setup (Definition 4.13) bounds this degree by $d_{\max} \coloneqq 2^k + 1$: a constraint of larger degree is rewritten by introducing *intermediate columns* I, witness columns each defined by a constraint $I - E = 0$ for a subexpression

E of degree at most $d_{\max}$, and computed in the first stage in which E can be computed. After this rewriting the AIR has constraints $C_0, ..., C_{n_C-1}$, all of degree at most $d_{\max}$, and we write $d \le d_{\max}$ for the largest of their degrees.

4.3.1. *The Bus Columns.*

Let $\chi = (\alpha, \gamma) \in \mathbb{K}^2$ be the key of the bus argument, with $\hat{e}_\gamma$ the compression of Section 3.4, and for an interaction $\iota = (b_\iota, m_\iota, \boldsymbol{e}_\iota)$ and a row r write

$$\delta_\iota(r) \coloneqq \alpha + \hat{e}_\gamma(b_\iota, \boldsymbol{e}_\iota(r)).$$

The interactions of the AIR are partitioned into *row interactions*, whose multiplicity and message read the trace, and *direct interactions*, whose multiplicity and message read only scalar values. The row interactions are further partitioned into groups $G_1, ..., G_h$, chosen at setup so that the constraints below have degree at most $d_{\max}$. As in Definition 4.10, substituting the interpolants of the columns into m_ι and δ_ι gives polynomials $\hat{m}_\iota, \hat{\delta}_\iota \in \mathbb{K}[X]$ with $\hat{m}_\iota(\omega^r) = m_\iota(r)$ and $\hat{\delta}_\iota(\omega^r) = \delta_\iota(r)$. The key enters the constraints below as coefficients, which is why the constraint polynomials have coefficients in $\mathbb{K}$.

Definition 4.11 (Bus Columns). The second stage of an AIR proof commits to the following columns, with the following constraints, each of which must vanish on H:

1. For each group G_t, the *fraction column* $\mathrm{im}_t(r) \coloneqq \sum_{\iota \in G_t} m_\iota(r)/\delta_\iota(r)$, with the constraint polynomial:
$$\widehat{\mathrm{im}}_t(X) \cdot \prod_{\iota \in G_t} \hat{\delta}_\iota(X) - \sum_{\iota \in G_t} \hat{m}_\iota(X) \prod_{\iota' \in G_t \setminus \{\iota\}} \hat{\delta}_{\iota'}(X).$$
2. The *running sum column* $\mathrm{S}(r) \coloneqq \sum_{r' \le r} \sum_{t=1}^{h} \mathrm{im}_t(r')$, with the constraint polynomial:
$$\hat{\mathrm{S}}(X) - \hat{\mathrm{S}}(\omega^{-1}X) \cdot (1 - L_1(X)) - \sum_{t=1}^{h} \widehat{\mathrm{im}}_t(X).$$

The fractions of the direct interactions are added once. Some of them are committed as second-stage AIR values $\mathrm{im}_u^{\mathrm{dir}}$, constrained like the fraction columns; the remaining ones are combined into a single fraction ν/Δ computed in place from the AIR values. The *bus contribution* of the proof is an *airgroup value* $s \in \mathbb{K}$ constrained at the last row by the polynomial:

$$L_1(\omega X) \cdot \left(\left(s - \hat{\mathrm{S}}(X) - \sum_u \mathrm{im}_u^{\mathrm{dir}} \right) \cdot \Delta - \nu \right).$$

Lemma 4.12 (Bus Contribution). *Let w be a witness and χ a key such that $\delta_\iota(r) \neq 0$ for every row interaction and every row, and such that the denominators of the direct interactions do not vanish. If the bus columns and s satisfy the constraints of Definition 4.11, then*

$$s = \sum_{r=0}^{N-1} \sum_{\text{row } \iota} \frac{m_\iota(r)}{\delta_\iota(r)} + \sum_{\text{direct } \iota} \frac{m_\iota}{\delta_\iota} = \mathrm{BusCont}_\chi(\mathcal{I}, w),$$

where, in the notation of Definition 2.4, a direct interaction is an interaction whose multiplicity is multiplied by the first-row selector, so that it is counted once.

Proof. The fraction constraints determine $\mathrm{im}_t(r)$ uniquely when the denominators are nonzero. At $X = \omega^0$ the running-sum constraint gives $\mathrm{S}(0) = \sum_t \mathrm{im}_t(0)$, since $L_1(1) = 1$; at $X = \omega^r$ with $r > 0$ it gives $\mathrm{S}(r) = \mathrm{S}(r-1) + \sum_t \mathrm{im}_t(r)$. Hence $\mathrm{S}(N-1) = \sum_r \sum_t \mathrm{im}_t(r)$, which is the sum over all rows and all row interactions. At $X = \omega^{N-1}$ the selector $L_1(\omega X) = L_1(1) = 1$ is active, and the

last constraint gives s as stated; at every other row it is void. The last equality is the definition of $\mathrm{BusCont}_\chi$ (Definition 2.8), grouping the row terms by message. □

4.4. **The Protocol.**

We fix an AIR and its trace height $N = 2^n$ and blowup 2^k. The FRI schedule fixes the number R of folding rounds and the *folding factors* $m_1, ..., m_R$, powers of two with $m_1 \cdot ... \cdot m_R$ dividing N. The layers of FRI are

$$\mathcal{L}_0 := D, \quad \mathcal{L}_i := \mathcal{L}_{i-1}^{m_i}, \quad N_0 := N, \quad N_i := N_{i-1}/m_i, \quad \mathcal{C}_i := \mathsf{RS}[\mathbb{K}, \mathcal{L}_i, N_i]$$

for $i \in [R]$, so that $\mathcal{C}_0 = \mathsf{RS}[\mathbb{K}, D, N]$, every $\mathcal{C}_i$ has rate ρ, and by Lemma 4.6 the fold of a codeword of $\mathcal{C}_{i-1}$ with factor m_i is a codeword of $\mathcal{C}_i$. We write $2^{b_i} := |\mathcal{L}_i|$, so that $b_0 = n_e$, and for a point $x \in \mathcal{L}_0$ we write $x^{(0)} := x$ and $x^{(i)} := \left(x^{(i-1)}\right)^{m_i} \in \mathcal{L}_i$. We also write $\mathcal{C}^+ := \mathsf{RS}[\mathbb{K}, D, N+1]$, of rate $\rho^+ := \rho + 1/|D|$, for the code in which the analysis decodes the committed columns (Remark 4.2).

The *setup* fixes everything about an AIR that does not depend on a particular proof.

Definition 4.13 (Setup). The *setup* is a deterministic, public procedure, run once for a version of ZisK, whose input is the description of the chips and the security target. For every chip it fixes:

1. The trace height $N = 2^n$, the blowup 2^k, the FRI schedule and the number of queries q, and, for the compiled argument, the arity a and the verification level L of the Merkle trees and the grinding bits g_{pow}.
2. The rewriting of the constraints into constraints of bounded degree with intermediate columns (Section 4.3), the grouping of the interactions into fraction columns (Section 4.3.1), the order $C_0, ..., C_{n_C-1}$ of the constraints, and the orderings κ and κ_o of the evaluations used by Protocol 4.16.
3. The evaluations on D of the fixed columns, and a commitment to them, which is the verification key $\mathsf{vk}_\mathcal{C}$ of the chip.

It also generates the recursive verifiers and their verification keys (Section 3.3). The setup does not depend on the program: the program code, which is cached data of the Rom chip, is committed once per program, and its root is one of the publics.

After the rewriting, the AIR has W columns, of which the fixed ones are given by the setup, and constraints $C_0, ..., C_{n_C-1}$ of degree at most d that read the offsets O. Let $\mathcal{E}$ be the set of pairs (f, o) such that column f is read at offset o by some constraint, together with the pairs $(q_t, 0)$ for the quotient chunks $q_0, ..., q_{d-2}$ below; let $O_\mathcal{E}$ be the set of offsets occurring in $\mathcal{E}$ and $t_o := |\{f : (f, o) \in \mathcal{E}\}|$. The orderings of the setup are bijections $\kappa : O_\mathcal{E} \to [0, |O_\mathcal{E}|)$ and $\kappa_o : \{f : (f, o) \in \mathcal{E}\} \to [0, t_o)$.

The protocol proves the constraints of the AIR after the setup's rewriting, with the bus key and the bus contribution in the instance. We call the columns of all stages, including the intermediate and bus columns, and the AIR values of both stages, the *expanded witness*.

Definition 4.14 (Expanded AIR Relation). For $\tau \in [M]$, $\mathcal{R}^{(\tau)}_{\mathrm{STARK}}$ is the set of pairs $((\boldsymbol{p}, \chi, s); (\boldsymbol{T}^+, \boldsymbol{a}^+))$ such that $\boldsymbol{T}^+$ assigns to every column of the expanded AIR, at every row of $[0, N)$, a value in $\mathbb{F}$ for the fixed, cached and first-stage columns and in $\mathbb{K}$ for the second-stage ones, the fixed columns taking their prescribed values, $\boldsymbol{a}^+ = \left(\boldsymbol{a}^{(1)}, \boldsymbol{a}^{(2)}\right)$ with $\boldsymbol{a}^{(1)}$ over $\mathbb{F}$, and every constraint $C_0, ..., C_{n_C-1}$ of the expanded AIR holds at every row, with the values $\boldsymbol{p}$, $\boldsymbol{a}^+$ and s of Section 4.3 and the challenges χ as coefficients.

The relation $\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$ of Definition 3.3, whose statement is $(\boldsymbol{p}, \chi, s, \mathrm{com})$ and whose witness is $w = (\boldsymbol{T}, \boldsymbol{v})$, differs from it in three respects.

- **Witness.** $\boldsymbol{T}$ consists of the fixed, cached and first-stage columns, and the AIR values of $\boldsymbol{v}$ are $\boldsymbol{a}^{(1)}$. The intermediate columns, the bus columns and $\boldsymbol{a}^{(2)}$ are functions of (w, χ), which the constraints force.
- **Commitments.** $\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$ binds the witness to the trace commitment com, and its local satisfaction requires the cached columns to open the commitment carried in $\boldsymbol{p}$. Both are properties of the compiled argument (Section 5.4).
- **Bus contribution.** The constraints force $s = \mathrm{BusCont}_{\chi}(\mathcal{I}_\tau, w)$ only when the key is not a pole of the witness. For a witness w, a key χ is a *pole* of w if a denominator $\delta_\iota(r)$ of a row interaction, or a denominator of a direct interaction, vanishes.

Lemma 4.15 (From the Expanded to the AIR Relation). *Let* $((\boldsymbol{p}, \chi, s); (\boldsymbol{T}^+, \boldsymbol{a}^+)) \in \mathcal{R}^{(\tau)}_{\mathrm{STARK}}$, *and let* $w = (\boldsymbol{T}, \boldsymbol{v})$ *be its restriction to the fixed, cached and first-stage columns, with values* $\boldsymbol{v}$ *whose AIR values are the first-stage ones* $\boldsymbol{a}^{(1)}$ *and whose publics and proof values are* $\boldsymbol{p}$. *If* χ *is not a pole of* w, *then* w *satisfies every constraint of the AIR before the rewriting, and* $s = \mathrm{BusCont}_{\chi}(\mathcal{I}_\tau, w)$.

Proof. An intermediate column I is constrained by $I - E = 0$, so it equals the subexpression it replaces, and every original constraint holds. If χ is not a pole of w, Lemma 4.12 applies to the bus columns and to s. □

Remark 4.1 (Poles). *At a pole the bus constraints do not determine* s. *If* $\delta_\iota(r) = 0$ *for a row interaction* ι *of a group* G_t *with* $m_\iota(r) = 0$, *the fraction constraint of Definition 4.11 holds at row* r *for every value of* $\mathrm{im}_t(r)$, *and the prover may choose that value, and with it* s, *freely. Since* χ *is part of the instance, an IOP for* $\mathcal{R}^{(\tau)}_{\mathrm{STARK}}$ *cannot exclude this. In ZisK the key is derived after the witness is committed (Section 3.1), and a fixed witness has a pole at a uniformly random key with probability at most* $(N|\mathcal{I}_\tau| + |\mathcal{I}_\tau|)/|\mathbb{K}|$: *for every* γ, *each row of each interaction excludes one value of* α. *The binding lemma of Section 5 charges this probability.*

We describe the protocol as an IOP (Section 4.2). Its instance is the public statement, the bus key and the bus contribution; the key is an input rather than a challenge because ZisK derives it once for all the AIR proofs of an execution (Section 3.1).

Protocol 4.16 (ZisK Proving Protocol). Inputs. The instance is $\mathbb{x} = (\boldsymbol{p}, \chi, s)$, with $\chi = (\alpha, \gamma)$. In the honest case, the prover holds an expanded witness (Definition 4.14) for $\mathbb{x}$.

Interaction phase.

1. **Trace (Stages 1 and 2).** The prover sends the oracles $D \to \mathbb{F}$ of the cached and first-stage columns, including the intermediate columns of that stage, and the first-stage AIR values $\boldsymbol{a}^{(1)}$, over $\mathbb{F}$. It then sends the oracles $D \to \mathbb{K}$ of the bus columns of Definition 4.11, computed with χ, and of the remaining intermediate columns, and the second-stage AIR values $\boldsymbol{a}^{(2)}$. In the honest case every oracle is the evaluation on D of the interpolant of its column. Together with the evaluations of the fixed columns, these oracles form the tuple $\boldsymbol{f} \coloneqq (f_1, ..., f_W)$. The verifier samples $v_c \leftarrow \mathbb{K}$.
2. **Quotient (Stage 3).** The prover sends oracles $\boldsymbol{q} \coloneqq (q_0, ..., q_{d-2})$, $q_t : D \to \mathbb{K}$. In the honest case

$$\hat{Q}(X) \coloneqq \frac{\sum_{j<n_C} v_c^j \cdot \hat{C}_j(X)}{Z_H(X)} = \sum_{t=0}^{d-2} \left(X^N\right)^t \cdot \hat{q}_t(X), \quad \hat{q}_t \in \mathbb{K}^{<N}[X],$$

which is in $\mathbb{K}^{<(d-1)N}[X]$ because $\deg\left(\hat{C}_j\right) \le d(N-1)$, and $q_t \coloneqq \hat{q}_t|_D$. The verifier samples $\xi \leftarrow \mathbb{K}$.

3. **Evaluations (Stage 4).** The prover sends $\boldsymbol{y} := \big(y_{f,o}\big)_{(f,o)\in\mathcal{E}}$; in the honest case $y_{f,o} := \hat{f}(\xi\omega^o)$. The verifier samples $v_1, v_2 \leftarrow \mathbb{K}$.
4. **FRI (Stage 5).** Let $F : D \to \mathbb{K}$ be the *DEEP function*

$$F := \sum_{o\in O_{\mathcal{E}}} v_1^{\kappa(o)} \sum_{f:(f,o)\in\mathcal{E}} v_2^{\kappa_o(f)} \cdot g_{f,o}, \quad g_{f,o}(x) := \frac{f(x) - y_{f,o}}{x - \xi\omega^o}.$$

The prover sends an oracle $F_0 : \mathcal{L}_0 \to \mathbb{K}$; in the honest case $F_0 := F$. For $i = 1, ..., R$, the verifier samples $\beta_i \leftarrow \mathbb{K}$ and the prover sends $F_i : \mathcal{L}_i \to \mathbb{K}$; in the honest case $F_i := \mathsf{Fold}(F_{i-1}, m_i, \beta_i)$. The last function, F_R, is sent in full, as its $|\mathcal{L}_R|$ values.
5. **Queries.** The verifier samples $x_1, ..., x_q \leftarrow \mathcal{L}_0$.

Decision phase. The verifier accepts if and only if:
1. *Constraint check.* With $\boldsymbol{v}$ the values of Section 4.3,

$$\sum_{j<n_C} v_c^j \cdot C_j\Big(\big(y_{f,o}\big)_{(f,o)}, \boldsymbol{v}\Big) = (\xi^N - 1)\sum_{t=0}^{d-2} \xi^{tN} \cdot y_{q_t,0}.$$

2. *Final function check.* $F_R \in \mathcal{C}_R$.
3. *Local fold checks.* For every query $x = x_\ell$: the verifier computes $F(x)$ from the values of the oracles $\boldsymbol{f}, \boldsymbol{q}$ at x and checks $F(x) = F_0(x)$; and for every $i \in [R]$ it reads F_{i-1} on $\mathsf{Block}\big(\mathcal{L}_{i-1}, m_i, x^{(i)}\big)$, computes $\mathsf{Fold}(F_{i-1}, m_i, \beta_i)\big(x^{(i)}\big)$ from these values, and checks that it equals $F_i\big(x^{(i)}\big)$.

The honest prover's functions satisfy every check. Its constraint polynomials vanish on H, so $\hat{Q}$ is a polynomial and the constraint check holds at every ξ. Every $g_{f,o}$ is the evaluation of a polynomial in $\mathbb{K}^{<N-1}[X]$, so $F \in \mathcal{C}_0$, and by Lemma 4.6 $F_i \in \mathcal{C}_i$ for every i. The only exception is a point $\xi\omega^o \in D$, where $g_{f,o}$ is undefined. The verifier computes $1/(x - \xi\omega^o)$ without checking for it, and for ξ uniform in $\mathbb{K}$ it occurs with probability at most $|O_{\mathcal{E}}||D|/|\mathbb{K}|$.

Remark 4.2 (The Off-by-One in the DEEP Quotients). *For an honest column in $\mathbb{K}^{<N}[X]$, the quotient $g_{f,o}$ is in $\mathbb{K}^{<N-1}[X]$, but the low-degree test only certifies membership in $\mathbb{K}^{<N}[X]$. If $g_{f,o}$ agrees with a polynomial $\hat{w} \in \mathbb{K}^{<N}[X]$, then f agrees with $(X - \xi\omega^o)\hat{w} + y_{f,o} \in \mathbb{K}^{<N+1}[X]$. This is why the analysis decodes $\boldsymbol{f}$ and $\boldsymbol{q}$ in $\mathcal{C}^+ = \mathsf{RS}[\mathbb{K}, D, N+1]$. The slack is harmless. A column $\hat{u} \in \mathbb{K}^{<N+1}[X]$ equals $\hat{u}_0 + cZ_H$ with $\hat{u}_0 \in \mathbb{K}^{<N}[X]$, so it takes the same values on H. Since $\hat{u}(\omega^o X) = \hat{u}_0(\omega^o X) + cZ_H(X)$, the constraint polynomials of $\hat{u}$ and $\hat{u}_0$ are congruent modulo Z_H. A quotient with chunks in $\mathbb{K}^{<N+1}[X]$ is in $\mathbb{K}^{<(d-1)N+1}[X]$.*

4.5. Round-by-Round Soundness.

Theorem 4.17 (Round-by-Round Knowledge Soundness). *Consider Protocol 4.16 for the chip of type τ, with the parameters of Section 4.4, and assume that* Powers *has mutual correlated agreement, with error $\varepsilon^{\mathrm{mca}}$, for $\mathcal{C}_0, ..., \mathcal{C}_R$. Let $\delta \in (0, 1 - \rho^+)$ be such that the interleaved code $(\mathcal{C}^+)^{W+d-1}$ is (δ, ℓ)-list decodable, with a list-decoding algorithm running in time $\mathsf{et}_{\mathrm{RS}}$, and let $\theta \in [0, 1]$ be a* tradeoff parameter. *Then Protocol 4.16 is an IOP for $\mathcal{R}^{(\tau)}_{\mathrm{STARK}}$ with round-by-round knowledge soundness errors*

$$\big(\varepsilon^{\mathrm{ali}}, \varepsilon^{\mathrm{ood}}, \varepsilon^{\mathrm{deep}}, \varepsilon_1^{\mathrm{fold}}, ..., \varepsilon_R^{\mathrm{fold}}, \varepsilon^{\mathrm{qry}}\big)$$

and extraction time $\mathsf{et}_{\mathrm{RS}} + O(\ell \cdot n_C \cdot N)$, where

$$\varepsilon^{\mathrm{ali}} \le \frac{\ell \cdot (n_C - 1)}{|\mathbb{K}|}, \qquad \varepsilon^{\mathrm{ood}} \le \frac{\ell \cdot d \cdot N + |O_{\mathcal{E}}| \cdot |D|}{|\mathbb{K}|},$$
$$\varepsilon^{\mathrm{deep}} \le \varepsilon^{\mathrm{mca}}(\mathcal{C}_0, |O_{\mathcal{E}}|, \delta) + \sum_{o \in O_{\mathcal{E}}} \varepsilon^{\mathrm{mca}}(\mathcal{C}_0, t_o, \delta), \quad \varepsilon_i^{\mathrm{fold}} \le \varepsilon^{\mathrm{mca}}(\mathcal{C}_i, m_i, \theta),$$
$$\varepsilon^{\mathrm{qry}} \le (1 - \min\{\theta, \delta\})^q.$$

The rounds are named after the challenge that ends them: the constraint combination v_c (*ali*, for algebraic linking), the out-of-domain point ξ, the DEEP coefficients (v_1, v_2), the folding challenges $\beta_1, ..., \beta_R$ and the queries. The round in which the prover sends F_0 ends with β_1, and we count it as the first folding round. The tradeoff parameter θ trades the folding errors, which grow with θ, against the query error, which the factor $1 - \min\{\theta, \delta\}$ clamps at $1 - \theta$ [GMW25].

Proof. We first fix notation, then define a state function and an extractor, and finally bound the probability that each round flips the state.

Notation. For a tuple $\boldsymbol{u} = \big(\hat{u}_f\big)_f$ of polynomials, one per column, we write $\hat{C}_j[\boldsymbol{u}]$ for the constraint polynomial of Definition 4.10 computed from $\boldsymbol{u}$ and the values $\boldsymbol{v}$, which the instance and the transcript determine. For a tuple $\boldsymbol{u}$ that also contains chunks $\hat{u}_{q_0}, ..., \hat{u}_{q_{d-2}}$, let $\hat{Q}_{\boldsymbol{u}} \coloneqq \sum_t X^{tN} \hat{u}_{q_t}$ and

$$\mathcal{D}_{\boldsymbol{u}}(X) \coloneqq \sum_{j < n_C} v_c^j \hat{C}_j[\boldsymbol{u}](X) - Z_H(X) \cdot \hat{Q}_{\boldsymbol{u}}(X).$$

The constraint check for $\boldsymbol{y}$ states that $\sum_j v_c^j C_j\big((y_{f,o}), \boldsymbol{v}\big) = (\xi^N - 1) \sum_t \xi^{tN} y_{q_t,0}$; when $y_{f,o} = \hat{u}_f(\xi\omega^o)$ for every $(f, o) \in \mathcal{E}$, it states that $\mathcal{D}_{\boldsymbol{u}}(\xi) = 0$. We call ξ *bad* if $\xi\omega^o \in D$ for some $o \in O_{\mathcal{E}}$. We write $\Lambda_1 \coloneqq \Lambda\Big((\mathcal{C}^+)^W, \boldsymbol{f}, \delta\Big)$ and $\Lambda_2 \coloneqq \Lambda\Big((\mathcal{C}^+)^{W+d-1}, (\boldsymbol{f}, \boldsymbol{q}), \delta\Big)$, identifying codewords with their interpolants. Both lists have at most ℓ elements (a list of the shorter tuple is at most as long), and the restriction of an element of Λ_2 to its first W coordinates is in Λ_1.

The state function. We define $\mathsf{State}(\mathbb{x}, \mathsf{tr})$ for transcripts that end with a verifier message; on the other transcripts it is given by the prover-moves property of Definition 4.8, and on the empty transcript it is 0. In every round the state is 1 if it already was, which gives the default property; otherwise:

1. *Ali.* $\mathsf{State}(\mathbb{x}, \mathsf{tr} \parallel v_c) = 1$ if and only if some $\boldsymbol{u} \in \Lambda_1$ satisfies $\sum_{j < n_C} v_c^j \hat{C}_j[\boldsymbol{u}] \equiv 0 \bmod Z_H$.
2. *Out of domain.* $\mathsf{State}(\mathbb{x}, \mathsf{tr} \parallel \xi) = 1$ if and only if ξ is bad, or some $\boldsymbol{u} \in \Lambda_2$ satisfies $\mathcal{D}_{\boldsymbol{u}}(\xi) = 0$.
3. *DEEP.* $\mathsf{State}(\mathbb{x}, \mathsf{tr} \parallel (v_1, v_2)) = 1$ if and only if the constraint check holds for $\boldsymbol{y}$ and $\Delta(F, \mathcal{C}_0) \le \delta$.
4. *Folding.* $\mathsf{State}(\mathbb{x}, \mathsf{tr} \parallel \beta_i) = 1$ if and only if β_i is bad for F_{i-1} at θ (Lemma 4.6, with $\mathcal{L} = \mathcal{L}_{i-1}$, $m = m_i$, $\mathcal{C} = \mathcal{C}_{i-1}$ and $\mathcal{C}' = \mathcal{C}_i$): there is $S \subset \mathcal{L}_{i-1}$ with $|S^{m_i}| \ge (1 - \theta) \cdot |\mathcal{L}_i|$ such that $\mathsf{Fold}(F_{i-1}, m_i, \beta_i)$ agrees on S^{m_i} with a codeword of $\mathcal{C}_i$, but F_{i-1} agrees on S with no codeword of $\mathcal{C}_{i-1}$.
5. *Queries.* $\mathsf{State}\big(\mathbb{x}, \mathsf{tr} \parallel (x_1, ..., x_q)\big) = 1$ if and only if the verifier accepts.

The empty-transcript and prover-moves properties of Definition 4.8 hold by definition, and the default property by construction. The full-transcript property holds because, if the state of a full transcript is 0, then in particular the verifier does not accept.

The extractor. On $(\mathbb{x}, \mathsf{tr})$, where tr ends with the first prover message, E list-decodes $\boldsymbol{f}$ to obtain Λ_1, and outputs the values on H of the first $\boldsymbol{u} \in \Lambda_1$ such that every $\hat{C}_j[\boldsymbol{u}]$ vanishes on H, together with $\big(\boldsymbol{a}^{(1)}, \boldsymbol{a}^{(2)}\big)$, if there is one. On all other transcripts it outputs $\perp$. It runs in time $\mathsf{et}_{\mathrm{RS}} + O(\ell \cdot n_C \cdot N)$. The first-stage columns of its output are over $\mathbb{F}$: if $\hat{u} \in \mathbb{K}^{<N+1}[X]$ agrees with

an $\mathbb{F}$-valued oracle on a set $S \subset D$ with $|S| \geq (1-\delta)|D| > N$, then the coordinates of $\hat{u}$ in an $\mathbb{F}$ -basis of $\mathbb{K}$ containing 1, other than the first, are polynomials over $\mathbb{F}$ of degree at most N that vanish on S, since $D \subset \mathbb{F}$; hence they are zero and $\hat{u} \in \mathbb{F}[X]$.

Bounding the errors. We bound, for each round, the probability that the state flips from 0 to 1. Only in the first round do we need the extractor; in every other round the probability is bounded whatever the transcript.

1. **Ali.** Suppose $\Pr_{v_c}[\mathsf{State}(\aleph, \mathsf{tr} \parallel v_c) = 1] > \ell(n_C - 1)/|\mathbb{K}|$. For a fixed $\boldsymbol{u} \in \Lambda_1$, if some $\hat{C}_j[\boldsymbol{u}]$ does not vanish at some $h \in H$, then $\sum_j v_c^j \hat{C}_j[\boldsymbol{u}](h)$ is a nonzero polynomial in v_c of degree at most $n_C - 1$, so $\sum_j v_c^j \hat{C}_j[\boldsymbol{u}] \equiv 0 \bmod Z_H$ for at most $n_C - 1$ values of v_c. By a union bound over the at most ℓ elements of Λ_1, some $\boldsymbol{u}$ makes every $\hat{C}_j[\boldsymbol{u}]$ vanish on H. Its values on H, with the AIR values, satisfy every constraint at every row, so E outputs a witness of $\mathcal{R}^{(\tau)}_{\mathrm{STARK}}$.
2. **Out of domain.** Suppose $\mathsf{State}(\aleph, \mathsf{tr}) = 0$: no $\boldsymbol{u} \in \Lambda_1$ makes $\sum_j v_c^j \hat{C}_j[\boldsymbol{u}]$ divisible by Z_H. Then for every $\boldsymbol{u} \in \Lambda_2$ the polynomial $\mathcal{D}_{\boldsymbol{u}}$ is nonzero, for otherwise Z_H would divide the combination for the restriction of $\boldsymbol{u}$, which is in Λ_1. Its degree is at most dN: the columns of $\boldsymbol{u}$ are in $\mathbb{K}^{<N+1}[X]$, so $\deg\big(\hat{C}_j[\boldsymbol{u}]\big) \leq dN$, and $\deg\big(Z_H \hat{Q}_{\boldsymbol{u}}\big) \leq N + (d-1)N$. Hence $\Pr_\xi[\mathcal{D}_{\boldsymbol{u}}(\xi) = 0] \leq dN/|\mathbb{K}|$, and a union bound over Λ_2 and over the $|O_{\mathcal{E}}| \cdot |D|$ values of ξ that are bad gives $\varepsilon^{\mathrm{ood}}$.
3. **DEEP.** Suppose $\mathsf{State}(\aleph, \mathsf{tr}) = 0$, and that the constraint check holds for $\boldsymbol{y}$; otherwise the new state is 0. Write $G_o \coloneqq \sum_f v_2^{\kappa_o(f)} g_{f,o}$, so that $F = \sum_o v_1^{\kappa(o)} G_o$. The functions $g_{f,o}$ are fixed before (v_1, v_2) is sampled. Call (v_1, v_2) good if v_2 is not bad, in the sense of Lemma 4.4 at δ, for the tuple $\big(g_{f,o}\big)_f$ of any $o \in O_{\mathcal{E}}$, and v_1 is not bad at δ for the tuple $(G_o)_{o \in O_{\mathcal{E}}}$, which is fixed once v_2 is. Since κ and every κ_o are bijections onto initial segments of $\mathbb{N}$, these are combinations by Powers, so (v_1, v_2) is good except with probability $\varepsilon^{\mathrm{deep}}$. We show that if (v_1, v_2) is good then $\Delta(F, \mathcal{C}_0) > \delta$.

   Suppose $\Delta(F, \mathcal{C}_0) \leq \delta$, and let $S \subset D$, with $|S| \geq (1-\delta)|D|$, be a set on which F agrees with a codeword. As v_1 is good, every G_o agrees with a codeword on S; as v_2 is good, every $g_{f,o}$ agrees on S with a codeword $\hat{w}_{f,o}$ of $\mathcal{C}_0$. Then f agrees on S with $\hat{u}_f^{(o)} \coloneqq (X - \xi\omega^o)\hat{w}_{f,o} + y_{f,o} \in \mathbb{K}^{<N+1}[X]$, for every o with $(f, o) \in \mathcal{E}$. Two such polynomials agree on S, and $|S| \geq (1-\delta)|D| > N$ because $\delta < 1 - \rho^+$, so they are equal; call the common polynomial $\hat{u}_f$. It satisfies $\hat{u}_f(\xi\omega^o) = y_{f,o}$ for every $(f, o) \in \mathcal{E}$, since ξ is not bad. The tuple $\boldsymbol{u} \coloneqq \big(\hat{u}_f\big)_f$, over the columns and the chunks, agrees with $(\boldsymbol{f}, \boldsymbol{q})$ on S, so $\boldsymbol{u} \in \Lambda_2$; and the constraint check for $\boldsymbol{y}$ is $\mathcal{D}_{\boldsymbol{u}}(\xi) = 0$. Hence $\mathsf{State}(\aleph, \mathsf{tr}) = 1$, a contradiction.
4. **Folding.** The oracle F_{i-1} is fixed before β_i is sampled, and the state flips exactly when β_i is bad for F_{i-1} at θ, which by Lemma 4.6 has probability at most $\varepsilon^{\mathrm{mca}}(\mathcal{C}_i, m_i, \theta)$.
5. **Queries.** Suppose $\mathsf{State}(\aleph, \mathsf{tr}) = 0$, and that the constraint check holds and $F_R \in \mathcal{C}_R$; otherwise the verifier rejects whatever the queries. Unwinding the definition of the state, $\Delta(F, \mathcal{C}_0) > \delta$ and no β_i is bad for F_{i-1} at θ. Let $\mathsf{AccQueries} \subset \mathcal{L}_0$ be the set of queries that pass the local fold checks. By Lemma 4.18 below, $|\mathsf{AccQueries}| \leq \max\{1-\theta, 1-\delta\} \cdot |\mathcal{L}_0|$. The q queries are independent and uniform, so all of them are in $\mathsf{AccQueries}$ with probability at most $(1 - \min\{\theta, \delta\})^q$.

It remains to prove Lemma 4.18. □

Lemma 4.18 (Query Phase). *Let*

$$\mathsf{AccQueries} \coloneqq \big\{x \in \mathcal{L}_0 : F(x) = F_0(x) \text{ and } \mathsf{Fold}(F_{i-1}, m_i, \beta_i)\big(x^{(i)}\big) = F_i\big(x^{(i)}\big) \text{ for every } i \in [R]\big\}$$

be the set of queries that pass the local fold checks, and suppose that $F_R \in \mathcal{C}_R$ and that no β_i is bad for F_{i-1} at θ. Then

$$|\mathsf{AccQueries}| \le \max\{1-\theta, 1-\Delta(F,\mathcal{C}_0)\} \cdot |\mathcal{L}_0|.$$

Proof. Following [GMW25], let $W_0 := \mathsf{AccQueries}$ and $W_i := W_{i-1}^{m_i} \subset \mathcal{L}_i$ for $i \in [R]$, the layers of the subgraph of accepted queries.

Folds are honest on the W_i. For every $i \in [R]$, $F_i[W_i] = \mathsf{Fold}(F_{i-1}, m_i, \beta_i)[W_i]$: every $w \in W_i$ is $x^{(i)}$ for some $x \in \mathsf{AccQueries}$, and the local fold check of x at layer i is this equality at w.

A small layer makes W_0 small. Since W_{i-1} is contained in the union of the blocks of the points of W_i, each of size m_i, $|W_{i-1}|/|\mathcal{L}_{i-1}| \le |W_i|/|\mathcal{L}_i|$. Hence, if $|W_i| < (1-\theta)|\mathcal{L}_i|$ for some i, then $|W_0| < (1-\theta)|\mathcal{L}_0|$.

Large layers propagate agreement. Suppose $|W_i| \ge (1-\theta)|\mathcal{L}_i|$ for every i. We show by induction, from $i = R$ down to 0, that F_i agrees on W_i with a codeword of $\mathcal{C}_i$. For $i = R$ this holds since $F_R \in \mathcal{C}_R$. If $F_i[W_i] = u_i[W_i]$ with $u_i \in \mathcal{C}_i$, then $\mathsf{Fold}(F_{i-1}, m_i, \beta_i)[W_i] = u_i[W_i]$ by honesty of the folds, and $W_i = W_{i-1}^{m_i}$ is large; since β_i is not bad for F_{i-1} at θ, the set $S = W_{i-1}$ does not witness badness, so F_{i-1} agrees on W_{i-1} with a codeword of $\mathcal{C}_{i-1}$.

In the first case $|\mathsf{AccQueries}| < (1-\theta)|\mathcal{L}_0|$. In the second, $F_0[W_0] = u_0[W_0]$ for some $u_0 \in \mathcal{C}_0$, and $F[W_0] = F_0[W_0]$ by definition of $\mathsf{AccQueries}$, so F agrees with u_0 on W_0 and $|\mathsf{AccQueries}| \le (1-\Delta(F,u_0))|\mathcal{L}_0| \le (1-\Delta(F,\mathcal{C}_0))|\mathcal{L}_0|$. □

Lemma 4.4 discharges the hypothesis on $\varepsilon^{\mathrm{mca}}$ of Theorem 4.17 up to the Johnson bound, and Theorem 4.2 bounds the list size ℓ; the natural choice is $\theta = \delta$.

Corollary 4.19 (Johnson Regime). *Let $\eta \in \left(0, 1-\sqrt{\rho^+}\right)$ and $\theta = \delta := 1-\sqrt{\rho^+}-\eta$. Then Protocol 4.16 has round-by-round knowledge soundness with $\ell \le 1/\left(2\eta\sqrt{\rho^+}\right)$ and errors*

$$\varepsilon^{\mathrm{ali}} \le \frac{\ell \cdot (n_C - 1)}{|\mathbb{K}|}, \quad \varepsilon^{\mathrm{ood}} \le \frac{\ell \cdot d \cdot N + |O_{\mathcal{E}}| \cdot |D|}{|\mathbb{K}|},$$
$$\varepsilon^{\mathrm{deep}} \le \varepsilon^{\mathrm{mca}}(\mathcal{C}_0, |\mathcal{E}|, \delta), \quad \varepsilon_i^{\mathrm{fold}} \le \varepsilon^{\mathrm{mca}}(\mathcal{C}_i, m_i, \delta),$$
$$\varepsilon^{\mathrm{qry}} \le \left(\sqrt{\rho^+} + \eta\right)^q,$$

with $\varepsilon^{\mathrm{mca}}$ as in Lemma 4.4, and extraction time polynomial in $|D| \cdot (W+d)$.

Proof. Since $\delta < 1-\sqrt{\rho^+}$ and every $\mathcal{C}_i$ has rate $\rho \le \rho^+$, Lemma 4.4 applies to every code at δ, and Theorem 4.2 bounds the list size of $(\mathcal{C}^+)^{W+d-1}$ at δ by $1/\left(2\eta\sqrt{\rho^+}\right)$; lists of this size can be computed in polynomial time. Theorem 4.17 with $\theta = \delta$ gives the errors, and $1 - \min\{\theta, \delta\} = 1 - \delta = \sqrt{\rho^+} + \eta$.

For the DEEP round, the error of Lemma 4.4 at $\delta < 1-\sqrt{\rho}$ has the form $\varepsilon^{\mathrm{mca}}(\mathcal{C}_0, m, \delta) = (m-1) \cdot c$, with c independent of m. Since $\sum_{o \in O_{\mathcal{E}}} t_o = |\mathcal{E}|$, the two terms of $\varepsilon^{\mathrm{deep}}$ in Theorem 4.17 add up to

$$(|O_{\mathcal{E}}| - 1) \cdot c + \sum_{o \in O_{\mathcal{E}}} (t_o - 1) \cdot c = (|\mathcal{E}| - 1) \cdot c = \varepsilon^{\mathrm{mca}}(\mathcal{C}_0, |\mathcal{E}|, \delta),$$

the error of batching all the quotients $g_{f,o}$ with the powers of a single challenge. □

With $|\mathbb{K}| \approx 2^{192}$, every error except $\varepsilon^{\mathrm{qry}}$ is negligible for the trace heights of ZisK, and the soundness of a proof is governed by the query phase and by grinding, which enters at compilation (Section 5.3). Section 6 computes the resulting security levels.

5. Non-Interactive Compilation and Extraction

The IOP of Section 4 becomes a non-interactive argument through the transformation of Ben-Sasson, Chiesa and Spooner [BCS16]. Oracles are replaced by Merkle roots and their queries by openings, and challenges by the outputs of a hash function applied to the transcript. This section describes the compiled argument as ZisK implements it, bounds its knowledge-soundness error, and identifies where ZisK departs from the textbook transformation.

5.1. Merkle Commitments.

All hashing is done with Poseidon2 [GKS23] over $\mathbb{F}$. We write $P_w : \mathbb{F}^w \to \mathbb{F}^w$ for the Poseidon2 permutation of width w, and $[u]_4$ for the first four coordinates of a vector u. A *digest* is an element of $\mathbb{F}^4$. Merkle trees are parametrized by their *arity* $a \geq 2$, and a tree of arity a uses the permutation of width $4a$, which compresses a digests at once.

Definition 5.1 (Linear Hash). Fix an arity a and let $r_a := 4(a-1)$. For $u = (u_1, ..., u_\ell) \in \mathbb{F}^\ell$ with $\ell \geq 1$, split u into chunks $u^{(1)}, ..., u^{(t)}$ of r_a consecutive elements, the last one possibly shorter, and let $\overline{u}^{(i)} \in \mathbb{F}^{r_a}$ be $u^{(i)}$ padded with zeros. The *linear hash* of u is $\mathrm{LH}_a(u) := [s_t]_4$, where

$$s_1 := P_{4a}\big(\overline{u}^{(1)} \parallel 0^4\big), \quad s_i := P_{4a}\Big(\overline{u}^{(i)} \parallel [s_{i-1}]_4\Big) \text{ for } 1 < i \leq t.$$

That is, a sponge of width $4a$, rate r_a and capacity 4.

Definition 5.2 (Merkle Commitment). Fix an arity a and a *verification level* $L \geq 0$. Let M be a matrix with R rows $M_0, ..., M_{R-1}$ over $\mathbb{F}$ (rows over $\mathbb{K}$ are flattened coordinatewise). The *leaves* of its Merkle tree are $\lambda_i := \mathrm{LH}_a(M_i)$. Nodes are computed level by level: a level of u digests is padded with zero digests to a multiple of a, and each group of a consecutive digests $(d_1, ..., d_a)$ is compressed into the parent

$$\mathrm{Comp}_a(d_1, ..., d_a) := [P_{4a}(d_1 \parallel ... \parallel d_a)]_4,$$

until a single digest remains, the *root* $\mathrm{MT}_a(M)$. An *opening* of row i is the row M_i together with, at each level up to level L below the root, the $a-1$ siblings of the current node. The proof carries, once per tree, the at most a^L nodes of level L. The verifier recomputes the path of each opening up to level L, compares the result with the corresponding node of that level, and hashes the level into the root. With $L = 0$ this is the usual Merkle tree.

The arity and the verification level are parameters of each AIR; ZisK uses arity $a = 4$ and, for most AIRs, a level with 4 or 16 nodes. Nothing below depends on the values of a and L, and we drop the subscript a when it is clear from the context. A group of polynomials $f_1, ..., f_c$ is *committed on* D by committing to the matrix whose row i is $(f_1(x_i), ..., f_c(x_i))$, so that one opening reveals the values of all of them at one point of D.

5.2. The Fiat–Shamir Transcript.

Definition 5.3 (Fiat–Shamir Transcript). A *transcript* holds a state $S \in \mathbb{F}^{16}$, initially zero, a buffer of pending elements of length at most 12, and an output buffer.

- *Observing* $u \in \mathbb{F}$ appends u to the pending buffer and empties the output buffer; when the pending buffer has 12 elements, the transcript sets $S \leftarrow P_{16}(\text{pending} \parallel [S]_4)$, empties the pending buffer, and fills the output buffer with the 16 coordinates of the new S.
- *Sampling* a field element returns the next element of the output buffer; if the output buffer is empty, the pending buffer is first padded with zeros and the state updated as above. Sampling $c \in \mathbb{K}$ samples its three coordinates in order.

- The *digest* of a vector u is $[S]_4$ for a fresh transcript that has observed u and then updated its state if elements were pending.

Elements of $\mathbb{K}$ are observed coordinate by coordinate.

An AIR proof is not compiled on its own. The prover first commits to the first stage of every segment. The *global challenge* $g \in \mathbb{K}$ of the execution is the hash of the public statement and of the accumulated typed commitments $\boldsymbol{c}$ of all the segments, and the bus key $\chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c}) = (\alpha, \gamma)$ is its expansion (Section 3.4). Every AIR transcript is seeded with g and samples (α, γ) first, so the bus key is common to all the AIR proofs of an execution and is fixed only after all their traces are. The compiled protocol replaces the oracles and challenges of Protocol 4.16 as follows.

Protocol 5.4 (ZisK Proving Protocol, Compiled). The transcript is initialized and observes the global challenge g.

1. **Trace.** The prover commits on D to the first-stage columns, including the intermediate columns of that stage, with root root_1, and computes $\boldsymbol{a}^{(1)}$. Neither root_1 nor $\boldsymbol{a}^{(1)}$ is observed. The cached columns are committed in a *custom tree*, whose root is not part of the proof: the verifier reads it from the coordinates of $\boldsymbol{p}$ that the AIR designates. The transcript samples α, then γ. The prover commits to the second-stage columns (root root_2), and the transcript observes root_2, then $\boldsymbol{a}^{(2)}$, then s, and samples v_c.
2. **Quotient.** The prover commits to $q_0, ..., q_{d-2}$ (root root_Q); the transcript observes root_Q and samples ξ.
3. **Evaluations.** The transcript observes the digest of $\boldsymbol{y}$ and samples v_1, then v_2.
4. **FRI.** For $i = 1, ..., R$, the prover commits to F_{i-1} by a Merkle tree on $\mathcal{L}_{i-1}$ whose leaves are the blocks of $x \mapsto x^{m_i}$: the leaf of index $r \in [0, 2^{b_i})$ is the vector of the m_i values of F_{i-1} at the positions $r + j2^{b_i}$, $j \in [0, m_i)$, of $\mathcal{L}_{i-1}$, which form a block. The transcript observes the root and samples β_i. The prover sends the 2^{b_R} values of F_R, and the transcript observes their digest.
5. **Grinding.** The transcript samples $c \in \mathbb{K}$. The prover finds $\nu \in \mathbb{F}$ such that $[P_4(c_0, c_1, c_2, \nu)]_1 < 2^{64 - g_{\mathrm{pow}}}$, where $[\cdot]_1$ is the first coordinate read as an integer in $[0, p)$; the verifier checks this inequality.
6. **Queries.** A fresh transcript observes (c_0, c_1, c_2, ν) and samples $\lceil q n_e / 63 \rceil$ field elements; the concatenation of the 63 low bits of each is cut into q indices $r_1, ..., r_q \in [0, 2^{n_e})$ of n_e bits each, and the queries are $x_{r_1}, ..., x_{r_q} \in D$. For a query of index r, the prover opens row r of the fixed tree, the custom trees, the stage trees and the quotient tree, and, in every FRI tree $i \in [R]$, the leaf of index $r \bmod 2^{b_i}$, which holds F_{i-1} on $\mathsf{Block}\big(\mathcal{L}_{i-1}, m_i, x^{(i)}\big)$.

The verifier checks every opening against its root, the root of the fixed tree being the verification key, and runs the decision phase of Protocol 4.16 on the opened values. The value $F_0(x)$ is at slot $\lfloor r / 2^{b_1} \rfloor$ of the opened leaf of tree 1, and $F_i\big(x^{(i)}\big)$, for $i < R$, at the corresponding slot of the leaf of tree $i + 1$, or, for $i = R$, at index $r \bmod 2^{b_R}$ of F_R.

Recursive verifiers. The recursive verifiers of Section 3.3 are proved with the same protocol, with three differences: the transcript is initialized with their verification key, the digest of their public inputs and root_1, instead of the global challenge; their second-stage challenges, if any, are sampled from that transcript; and they have no external bus key and no airgroup value read by a parent.

Table 3 summarizes what the transcript of each kind of proof observes and samples, in order.

Step	AIR proof	Recursive verifier
seed	the global challenge g only	the verification key, the digest of the public inputs, root_1
stage 2	sample α, γ; observe root_2, $\boldsymbol{a}^{(2)}$ and s	sample the stage-2 challenges, if any; observe root_2 and $\boldsymbol{a}^{(2)}$
stage 3	sample v_c; observe root_Q	same
stage 4	sample ξ; observe the digest of the evaluations	same
stage 5	sample v_1, v_2; per FRI round, observe the root and sample β_i; observe the digest of F_R	same
grinding	sample c	same
queries	fresh transcript over (c, ν)	same

TABLE 3. What the Fiat–Shamir transcript observes and samples, in order.

Two features of the AIR transcript matter for the analysis.

1. **The bus key is external.** The transcript of an AIR proof observes the global challenge g and nothing else before sampling α and γ, so the bus key is the same for every AIR proof of an execution: it is $\chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c})$ of Section 3.1, the global challenge being the output of the hash and (α, γ) its fixed expansion.
2. **The first stage is not observed.** Neither root_1 nor $\boldsymbol{a}^{(1)}$ is observed by the AIR transcript. They are bound instead through the global challenge, which is derived from $h(\tau, \mathrm{com})$ with $\mathrm{com} = \left(\mathrm{root}_1, \boldsymbol{a}^{(1)}\right)$ (Section 3.4). Likewise, the public statement is not observed by the AIR transcript but by the derivation of the global challenge.

Both are deviations from the textbook Fiat–Shamir transformation, in which everything the verifier reads is observed before the challenges that depend on it. The first is harmless for a single proof, since the key is part of the statement of $\mathcal{R}_{\mathrm{AIR}}^{(\tau)}$; the second is not, and the binding lemma of Section 5.4 shows that the global challenge restores it. Everything else the verifier of an AIR proof reads, the bus contribution s included, is observed before the challenges that depend on it.

5.3. The BCS Transformation.

Let $(\mathbf{P}, \mathbf{V})$ be an IOP whose oracles are functions on evaluation domains. Its *BCS compilation* is the non-interactive argument in which the prover:

1. Replaces every oracle by the root of a Merkle tree on its evaluation table (Definition 5.2), grouping the oracles of one round in one tree.
2. Obtains every challenge by sampling the Fiat–Shamir transcript (Definition 5.3), after having observed the instance and all the preceding prover messages, roots and plain values.
3. Answers every query of the verifier with a Merkle opening.

The verifier recomputes the challenges from the transcript, checks the openings, and runs the decision of $\mathbf{V}$ on the opened values. As in Definition 2.15, we model the transcript as a random oracle, which, given the sequence of observed elements, returns the sampled elements, and the compression functions of the Merkle trees as independent random oracles with outputs in $\mathbb{F}^4$. A proof is then determined by the adversary's oracle queries, and a *straight-line* extractor may read the list of these queries and their answers, but may not rewind the adversary.

Theorem 5.5 (BCS Compilation [BCS16, Can+19, Blo+23]). *Let $(\mathbf{P}, \mathbf{V})$ be a public-coin IOP for $\mathcal{R}$ with round-by-round knowledge soundness errors $(\varepsilon_1, \ldots, \varepsilon_k)$ and extraction time* et*. There is a straight-line extractor for its BCS compilation, running in time polynomial in the size of the query list and* et*, such that for every adversary making at most Q queries to the random*

oracles, the probability that it outputs an instance $\mathbb{x}$ and an accepting proof while the extractor outputs no $\mathbb{w}$ with $(\mathbb{x};\mathbb{w}) \in \mathcal{R}$ is at most

$$Q \cdot \max_{i\in[k]} \varepsilon_i + \varepsilon_{\mathrm{MT}}(Q),$$

where $\varepsilon_{\mathrm{MT}}(Q) = O(Q^2/p^4)$ bounds the probability that the query list contains a collision or an inversion of the hash.

The theorem measures soundness by the maximum error over the rounds, per query, rather than by their sum: the adversary may retry any round as often as its budget allows. For the parameters of ZisK, the maximum in Theorem 4.17 is $\varepsilon^{\mathrm{qry}} = (1 - \min\{\theta, \delta\})^q$, and two features of Protocol 5.4 refine it.

Lemma 5.6 (Grinding and Query Derivation). *In the BCS compilation of Protocol 4.16 by Protocol 5.4, the query round may be taken with error*

$$\varepsilon^{\mathrm{qry}}_{\mathrm{BCS}} := \frac{2^{64-g_{\mathrm{pow}}}}{p} \cdot \kappa \cdot (1 - \min\{\theta, \delta\})^q, \quad \kappa := \left(2^{64}/p\right)^{\lceil q \cdot n_e/63 \rceil},$$

in place of $\varepsilon^{\mathrm{qry}}$ in Theorem 5.5.

Proof. In Protocol 5.4 the query indices are derived from a fresh transcript over (c, ν), where c is the grinding challenge and ν a nonce with $[P_4(c_0, c_1, c_2, \nu)]_1 < 2^{64-g_{\mathrm{pow}}}$. A query of the adversary that produces accepting query indices must evaluate both oracles at a pair (c, ν); we model the grinding permutation and the query transcript as independent random oracles. For a fixed pair, the grinding check passes with probability $2^{64-g_{\mathrm{pow}}}/p$. The query indices are obtained by concatenating the low 63 bits of $\lceil q \cdot n_e/63 \rceil$ uniform field elements. A uniform element of $[0, p)$ has each residue modulo 2^{63} with probability at most $2/p = 2^{-63} \cdot 2^{64}/p$. The joint distribution of the bits therefore assigns to every string at most κ times its uniform probability, and so does the distribution of the indices to every tuple. Hence all q indices land in the set AccQueries of Lemma 4.18 with probability at most $\kappa \cdot (1 - \min\{\theta, \delta\})^q$. The two events depend on independent oracles, so a single query achieves both with probability at most the product of the two bounds. □

Since $2^{64}/p = 1 + 2^{-32} + O(2^{-64})$, κ is $1 + O(2^{-25})$ for every AIR of ZisK, and the grinding contributes a factor of about $2^{-g_{\mathrm{pow}}}$. Grinding does not change the other rounds; their errors are negligible in comparison (Corollary 4.19).

The textbook transformation and ZisK. The theorem applies to the recursive verifiers as they are: their transcript observes every message before the challenge that depends on it (Table 3). For AIR proofs, Protocol 5.4 departs from the transformation in the two ways listed in Section 5.2. The first is harmless; the second is not, as we now show.

5.4. Binding Through the Global Challenge.

Remark 5.1 (An AIR Proof Alone Is Not Sound). *Consider the knowledge-soundness game of Definition 2.12 for one AIR argument, in which the adversary chooses the statement $(\boldsymbol{p}, \chi, s, \mathrm{com})$ and the proof. Since root_1 is not observed, the challenges v_c and ξ are functions of the global challenge and of the later commitments only. The adversary can choose the second-stage columns and the quotient, observe them, obtain ξ, and only then choose first-stage columns in $\mathbb{F}^{<N}[X]$ that take, at the points $\xi\omega^o$, values satisfying the constraint check. One field equation suffices, and a column over $\mathbb{F}$ read at fewer than $N/3$ points of $\mathbb{K}$ can be interpolated through any values. Every committed function is then a low-degree polynomial, so the low-degree test passes, while the trace need not satisfy any constraint. Knowledge soundness of $\Pi^{(\tau)}_{\mathrm{AIR}}$ in the sense of Definition 2.12 therefore fails, and what holds is the joint statement of Lemma 5.8.*

In ZisK the bus key is not chosen by the prover: it is $\chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c})$, expanded from the global challenge g, where $\boldsymbol{c} = \sum_i h(\tau_i, \mathrm{com}_i)$ accumulates the typed commitments of all the leaves, and $\mathrm{com}_i = \left(\mathrm{root}_{1,i}, \boldsymbol{a}_i^{(1)}\right)$ contains the first-stage root. Preimage awareness of the accumulator (Definition 2.14) fixes the commitments when the global challenge is first computed, and Merkle extractability fixes the oracles behind them. We capture this in a joint game.

Definition 5.7 (Joint AIR Extraction Game). For a PPT adversary $\mathcal{A}$ and an extractor $\mathcal{E}_{\mathrm{J}}$, the experiment $\mathsf{Exp}^{\mathsf{joint}}_{(\mathcal{E}_{\mathrm{J}},\mathcal{A})}(\lambda)$ is:

1. **Adversary.** $\mathcal{A}(1^\lambda)$ outputs a public statement $\boldsymbol{p}$ and, for $i \in [n]$ with $1 \le n < 2^B$, a type τ_i, a commitment com_i, a bus contribution s_i and an AIR proof π_i.
2. **Filter.** Let $\boldsymbol{c} := \sum_i h(\tau_i, \mathrm{com}_i)$ and $\chi := \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c})$, with global challenge g. If some π_i is rejected by the verifier of Protocol 5.4 for type τ_i, transcript seed g, statement $(\boldsymbol{p}, \chi, s_i)$ and first-stage commitment com_i, return 0.
3. **Extraction.** Run $(w_i)_{i\in[n]} \leftarrow \mathcal{E}_{\mathrm{J}}\big(\mathcal{T}, \boldsymbol{p}, (\tau_i, \mathrm{com}_i, s_i, \pi_i)_i\big)$, where $\mathcal{T}$ is the list of $\mathcal{A}$'s oracle queries.
4. **Winning condition.** Return 1 if and only if $((\boldsymbol{p}, \chi, s_i, \mathrm{com}_i); w_i) \notin \mathcal{R}^{(\tau_i)}_{\mathrm{AIR}}$ for some i.

We write $\mathsf{Adv}^{\mathsf{joint}}_{\mathcal{E}_{\mathrm{J}}}(\mathcal{A})$ for the probability that the experiment returns 1.

The game is the situation of the leaves of an accepting aggregation tree: the Final Verifier checks that χ was derived from $\boldsymbol{p}$ and from the accumulator of the leaves' typed commitments (Definition 3.9), and every leaf verifier checks an AIR proof under that key, directly or through a compressor (Remark 6.1).

Lemma 5.8 (Binding Through the Global Challenge). *For every chip type τ let ε^*_τ be the largest of the errors of Theorem 4.17, with $\varepsilon^{\mathrm{qry}}$ replaced by $\varepsilon^{\mathrm{qry}}_{\mathrm{BCS}}$ (Lemma 5.6), and let $\varepsilon^* := \max_\tau \varepsilon^*_\tau$ and $\Lambda_{\mathrm{pole}} := (2^B - 1) \cdot \max_\tau (N_\tau + 1)|\mathcal{I}_\tau|$. There is a straight-line extractor $\mathcal{E}_{\mathrm{J}}$ such that for every adversary $\mathcal{A}$ making at most Q oracle queries there is an adversary $\mathcal{D}$ against preimage awareness, with $\mathsf{Time}(\mathcal{D}) = \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$, and*

$$\mathsf{Adv}^{\mathsf{joint}}_{\mathcal{E}_{\mathrm{J}}}(\mathcal{A}) \le Q \cdot \varepsilon^* + \mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{D}) + \varepsilon_{\mathrm{MT}}(Q) + \frac{Q \cdot \Lambda_{\mathrm{pole}}}{|\mathbb{K}|}.$$

Proof. The extractor $\mathcal{E}_{\mathrm{J}}$ locates in $\mathcal{T}$ the first query to $\mathsf{DeriveKey}$ at $(\boldsymbol{p}, \boldsymbol{c})$, and runs the extractor $\mathcal{E}_{\mathcal{H}}$ of Definition 2.14 on the prefix of $\mathcal{T}$ before it, obtaining a multiset of typed commitments. It then reconstructs, as in Theorem 5.5, the Merkle trees of every $\mathrm{root}_{1,i}$, $\mathrm{root}_{2,i}$, quotient and FRI root of every proof, and for each i runs the extractor of Theorem 4.17 for type τ_i on the transcript of π_i preceding the query that flipped its state. It outputs the restriction (Lemma 4.15) of the expanded witnesses it obtains. We bound the probability that the experiment returns 1 by four events.

Accumulator. If the multiset extracted at the first query to $\mathsf{DeriveKey}$ differs from $\{(\tau_i, \mathrm{com}_i)\}_i$, then $\mathcal{D}$, which runs $\mathcal{A}$, announces $\boldsymbol{c}$ at that query and outputs the multiset, breaks preimage awareness. Otherwise every com_i, and with it $\mathrm{root}_{1,i}$ and $\boldsymbol{a}_i^{(1)}$, was fixed in $\mathcal{T}$ before g, and hence before every challenge of every AIR proof.

Merkle trees. Except with probability $\varepsilon_{\mathrm{MT}}(Q)$, every root in the proofs has a unique preimage in $\mathcal{T}$, computed before the root was first used as an oracle input, and every accepted opening agrees with it. Together with the previous event, the first-stage oracles of every proof are determined before g. The cached columns are determined by the root that the verifier reads from $\boldsymbol{p}$, which is an input of $\mathsf{DeriveKey}$, so they too are fixed before g.

Rounds. Under the two previous events, the transcript of π_i is a BCS transcript of Protocol 4.16 with instance $(\boldsymbol{p}, \chi, s_i)$. Its first message, the first- and second-stage oracles with $\boldsymbol{a}_i^{(1)}$ and $\boldsymbol{a}_i^{(2)}$, is fixed before v_c: the first stage before g, the second stage, $\boldsymbol{a}_i^{(2)}$ and s_i through the transcript, which observes them after g and before sampling v_c. The instance is fixed before v_c as well, since $\boldsymbol{p}$ is an input of the query that produced g, χ is determined by g, and s_i is observed. Every later message is observed before the challenge that follows it. Every query of $\mathcal{A}$ to the transcript oracle of any proof extends a partial transcript of one of the IOPs by one challenge, and the argument of Theorem 5.5, with Lemma 5.6 for the query round, shows that, except with probability $Q \cdot \varepsilon^*$, the extractor of Theorem 4.17 outputs an expanded witness of $\mathcal{R}_{\mathrm{STARK}}^{(\tau_i)}$ for every i.

Poles. The expanded witness contains the witness w_i of Lemma 4.15, which, under the first two events, was fixed in $\mathcal{T}$ before g. Every query to $\mathsf{DeriveKey}$ returns a fresh key, which is a pole of a fixed collection of at most $2^B - 1$ witnesses with probability at most $\Lambda_{\mathrm{pole}}/|\mathbb{K}|$ (Remark 4.1); a union bound over the Q queries bounds the probability that χ is a pole of some w_i.

Outside the four events, Lemma 4.15 gives $s_i = \mathrm{BusCont}_{\chi}\big(\mathcal{I}_{\tau_i}, w_i\big)$ and every constraint of $\mathcal{C}_{\tau_i}$; the cached columns open the commitment in $\boldsymbol{p}$, so $\mathrm{Local}_{\tau_i}(\boldsymbol{p}, w_i)$ holds; and $\mathsf{Commit}(w_i) = \mathrm{com}_i$ because the extracted first-stage oracles are the preimage of $\mathrm{root}_{1,i}$. Hence $((\boldsymbol{p}, \chi, s_i, \mathrm{com}_i); w_i) \in \mathcal{R}_{\mathrm{AIR}}^{(\tau_i)}$ for every i. □

The extractor $\mathcal{E}_{\mathrm{J}}$ uses the query list of the whole execution and the preimage-awareness extractor; its output is a collection of witnesses, not a witness per proof. This is the form in which the AIR arguments are used in Section 8: the witnesses of all the leaves are extracted at once.

6. Knowledge Soundness of the Arguments

This section states the knowledge soundness of every argument of ZisK. The analysis of Section 4 and the compilation of Section 5 apply to the AIR of every chip and of every recursive verifier. Two further ingredients connect them to the relations of Section 3.3: the assumption that the AIR of every recursive verifier enforces its relation, which turns a satisfying witness of that AIR into a witness of the relation, and the compressor lemma, which shows that a compressor adds one term to the error.

6.1. Statement.

For an argument Π of ZisK we write ε_{Π}^* for the largest of the round-by-round errors of Theorem 4.17 for its AIR, with $\varepsilon^{\mathrm{qry}}$ replaced by $\varepsilon_{\mathrm{BCS}}^{\mathrm{qry}}$ (Lemma 5.6), and ε^* for the largest of the $\varepsilon_{\Pi_{\mathrm{AIR}}^{(\tau)}}^*$ over $\tau \in [M]$. The AIR arguments and the recursive arguments are treated separately: an AIR proof is not knowledge-sound on its own (Remark 5.1), and its guarantee is a joint one.

The recursive arguments prove that the AIR of a recursive verifier is satisfied, while the relations of Section 3.3 state what that verifier checks. Relating the two is a property of how the verifier is arithmetized, not of the proof system, and this paper assumes it.

Definition 6.1 (Enforcing a Relation). The AIR $\mathcal{C}_X$ of a recursive verifier *enforces* the relation $\mathcal{R}_X$ if, for every $\mathbb{x}$, every witness that satisfies the constraints of $\mathcal{C}_X$ with publics $\mathbb{x}$ contains a $\mathbb{w}$ with $(\mathbb{x}; \mathbb{w}) \in \mathcal{R}_X$.

Theorem 6.2 (Knowledge Soundness of the Arguments). *Let Q bound the number of oracle queries of the adversary.*

1. *(*AIR arguments.*) There is a straight-line extractor $\mathcal{E}_{\mathrm{J}}$ that extracts the witnesses of all the AIR proofs of an execution at once: for every adversary $\mathcal{A}$ there is an adversary $\mathcal{D}$*

against preimage awareness, with $\mathsf{Time}(\mathcal{D}) = \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$*, such that, with* Λ_{pole} *as in Lemma 5.8,*

$$\mathsf{Adv}^{\mathsf{joint}}_{\mathcal{E}_{\mathrm{J}}}(\mathcal{A}) \le Q \cdot \varepsilon^* + \mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{D}) + \varepsilon_{\mathrm{MT}}(Q) + \frac{Q \cdot \Lambda_{\mathrm{pole}}}{|\mathbb{K}|}.$$

2. *(*Recursive arguments.*) Let* X *be* $\mathrm{LEAF}^{(\tau)}$ *for some* $\tau \in [M]$*,* AGG*,* FINAL*, or a compressor, and assume that the AIR* $\mathcal{C}_X$ *of its verifier enforces* $\mathcal{R}_X$ *(Definition 6.1). Then* Π_X *is knowledge-sound for* $\mathcal{R}_X$ *with a straight-line extractor, and for every adversary* $\mathcal{A}$

$$\mathsf{Adv}^{\mathsf{ks}}_{\Pi_X}(\mathcal{A}) \le Q \cdot \varepsilon^*_{\Pi_X} + \varepsilon_{\mathrm{MT}}(Q).$$

Proof. The first statement is Lemma 5.8. For the second, Π_X is the BCS compilation of Protocol 4.16 for $\mathcal{C}_X$ by Protocol 5.4, whose transcript observes every message before the challenge that depends on it (Table 3). Theorem 5.5, with the round-by-round errors of Theorem 4.17 and the query round refined by Lemma 5.6, gives a straight-line extractor that, except with probability $Q \cdot \varepsilon^*_{\Pi_X} + \varepsilon_{\mathrm{MT}}(Q)$, outputs an expanded witness of the IOP relation for $\mathcal{C}_X$. Its restriction to the columns of $\mathcal{C}_X$ satisfies the constraints of $\mathcal{C}_X$ with publics $\mathbb{x}$ (Lemma 4.15, for an AIR without interactions), and by hypothesis contains a $\mathbb{w}$ with $(\mathbb{x}; \mathbb{w}) \in \mathcal{R}_X$, which the extractor outputs. □

A compressor of an argument Π_X (Section 3.1) proves the relation $\mathcal{R}^{(X)}_{\mathrm{COMP}}$ of Definition 3.4, and its proofs stand in for those of Π_X. Composing the two arguments preserves knowledge soundness.

Lemma 6.3 (Compressors Preserve the Relation). *Let* Π_X *be knowledge-sound for* $\mathcal{R}_X$ *with extractor* $\mathcal{E}_X$*, and* Π_{COMP} *knowledge-sound for* $\mathcal{R}^{(X)}_{\mathrm{COMP}}$ *with extractor* $\mathcal{E}_{\mathrm{COMP}}$*. The argument whose proofs for* $\mathbb{x}$ *are the proofs of* Π_{COMP} *for* $\mathbb{x}$ *is knowledge-sound for* $\mathcal{R}_X$ *with the extractor* $\mathcal{E}_X(\mathbb{x}, \mathcal{E}_{\mathrm{COMP}}(\mathbb{x}, \cdot))$*, and for every adversary* $\mathcal{A}$ *there are reductions* $\mathcal{D}_1, \mathcal{D}_2$ *running in time at most* $2 \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ *with*

$$\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{COMP}} \circ \Pi_X}(\mathcal{A}) \le \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{COMP}}}(\mathcal{D}_1) + \mathsf{Adv}^{\mathsf{ks}}_{\Pi_X}(\mathcal{D}_2).$$

Proof. If the compressor proof is accepted, then except with probability $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{COMP}}}(\mathcal{D}_1)$, where $\mathcal{D}_1 := \mathcal{A}$, the extracted π satisfies $\mathbf{V}(\mathsf{vk}_X, \mathbb{x}, \pi) = 1$. The adversary $\mathcal{D}_2$ runs $\mathcal{A}$ and $\mathcal{E}_{\mathrm{COMP}}$ and outputs $(\mathbb{x}, \pi)$; it wins the knowledge-soundness game of Π_X exactly when π is accepted and $\mathcal{E}_X$ fails. Its running time is that of $\mathcal{A}$ plus that of $\mathcal{E}_{\mathrm{COMP}}$, at most $2 \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ by Definition 2.12. □

The lemma composes straight-line extractors and is subject to the first idealization of Remark 2.1. A compressor adds one term to the error, and its own parameters, rate and number of queries, enter only that term. ZisK compresses only AIR proofs, the case of the following remark.

Remark 6.1 (Compressors of AIR Proofs). *Lemma 6.3 does not apply verbatim to a compressor of an AIR proof, since* $\Pi^{(\tau)}_{\mathrm{AIR}}$ *is not knowledge-sound on its own. Its proof, however, uses the knowledge soundness of* Π_X *only on the proof* π *that the compressor's extractor outputs, which is accepting and occurs in the adversary's query list. Extracting the compressed AIR proofs first and then applying Lemma 5.8 to the whole collection gives the first statement of Theorem 6.2 with one more term* $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{COMP}}}$ *for each compressor argument.*

6.2. Concrete Parameters.

The parameters of every argument of ZisK, its rate ρ, number of queries q, grinding bits g_{pow}, folding factors and constraint degree, are those of the proving backend [Zis26]. In the Johnson regime (Corollary 4.19), every error of Theorem 4.17 except the query error is, for a fixed η, a polynomial in $|D|$ divided by $|\mathbb{K}| \approx 2^{192}$, hence negligible for the trace heights of ZisK, and the query error of the compiled protocol is, by Lemma 5.6,

$$\varepsilon_{\mathrm{BCS}}^{\mathrm{qry}} \approx 2^{-g_{\mathrm{pow}}} \cdot \left(\sqrt{\rho^{+}} + \eta\right)^{q}.$$

Table 4 lists, for every family of arguments, the limit $g_{\mathrm{pow}} + q \cdot \log_2(1/\sqrt{\rho})$ of $-\log_2 \varepsilon_{\mathrm{BCS}}^{\mathrm{qry}}$ as $\eta \to 0$ and $\rho^{+} \to \rho$, and the level reported by `soundcalc` [Eth26], which fixes η and accounts for every round. The other errors grow as η decreases, the list size as η^{-1} and $\varepsilon^{\mathrm{mca}}$ as η^{-5}, so η cannot be taken to 0; this is the gap between the two columns.

Arguments	$\boldsymbol{\rho}$	$\boldsymbol{q}$	$\boldsymbol{g_{\mathrm{pow}}}$	$\boldsymbol{g_{\mathrm{pow}} + q \log_2 \rho^{-1/2}}$	**soundcalc**
AIRs (except below)	1/2	229–232	16	130.5–132	128
Rom	1/2	221	20	130.5	128
Keccakf	1/2	217	23	131.5	128
Poseidon2	1/4	114	16	130	128
Compressors	1/4	110	20	130	128
Leaf and aggregation nodes	1/8	73	20	129.5	128
Final Verifier	1/32	43	22	129.5	128

TABLE 4. Security level, in bits, of the query round of every argument of ZisK in the Johnson regime: the idealized bound of Lemma 5.6 and the level reported by `soundcalc`.

These are errors per query. By Theorem 5.5, an adversary making Q queries gains a factor Q, so a level of 128 bits bounds its success probability by $Q \cdot 2^{-128}$.

Part II: Recursion

We prove that the ZisK aggregation tree is *correct-witness extractable*: any adversary that produces an accepting final proof for an external statement $\boldsymbol{p}_{\text{ext}}$ can be used to extract, in a straight-line fashion, proof values $\boldsymbol{p}_{\text{pv}}$ and a typed witness collection L such that every witness satisfies its AIR and is consistent with $\boldsymbol{p} = (\boldsymbol{p}_{\text{ext}}, \boldsymbol{p}_{\text{pv}})$, and every bus balances across the whole collection together with the global messages of $\boldsymbol{p}$. The proof descends the aggregation tree of Section 3.1 layer by layer, reducing everything to the guarantees of the argument systems of ZisK established in Part I (Theorem 6.2), the straight-line extractability of the trace commitment Com, the preimage awareness of the accumulator $\mathcal{H}$, and the soundness of the bus fingerprint.

This part follows the reference architecture of the EF zkVM security sprint [Kad+26] in structure as well as in method: a bridging proposition, one extraction game and lemma per layer of the recursion, and a main theorem whose bound is an explicit sum over those lemmas; the idealized-model remark and the definitions of the assumptions are in Section 2. Where ZisK departs from that architecture, the departure is flagged at the point where it matters.

Remark (Scope). *The target relation of this part speaks about AIR witnesses, not about guest executions: it is the recursion layer of the chain that leads from an accepting final proof to a correct execution of the guest program. The semantic layer (that a balanced collection of locally satisfying witnesses encodes a single execution of the committed program, including the memory argument and continuation stitching) is Part III; completeness, zero knowledge and any wrapper placed above the Final Verifier are outside the analysis (Section 17).*

7. From the Root Check to Bus Balancing

We begin with the root check. The Final Verifier checks a single field equation, $s + \mathcal{F}_\chi(\mathcal{G}(\boldsymbol{p})) = 0$, on a value accumulated from the leaves under a challenge that the tree itself certifies to be derived from the statement and the leaves' commitments. By additivity of the fingerprint the left-hand side is $\mathcal{F}_\chi(\mathcal{M}_L + \mathcal{G}(\boldsymbol{p}))$ for the extracted collection L, so the check is a fingerprint zero test on the signed multiplicity of L together with the global messages, whereas the target relation demands $\mathcal{M}_L + \mathcal{G}(\boldsymbol{p}) = 0$. This section bridges the gap. We first record that LogUp is a bus fingerprint in the sense of Definition 2.8, then that a zero test at a random challenge certifies balance, and finally that the challenge the tree certifies is random enough.

Recall ZisK's fingerprint from Section 3.4. A key is a challenge pair $\chi = (\alpha, \gamma) \in \mathbb{K}^2$, a message (b, v) is compressed to $\hat{e}_\gamma(b, v) = b + \sum_{j=1}^{\ell_b} \gamma^j v_j \in \mathbb{K}$, and

$$\mathcal{F}_\chi^{\text{LogUp}}(D) = \sum_{(b,v) \in \text{ supp } D} \frac{D(b, v)}{\alpha + \hat{e}_\gamma(b, v)},$$

with $\mathcal{F}_\chi^{\text{LogUp}}(D) := \perp$ if some denominator vanishes.

Lemma 7.1 (LogUp Fingerprint). *$\mathcal{F}^{\text{LogUp}}$ is a statistically sound bus fingerprint, with key space $\mathbb{K}^2$, values in $\mathbb{K}$ and every nonzero multiset admissible, with error*

$$\varepsilon_{\mathcal{F}}(\Lambda) = \frac{\ell \cdot \binom{\Lambda}{2} + \max(\Lambda - 1, 0)}{|\mathbb{K}|}.$$

Proof. Additivity. $\mathcal{F}_\chi^{\text{LogUp}}(0) = 0$ is the empty sum. If $\mathcal{F}_\chi^{\text{LogUp}}(D)$ and $\mathcal{F}_\chi^{\text{LogUp}}(D')$ are both defined, no message of supp $D \cup$ supp D' is a pole; the support of $D + D'$ is contained in that union, and the sum defining $\mathcal{F}_\chi^{\text{LogUp}}(D + D')$ splits message by message.

Soundness. If $\Lambda = 0$, there is no admissible nonzero multiset, so the advantage and the stated error are both zero. Assume $\Lambda \geq 1$. The key is drawn after the adversary has output its multiset, so it suffices to fix an arbitrary $D \neq 0$ with $|D| \leq \Lambda$ and to bound the probability over the key alone. Let $\chi = (\alpha, \gamma)$ be uniform in $\mathbb{K}^2$ and write $z_\gamma(b, v) := \hat{e}_\gamma(b, v)$ for $(b, v) \in \operatorname{supp} D$.

Compression collisions. Let E_γ be the event that $z_\gamma(b, v) = z_\gamma(b', v')$ for two distinct messages in $\operatorname{supp} D$. For a fixed pair of messages the difference is a polynomial in γ of degree at most ℓ, and it is nonzero: if $v \neq v'$ some if $b \neq b'$ the constant term $b - b'$ is nonzero, and if $b = b'$ then $\ell_b = \ell_{b'}$ and $v \neq v'$, so some coefficient of a positive power of γ is nonzero. A uniform γ is a root with probability at most $\ell/|\mathbb{K}|$, so $\Pr[E_\gamma] \leq \binom{\Lambda}{2} \cdot \ell/|\mathbb{K}|$.

Zero test. Fix γ with $\neg E_\gamma$, so the values $z_\gamma(b, v)$ are pairwise distinct, and define

$$P(X) := \sum_{(b,v) \in \operatorname{supp} D} D(b, v) \cdot \prod_{(b',v') \in \operatorname{supp} D \setminus \{(b,v)\}} \big(X + z_\gamma(b', v')\big) \in \mathbb{K}[X].$$

For any $(b_0, v_0) \in \operatorname{supp} D$,

$$P\big(-z_\gamma(b_0, v_0)\big) = D(b_0, v_0) \cdot \prod_{(b',v') \neq (b_0,v_0)} \big(z_\gamma(b', v') - z_\gamma(b_0, v_0)\big) \neq 0$$

because all factors are nonzero. Hence $P \neq 0$ and $\deg(P) \leq |D| - 1 \leq \Lambda - 1$. If $\mathcal{F}_\chi^{\mathrm{LogUp}}(D) = 0$, then all denominators are nonzero and multiplying through by their product gives $P(\alpha) = 0$, so α is a root of P. As α is uniform and independent of γ, this has probability at most $(\Lambda - 1)/|\mathbb{K}|$. A union bound over the two events gives the claim, and the bound is non-decreasing in Λ. □

Lemma 7.2 (Zero Test). *Let $\mathcal{F}$ be a statistically sound bus fingerprint with key space $\mathcal{X}$ that admits every nonzero multiset, let $\boldsymbol{p} \in \mathcal{P}$, let L be a typed witness collection with $\neg\mathrm{Balanced}_{\boldsymbol{p}}(L)$, and let χ be uniform in $\mathcal{X}$. Then*

$$\Pr_\chi\left[\sum_{i=1}^{|L|} \mathrm{BusCont}_\chi\big(\mathcal{I}_{\tau_i}, w_i\big) + \mathcal{F}_\chi(\mathcal{G}(\boldsymbol{p})) = 0\right] \leq \varepsilon_{\mathcal{F}}(\Lambda(L) + g).$$

Proof. If the sum is 0, every summand is defined, so by additivity the sum equals $\mathcal{F}_\chi(\mathcal{M}_L + \mathcal{G}(\boldsymbol{p}))$. Since L is unbalanced for $\boldsymbol{p}$, $\mathcal{M}_L + \mathcal{G}(\boldsymbol{p}) \neq 0$, and its support has at most $\Lambda(L) + g$ elements (Definition 2.10), so the soundness of $\mathcal{F}$ bounds the probability by $\varepsilon_{\mathcal{F}}(\Lambda(L) + g)$. □

The zero test is exactly where the field-valued definition of Balanced enters: it certifies that every signed multiplicity vanishes in $\mathbb{F}$, no more. We now define the game that captures what the root checks certify about an extracted collection, and bound it.

Definition 7.3 (Bus-Argument Soundness Game). For an adversary $\mathcal{A}$ with access to the random oracles, the experiment $\mathsf{Exp}^{\mathsf{bus}}_{\mathcal{A}}(\lambda)$ proceeds as follows.

1. **Adversary.** $\mathcal{A}(1^\lambda)$ outputs a public statement $\boldsymbol{p} \in \mathcal{P}$, a typed witness collection $L = ((\tau_i, w_i))_{i \in [|L|]}$ and commitments $(\mathrm{com}_i)_{i \in [|L|]}$.
2. **Derivation.** Set $\boldsymbol{c} := \sum_{i=1}^{|L|} h(\tau_i, \mathrm{com}_i) \in \mathbb{Z}_p^n$ and $\chi := \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c})$.
3. **Winning condition.** Return 1 if and only if $1 \leq |L| < 2^B$; $\mathsf{Commit}(w_i) = \mathrm{com}_i$ for every i; $\sum_{i=1}^{|L|} \mathrm{BusCont}_\chi\big(\mathcal{I}_{\tau_i}, w_i\big) + \mathcal{F}_\chi(\mathcal{G}(\boldsymbol{p})) = 0$; and $\neg\mathrm{Balanced}_{\boldsymbol{p}}(L)$.

We write $\mathsf{Adv}^{\mathsf{bus}}(\mathcal{A}) := \Pr\big[\mathsf{Exp}^{\mathsf{bus}}_{\mathcal{A}}(\lambda) = 1\big]$.

Substituting $s_i := \mathrm{BusCont}_\chi\big(\mathcal{I}_{\tau_i}, w_i\big)$ and $\boldsymbol{c}_i := h(\tau_i, \mathrm{com}_i)$ shows that the winning condition is precisely "the root checks $s + \mathcal{F}_\chi(\mathcal{G}(\boldsymbol{p})) = 0$ and $\chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c})$ pass on values honestly

computed from L, yet L is unbalanced for $\boldsymbol{p}$". This is the form in which the game is invoked in the proof of Theorem 9.5.

Proposition 7.4 (Bus Soundness). *Let $\mathcal{F}$ be the bus fingerprint of the system, and let $\mathcal{A}$ be a PPT adversary making at most Q random-oracle queries. Under the idealizations of Remark 2.1:*

1. *There are reduction adversaries $\mathcal{D}^{\mathrm{pa}}$ and $\mathcal{D}^{\mathrm{ext}}$, each running $\mathcal{A}$ once in time $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$, such that*
$$\mathsf{Adv}^{\mathsf{bus}}(\mathcal{A}) \le (Q+1)\cdot \varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{pa}}) + \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D}^{\mathrm{ext}}).$$
2. *Let t^* be the first query of $\mathcal{A}$ to $\mathsf{DeriveKey}$ at $(\boldsymbol{p},\boldsymbol{c})$, or the game's derivation if there is none. The probability that $\mathcal{A}$ wins while the extractors of $\mathcal{H}$ and of Com, run on the queries made before t^*, return the multiset $\{(\tau_i,\mathrm{com}_i)\}_i$ and the witnesses w_i, respectively, is at most $(Q+1)\cdot\varepsilon_{\mathcal{F}}(\Lambda_{\max})$.*

Proof. Let $\big(\boldsymbol{p}, L, (\mathrm{com}_i)_i\big)$ be $\mathcal{A}$'s output and $\boldsymbol{c}$, χ as in the game, and let $\mathcal{S} := \{(\tau_i,\mathrm{com}_i)\}_i$ be the multiset of typed commitments of the output, so that $\boldsymbol{c} = \mathcal{H}(\mathcal{S})$. Number the $\mathsf{DeriveKey}$ queries in the order they are made, counting only the first query at each point (a repeated query returns the answer already given), and let $t^* \in [Q+1]$ be the index of the query at $(\boldsymbol{p},\boldsymbol{c})$, with $t^* := Q+1$ if $\mathcal{A}$ never queried it; in the latter case the game's own derivation plays the role of a final, fresh query. For $t \in [Q+1]$ write $(\boldsymbol{p}_t,\boldsymbol{c}_t)$ for the t-th query point, χ_t for its answer and $\mathcal{T}_{<t}$ for the transcript of all oracle queries and answers made before it. We define bad events, bound each by a reduction to one of the assumptions or by Lemma 7.2, and show that off all of them the adversary cannot win.

Bad events.

- E_{pa}: $|L| < 2^B$ and $\mathcal{E}_{\mathcal{H}}(\mathcal{T}_{<t^*},\boldsymbol{c}) \neq \mathcal{S}$. The reduction $\mathcal{D}^{\mathrm{pa}}$ runs $\mathcal{A}$, announces every $\mathsf{DeriveKey}$ query point at the time of the query, and outputs $\mathcal{S}$. Its announcement at time t^* is $\boldsymbol{c}$ (if $t^* = Q+1$, the game's derivation is that announcement), so E_{pa} is its winning event of Definition 2.14 for that announcement and $\Pr\big[E_{\mathrm{pa}}\big] \le \mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{pa}})$.
- E_{ext}: $\neg E_{\mathrm{pa}}$ and $\mathcal{E}_{\mathsf{Com}}(\mathcal{T}_{<t^*},\mathrm{com}_i) \neq w_i$ for some i. Off E_{pa} the extracted multiset at time t^* is $\mathcal{S}$, so this is the failure event of Definition 2.13 for the adversary $\mathcal{D}^{\mathrm{ext}}$ that runs $\mathcal{A}$, at every $\mathsf{DeriveKey}$ query t announces the commitments of $\mathcal{E}_{\mathcal{H}}(\mathcal{T}_{<t},\boldsymbol{c}_t)$, and outputs $(\mathrm{com}_i,w_i)_i$; its announcement at time t^* contains every com_i. Hence $\Pr[E_{\mathrm{ext}}] \le \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D}^{\mathrm{ext}})$.
- $E^{(t)}_{\chi}$ for $t \in [Q+1]$: define the *candidate collection* L^*_t as follows. If $\mathcal{E}_{\mathcal{H}}(\mathcal{T}_{<t},\boldsymbol{c}_t)$ returns a multiset $\mathcal{S}_t$, let $L^*_t := ((\tau,\mathcal{E}_{\mathsf{Com}}(\mathcal{T}_{<t},\mathrm{com})))_{(\tau,\,\mathrm{com})\in\mathcal{S}_t}$ in a canonical order, discarding the collection if some extraction returns $\perp$; otherwise $L^*_t := \perp$. The event $E^{(t)}_{\chi}$ is that $L^*_t \neq \perp$, $|L^*_t| < 2^B$, $\neg\mathrm{Balanced}_{\boldsymbol{p}_t}(L^*_t)$ and $\sum_i \mathrm{BusCont}_{\chi_t}\big(\mathcal{I}_{\tau_i},w_i\big) + \mathcal{F}_{\chi_t}(\mathcal{G}(\boldsymbol{p}_t)) = 0$ over the entries of L^*_t. The candidate is a function of $\mathcal{T}_{<t}$ alone, whereas χ_t is uniform in $\mathbb{K}^2$ and independent of $\mathcal{T}_{<t}$ because t is the first query at its point; since $|L^*_t| < 2^B$ implies $\Lambda(L^*_t)+g \le \Lambda_{\max}$, Lemma 7.2 gives $\Pr\big[E^{(t)}_{\chi}\big] \le \varepsilon_{\mathcal{F}}(\Lambda_{\max})$.

Step A: off all events, the adversary does not win. Suppose $\mathcal{A}$ wins and none of E_{pa}, E_{ext}, $E^{(t^*)}_{\chi}$ occurs. The win gives $|L| < 2^B$, so off E_{pa}, $\mathcal{E}_{\mathcal{H}}(\mathcal{T}_{<t^*},\boldsymbol{c}_{t^*}) = \mathcal{S}$. Off E_{ext}, $\mathcal{E}_{\mathsf{Com}}(\mathcal{T}_{<t^*},\mathrm{com}_i) = w_i$ for every i, so $L^*_{t^*}$ is a reordering of L; every quantity in the winning condition is invariant under reordering. The winning condition then says $\neg\mathrm{Balanced}_{\boldsymbol{p}}(L^*_{t^*})$ and $\sum_i \mathrm{BusCont}_{\chi_{t^*}}\big(\mathcal{I}_{\tau_i},w_i\big) + \mathcal{F}_{\chi_{t^*}}(\mathcal{G}(\boldsymbol{p})) = 0$ with $\chi = \chi_{t^*}$ and $\boldsymbol{p} = \boldsymbol{p}_{t^*}$, which is $E^{(t^*)}_{\chi}$: a contradiction.

Conclusion. By Step A a win implies one of the bad events, and a union bound over E_{pa}, E_{ext} and $E^{(1)}_{\chi},\ldots,E^{(Q+1)}_{\chi}$ gives the first statement. The condition of the second statement excludes E_{pa} and E_{ext}, so a win under it implies one of $E^{(1)}_{\chi},\ldots,E^{(Q+1)}_{\chi}$, which gives the second. Both reductions run $\mathcal{A}$ once and perform a scan of the transcript, so they run in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. □

Remark 7.1 (Removing the Idealization). *Without preimage awareness of $\mathcal{H}$, the second idealization of Remark 2.1, the event E_{pa} must be handled by rewinding, using only the collision resistance of $\mathcal{H}$ on multisets of cardinality below 2^B, with advantage $\mathsf{Adv}^{\mathsf{cr}}_{\mathcal{H}}(\mathcal{A}) := \Pr\big[(\mathcal{S},\mathcal{S}') \leftarrow \mathcal{A} : \mathcal{S} \neq \mathcal{S}' \wedge |\mathcal{S}|, |\mathcal{S}'| < 2^B \wedge \mathcal{H}(\mathcal{S}) = \mathcal{H}(\mathcal{S}')\big]$, which for LtHash reduces to the short-integer-solution problem. Run $\mathcal{A}$ a second time from its state just before the query t^*, answering it with a fresh χ'. If both runs win with the same accumulator and the output collections differ, the two multisets (or two openings of one commitment) form a collision for $\mathcal{H}$ or a break of Definition 2.13; if they coincide, a single unbalanced collection has passed the zero test at two independent challenges, which Lemma 7.2 bounds. The general forking lemma [BN06] then yields, schematically,*

$$\mathsf{Adv}^{\mathsf{bus}}(\mathcal{A}) \leq \varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \sqrt{(Q+1)\big(\varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \mathsf{Adv}^{\mathsf{cr}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{cr}}) + \mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}(\mathcal{D}^{\mathrm{ext}})\big)} + \frac{Q+1}{|\mathbb{K}|},$$

with reductions that run $\mathcal{A}$ twice. The square-root loss is inherent to rewinding at a Fiat–Shamir query [FS87] and is far from concretely tight. Making the linear bound of Proposition 7.4 rigorous is a proof obligation for the full security argument: either preimage awareness of LtHash as a knowledge-type assumption (an efficient extractor that recovers the multiset from an accumulator the adversary produced), or an accumulator that is preimage aware in the random-oracle model by construction, at the cost of the homomorphism that makes independent aggregation possible.

Remark 7.2 (Field Versus Integer Balance). *Proposition 7.4 delivers* $\mathrm{Balanced}_{\boldsymbol{p}}(L)$ *with multiplicities in $\mathbb{F}$. To conclude that every message is sent and received the same number of times as integers one needs, in addition, that every multiplicity column of every AIR is range-checked to $[0, m_{\max}]$ (or to a symmetric interval with a fixed signed encoding) and that $\Lambda_{\max} \cdot m_{\max} < p$, so that vanishing in $\mathbb{F}$ lifts to vanishing in $\mathbb{Z}$. For the buses whose multiplicities are 0, 1 and -1, Lemma 11.2 needs only $\Lambda_{\max} < p$; the other buses need only the weaker statement that a received message is also sent.*

8. Descending the Aggregation Tree

We state one extraction game and one lemma per layer of the aggregation tree, proceeding from the Final Verifier inward to the compressors. Each lemma composes the straight-line extractors guaranteed by Definition 2.12. The AIR layer has no lemma of its own: it is the joint extraction of Part I (Theorem 6.2).

Advantage notation. We collect the advantages introduced above: $\mathsf{Adv}^{\mathsf{ks}}_{\Pi}$ (knowledge-soundness error, Definition 2.12), $\mathsf{Adv}^{\mathsf{ext}}_{\mathsf{Com}}$ (Definition 2.13), $\mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}$ (Definition 2.14), $\mathsf{Adv}^{\mathsf{joint}}$ (Definition 5.7) and $\mathsf{Adv}^{\mathsf{bus}}$ (Definition 7.3). All advantages are functions of λ. In each lemma we exhibit explicit reduction adversaries $\mathcal{D}$ and state their running times; when a bound has the form $k \cdot \mathsf{Adv}^{\cdot}(\mathcal{D})$, the reduction $\mathcal{D}$ denotes the maximum-advantage adversary among the k per-node reductions constructed in the proof, whose running time the lemma states. Probabilities in the games below are over the coins of the adversary, the extractor and the random oracles; verification is deterministic.

Final layer. The descent starts from the Final Verifier's proof.

Definition 8.1 (Final Extraction Game). For an extractor $\mathcal{E}_{\mathrm{FIN}}$ and a PPT adversary $\mathcal{A}$, the experiment $\mathsf{Exp}^{\mathsf{fin}}_{(\mathcal{E}_{\mathrm{FIN}},\mathcal{A})}(\lambda)$ is:

1. **Adversary.** $\mathcal{A}(1^\lambda)$ outputs an external statement $\boldsymbol{p}_{\mathrm{ext}} \in \mathcal{P}_{\mathrm{ext}}$ and a proof π_{FIN}.
2. **Filter.** If $\mathbf{V}_{\mathrm{FINAL}}(\boldsymbol{p}_{\mathrm{ext}}, \pi_{\mathrm{FIN}}) \neq 1$, return 0.
3. **Extraction.** Run $\big(\boldsymbol{p}_{\mathrm{pv}}, \chi, \sigma, \pi\big) \leftarrow \mathcal{E}_{\mathrm{FIN}}(\boldsymbol{p}_{\mathrm{ext}}, \pi_{\mathrm{FIN}})$, set $\boldsymbol{p} := \big(\boldsymbol{p}_{\mathrm{ext}}, \boldsymbol{p}_{\mathrm{pv}}\big)$, with $\sigma = (\tau, \kappa, \boldsymbol{c}, s)$.

4. **Winning condition.** Return 1 if and only if $\big(\boldsymbol{p}_{\text{ext}}; \big(\boldsymbol{p}_{\text{pv}}, \chi, \sigma, \pi\big)\big) \notin \mathcal{R}_{\text{FINAL}}$, i.e. unless $\mathsf{Typed}(\tau, \kappa)$, $\mathbf{V}^{(\tau)}_{\text{REC}}[\mathsf{vk}_{\text{AGG}}]((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}_{\text{AGG}}), \pi) = 1$, $s + \mathcal{F}_{\chi}(\mathcal{G}(\boldsymbol{p})) = 0$ and $\chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c})$.

We write $\mathsf{Adv}^{\mathsf{fin}}_{\mathcal{E}_{\text{FIN}}}(\mathcal{A})$ for the probability that the experiment $\mathsf{Exp}^{\mathsf{fin}}_{(\mathcal{E}_{\text{FIN}}, \mathcal{A})}(\lambda)$ returns 1.

Lemma 8.2 (Final Extraction). *Assume Π_{FINAL} is knowledge-sound and let $\mathcal{E}_{\text{FIN}} := \mathcal{E}_{\Pi_{\text{FINAL}}}$ be its extractor. For every PPT adversary $\mathcal{A}$ there is a reduction $\mathcal{D}_{\text{FIN}}$ with $\mathsf{Time}(\mathcal{D}_{\text{FIN}}) = \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ and*

$$\mathsf{Adv}^{\mathsf{fin}}_{\mathcal{E}_{\text{FIN}}}(\mathcal{A}) = \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\text{FINAL}}}(\mathcal{D}_{\text{FIN}}).$$

Proof. $\mathcal{D}_{\text{FIN}}$ runs $\mathcal{A}(1^{\lambda})$ and forwards $(\boldsymbol{p}_{\text{ext}}, \pi_{\text{FIN}})$ to the knowledge-soundness challenger of Π_{FINAL}. The challenger applies the same filter and runs the same extractor, and Definition 8.1 returns 1 exactly when π_{FIN} verifies but $\big(\boldsymbol{p}_{\text{ext}}; \big(\boldsymbol{p}_{\text{pv}}, \chi, \sigma, \pi\big)\big) \notin \mathcal{R}_{\text{FINAL}}$, which is the event that $\mathcal{D}_{\text{FIN}}$ wins. Since Π^{*}, the final argument system of Definition 9.3, and Π_{FINAL} are the same algorithm, this is the base of the composition and no additional work is required. □

Aggregation layer. The aggregation extractor unrolls the tree below the Final Verifier's child until it reaches leaf-verifier proofs. The counter κ of the child statement tells it in advance how many leaves to expect, and it guarantees that it never has to descend further than $\kappa - 1$ aggregation nodes.

Definition 8.3 (Aggregation Extraction Game). For a counter $\kappa \in [1, 2^{B})$, an extractor $\mathcal{E}_{\text{REC}}$ and a PPT adversary $\mathcal{A}$, the experiment $\mathsf{Exp}^{\mathsf{agg}}_{(\kappa, \mathcal{E}_{\text{REC}}, \mathcal{A})}(\lambda)$ is:

1. **Adversary.** $\mathcal{A}(1^{\lambda})$ outputs $(\boldsymbol{p}, \chi, \sigma, \pi)$ with $\sigma = (\tau, \kappa, \boldsymbol{c}, s)$.
2. **Filter.** If $\neg\, \mathsf{Typed}(\tau, \kappa)$ or $\mathbf{V}^{(\tau)}_{\text{REC}}[\mathsf{vk}_{\text{AGG}}]((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}_{\text{AGG}}), \pi) \neq 1$, return 0.
3. **Extraction.** Run $((\sigma_1, \pi_1), ..., (\sigma_{\kappa}, \pi_{\kappa})) \leftarrow \mathcal{E}_{\text{REC}}(\boldsymbol{p}, \chi, \sigma, \pi)$, with $\sigma_i = (\tau_i, \kappa_i, \boldsymbol{c}_i, s_i)$.
4. **Winning condition.** Return 1 unless, for every $i \in [\kappa]$, $\tau_i \in [M]$, $\kappa_i = 1$ and $\mathbf{V}^{(\tau_i)}_{\text{LEAF}}((\boldsymbol{p}, \chi, \sigma_i, \mathsf{vk}_{\text{AGG}}), \pi_i) = 1$, and moreover $\sum_{i=1}^{\kappa} s_i = s$ and $\sum_{i=1}^{\kappa} \boldsymbol{c}_i = \boldsymbol{c}$.

We write $\mathsf{Adv}^{\mathsf{agg}}_{(\kappa, \mathcal{E}_{\text{REC}})}(\mathcal{A})$ for the probability that the experiment returns 1.

Lemma 8.4 (Aggregation Tree Unrolling). *Assume Π_{AGG} is knowledge-sound and let $\kappa \in [1, 2^{B})$. There is an extractor $\mathcal{E}_{\text{REC}}$, given in the proof, such that for every PPT adversary $\mathcal{A}$ there is a reduction $\mathcal{D}_{\text{AGG}}$ running in $\kappa \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ with*

$$\mathsf{Adv}^{\mathsf{agg}}_{(\kappa, \mathcal{E}_{\text{REC}})}(\mathcal{A}) \leq (\kappa - 1) \cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\text{AGG}}}(\mathcal{D}_{\text{AGG}}).$$

$\mathcal{E}_{\text{REC}}$ makes at most $\kappa - 1$ calls to $\mathcal{E}_{\Pi_{\text{AGG}}}$ and runs in $(\kappa - 1) \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.

Proof. The game returns 0 unless the filter passes, so we assume it does and bound the probability that extraction violates the winning condition. The argument runs in four steps.

Step 1: recursion tree and bad events. $\mathcal{E}_{\text{REC}}$ unrolls a tree depth-first. A node v is a pair $(\mathbb{x}_v, \pi_v)$ with $\mathbb{x}_v = (\boldsymbol{p}, \chi, \sigma_v, \mathsf{vk}_{\text{AGG}})$; the root is $\mathcal{A}$'s output. If $\sigma_v.\tau \in [M]$, the node is a *leaf* and the extractor records (σ_v, π_v). If $\sigma_v.\tau = M + 1$, the extractor runs $\mathcal{E}_{\Pi_{\text{AGG}}}(\mathbb{x}_v, \pi_v)$ to obtain a candidate witness $((\sigma_X, \pi_X))_{X \in [A]}$ and recurses, in slot order, into every non-null child $((\boldsymbol{p}, \chi, \sigma_X, \mathsf{vk}_{\text{AGG}}), \pi_X)$; null slots are skipped. If the candidate is not a valid $\mathcal{R}_{\text{AGG}}$-witness, the extractor aborts. For $t \geq 1$ let

$$B_t : \quad \text{the first } \mathcal{E}_{\Pi_{\text{AGG}}} \text{ invocation in unroll order that does not return a valid } \mathcal{R}_{\text{AGG}}\text{-witness for its node's statement is invocation } t\ .$$

The events are disjoint, and at every invocation the node's proof verifies under $\mathbf{V}(\mathsf{vk}_{\mathrm{AGG}},\cdot,\cdot)$: the root's by the filter (since $\mathsf{Typed}(\tau,\kappa)$ with $\tau = M+1$ and $\mathbf{V}_{\mathrm{REC}}^{(M+1)}[\mathsf{vk}_{\mathrm{AGG}}] = \mathbf{V}(\mathsf{vk}_{\mathrm{AGG}},\cdot,\cdot)$), a child's by its parent's valid witness. Now count. Before the first failure, the $t-1$ valid aggregation nodes form a tree in which every node has at least two non-null children, so each expansion replaces one frontier node by at least two and the frontier, the nodes not yet expanded, has at least t nodes. By the counter invariant every frontier node carries an integer counter at least 1, and the counters of the frontier sum to κ. Hence $t \le \kappa$. Moreover, a frontier node of type $M+1$ has counter at least 2 by Typed, so if $\kappa - 1$ valid nodes exist the frontier has κ nodes with counter exactly 1, all of type in $[M]$, and no further invocation exists. Only $B_1, ..., B_{\kappa-1}$ are possible, and off all of them the tree has at most $\kappa - 1$ aggregation nodes and exactly κ leaves; for $\kappa = 1$ the root is a leaf and there is nothing to extract.

Step 2: one reduction per invocation. $\mathcal{D}_{\mathrm{AGG}}^{(t)}$ runs $\mathcal{A}(1^\lambda)$, unrolls the tree up to the t-th invocation (which is deterministic given $\mathcal{A}$'s output and the earlier extractions) and forwards that node's $(\varkappa_v, \pi_v)$ to the knowledge-soundness challenger of Π_{AGG}. It wins whenever B_t occurs, so $\Pr[B_t] \le \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}\left(\mathcal{D}_{\mathrm{AGG}}^{(t)}\right)$, and, running $\mathcal{A}$ and at most $t-1$ extractions, it runs in $t \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.

Step 3: off the bad events, the extractor succeeds. Assume $\neg B_1 \wedge ... \wedge \neg B_{\kappa-1}$; every statement below is then deterministic. We show by strong induction on the counter κ_v that for every node v of the unrolled tree, the leaves recorded below v number exactly κ_v, each has a type in $[M]$ and counter 1, each verifies under $\mathbf{V}_{\mathrm{LEAF}}^{(\tau_i)}$ on the statement $(\boldsymbol{p}, \chi, \sigma_i, \mathsf{vk}_{\mathrm{AGG}})$, and their partial sums and challenge contributions add up to $\sigma_v.s$ and $\sigma_v.\boldsymbol{c}$.

- *Leaf nodes.* If $\sigma_v.\tau \in [M]$, then Typed (asserted by the parent's valid witness, or by the filter for the root) gives $\kappa_v = 1$, and the same source gives $\mathbf{V}_{\mathrm{REC}}^{(\tau_v)}[\mathsf{vk}_{\mathrm{AGG}}](\varkappa_v, \pi_v) = \mathbf{V}_{\mathrm{LEAF}}^{(\tau_v)}(\varkappa_v, \pi_v) = 1$. The sums are trivial.
- *Aggregation nodes.* If $\sigma_v.\tau = M+1$, the valid witness returned for v provides at least two typed non-null children whose counters sum to κ_v over the integers, with $\kappa_X \in [1, \kappa_v - 1]$, each verifying under $\mathbf{V}_{\mathrm{REC}}^{(\tau_X)}[\mathsf{vk}_{\mathrm{AGG}}]$ on a statement that, by the propagation invariant, carries the same $(\boldsymbol{p}, \chi, \mathsf{vk}_{\mathrm{AGG}})$; a child of type $M+1$ is therefore verified by $\mathbf{V}(\mathsf{vk}_{\mathrm{AGG}}, \cdot, \cdot) = \mathbf{V}_{\mathrm{AGG}}$ and is itself an aggregation node of the tree. The induction hypothesis applies to every non-null child; concatenating their leaf lists gives $\sum_{X \in N} \kappa_X = \kappa_v$ leaves, and $s_v = \sum_{X \in N} s_X$, $\boldsymbol{c}_v = \sum_{X \in N} \boldsymbol{c}_X$ make the sums telescope.

Applied to the root, this is exactly the winning condition of Definition 8.3.

Step 4: conclusion. Off all bad events the extractor meets the winning condition, so the game returns 1 only if some B_t occurs, and only $B_1, ..., B_{\kappa-1}$ are possible. By a union bound

$$\mathsf{Adv}^{\mathsf{agg}}_{(\kappa,\mathcal{E}_{\mathrm{REC}})}(\mathcal{A}) \le \sum_{t=1}^{\kappa-1} \Pr[B_t] \le (\kappa-1) \cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}}),$$

with $\mathcal{D}_{\mathrm{AGG}}$ the maximum-advantage reduction among the $\mathcal{D}_{\mathrm{AGG}}^{(t)}$. The extractor invokes $\mathcal{E}_{\Pi_{\mathrm{AGG}}}$ once per aggregation node, at most $\kappa - 1$ times, each call costing at most $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. □

The proof relies on the typing predicate at three places where a proof could *verify* for a statement outside the relation's support, which knowledge soundness excludes only up to its error probability: a child of type $M+1$ with counter 1 (whose statement has no witness), a child of type in $[M]$ with a counter other than 1 (outside the support of the leaf relation), and a child type outside $[M+1]$ (for which no verifier is selected). In each case the parent's valid witness excludes the child, so the offending proof is charged to the parent's invocation. Null children need no such argument: they are never verified and enter none of the sums, so they carry nothing to extract. What the counting does need is that every aggregation node has at least two non-null children; without it a

chain of aggregation nodes with a single non-null child each would keep the counter constant, and the number of invocations would no longer be bounded by κ.

Leaf-verifier layer. Every leaf-verifier proof yields the proof of its segment.

Definition 8.5 (Leaf-Verifier Extraction Game). For $\tau \in [M]$, an extractor $\mathcal{E}^{(\tau)}_{\mathrm{LEAF}}$ and a PPT adversary $\mathcal{A}$, the experiment $\mathsf{Exp}^{\mathsf{leaf}}_{\left(\tau,\mathcal{E}^{(\tau)}_{\mathrm{LEAF}},\mathcal{A}\right)}(\lambda)$ is:

1. **Adversary.** $\mathcal{A}(1^\lambda)$ outputs $(\boldsymbol{p}, \chi, \sigma, \mathsf{vk})$ with $\sigma = (\tau', \kappa, \boldsymbol{c}, s)$, and a proof π_{LEAF}.
2. **Filter.** If $\mathbf{V}^{(\tau)}_{\mathrm{LEAF}}((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}), \pi_{\mathrm{LEAF}}) \neq 1$, return 0.
3. **Extraction.** Run $(\mathrm{com}, \pi) \leftarrow \mathcal{E}^{(\tau)}_{\mathrm{LEAF}}((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}), \pi_{\mathrm{LEAF}})$.
4. **Winning condition.** Return 1 if and only if $((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}); (\mathrm{com}, \pi)) \notin \mathcal{R}^{(\tau)}_{\mathrm{LEAF}}$, i.e. unless $\tau' = \tau$, $\kappa = 1$, $\mathbf{V}^{(\tau)}_{\mathrm{SEG}}((\boldsymbol{p}, \chi, s, \mathrm{com}), \pi) = 1$ and $\boldsymbol{c} = h(\tau, \mathrm{com})$.

We write $\mathsf{Adv}^{\mathsf{leaf}}_{\left(\tau,\mathcal{E}^{(\tau)}_{\mathrm{LEAF}}\right)}(\mathcal{A})$ for the probability that the experiment returns 1.

Lemma 8.6 (Leaf-Verifier Extraction). *Assume $\Pi^{(\tau)}_{\mathrm{LEAF}}$ is knowledge-sound and let $\mathcal{E}^{(\tau)}_{\mathrm{LEAF}} := \mathcal{E}_{\Pi^{(\tau)}_{\mathrm{LEAF}}}$. For every PPT adversary $\mathcal{A}$,*

$$\mathsf{Adv}^{\mathsf{leaf}}_{\left(\tau,\mathcal{E}^{(\tau)}_{\mathrm{LEAF}}\right)}(\mathcal{A}) = \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{LEAF}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{LEAF}}\right), \quad \mathcal{D}^{(\tau)}_{\mathrm{LEAF}} := \mathcal{A}.$$

Proof. The experiment is the knowledge-soundness game of $\Pi^{(\tau)}_{\mathrm{LEAF}}$: same filter, same extractor, and the winning condition is non-membership of the extracted pair in $\mathcal{R}^{(\tau)}_{\mathrm{LEAF}}$. A statement with $\tau' \neq \tau$ or $\kappa \neq 1$ lies outside the support of the relation; a verifying proof for it is a knowledge-soundness break, and the experiment returns 1 for it as well, so the two advantages coincide. The key vk is not constrained at this layer and is only forwarded. □

The type under which a leaf is extracted is the type $\sigma_i.\tau$ read from its statement, which is also the type that selected the verifier in the aggregation layer. The extracted proof is therefore accepted by the segment verifier of type $\sigma_i.\tau$, and this is what makes the output of the composed extractor a *typed* collection.

Compressor layer. For a leaf of a compressed type, the segment verifier is the compressor's, and the proof extracted from the leaf verifier is a compressor proof. One more extraction recovers the AIR proof it verifies.

Definition 8.7 (Compressor Extraction Game). For $\tau \in T_{\mathrm{COMP}}$, an extractor $\mathcal{E}^{(\tau)}_{\mathrm{COMP}}$ and a PPT adversary $\mathcal{A}$, the experiment $\mathsf{Exp}^{\mathsf{comp}}_{\left(\tau,\mathcal{E}^{(\tau)}_{\mathrm{COMP}},\mathcal{A}\right)}(\lambda)$ is:

1. **Adversary.** $\mathcal{A}(1^\lambda)$ outputs a statement $\mathbb{x} = (\boldsymbol{p}, \chi, s, \mathrm{com})$ and a proof π_{COMP}.
2. **Filter.** If $\mathbf{V}^{(\tau)}_{\mathrm{COMP}}(\mathbb{x}, \pi_{\mathrm{COMP}}) \neq 1$, return 0.
3. **Extraction.** Run $\pi_{\mathrm{AIR}} \leftarrow \mathcal{E}^{(\tau)}_{\mathrm{COMP}}(\mathbb{x}, \pi_{\mathrm{COMP}})$.
4. **Winning condition.** Return 1 if and only if $\mathbf{V}^{(\tau)}_{\mathrm{AIR}}(\mathbb{x}, \pi_{\mathrm{AIR}}) \neq 1$, i.e. unless $(\mathbb{x}; \pi_{\mathrm{AIR}})$ is in the compressor relation of $\Pi^{(\tau)}_{\mathrm{AIR}}$ (Definition 3.4).

We write $\mathsf{Adv}^{\mathsf{comp}}_{\left(\tau,\mathcal{E}^{(\tau)}_{\mathrm{COMP}}\right)}(\mathcal{A})$ for the probability that the experiment returns 1.

Lemma 8.8 (Compressor Extraction). *Assume $\Pi^{(\tau)}_{\mathrm{COMP}}$ is knowledge-sound and let $\mathcal{E}^{(\tau)}_{\mathrm{COMP}} := \mathcal{E}_{\Pi^{(\tau)}_{\mathrm{COMP}}}$. For every PPT adversary $\mathcal{A}$,*

$$\mathsf{Adv}^{\mathsf{comp}}_{\left(\tau,\mathcal{E}^{(\tau)}_{\mathrm{COMP}}\right)}(\mathcal{A}) = \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{COMP}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{COMP}}\right), \quad \mathcal{D}^{(\tau)}_{\mathrm{COMP}} := \mathcal{A}.$$

Proof. The experiment is the knowledge-soundness game of $\Pi^{(\tau)}_{\mathrm{COMP}}$ for the compressor relation of $\Pi^{(\tau)}_{\mathrm{AIR}}$. □

For a leaf of a type $\tau \notin T_{\mathrm{COMP}}$ the segment verifier is $\mathbf{V}^{(\tau)}_{\mathrm{AIR}}$, and there is nothing to extract. After this layer, every leaf carries an AIR proof accepted by the AIR verifier of its type.

AIR layer. The AIR proofs are not extracted one by one, since an AIR proof is not knowledge-sound on its own (Remark 5.1). After the previous layers, the leaves of an accepting tree carry exactly the data of the joint AIR extraction game (Definition 5.7): a public statement $\boldsymbol{p}$, fewer than 2^B typed commitments com_i with bus contributions s_i, the key $\chi = \mathsf{DeriveKey}\big(\boldsymbol{p}, \sum_i h(\tau_i, \mathrm{com}_i)\big)$ certified by the Final layer, and AIR proofs accepted under that key. The AIR layer is therefore a single call to the extractor $\mathcal{E}_{\mathrm{J}}$ of the first statement of Theorem 6.2, which outputs the witnesses of all the leaves at once.

9. Main Theorem of the Recursion

9.1. The Target Relation.

Definition 9.1 (ZisK Proof System). A *ZisK proof system* is a tuple

$$\mathsf{ZisK} = \left(\mathfrak{A}, \mathsf{Com}, \mathsf{Com}_{\mathrm{cach}}, h, \mathcal{H}, \mathsf{DeriveKey}, \mathcal{F}, \left\{\Pi^{(\tau)}_{\mathrm{AIR}}\right\}_{\tau\in[M]}, \left\{\Pi^{(\tau)}_{\mathrm{COMP}}\right\}_{\tau\in T_{\mathrm{COMP}}}, \left\{\Pi^{(\tau)}_{\mathrm{LEAF}}\right\}_{\tau\in[M]}, \Pi_{\mathrm{AGG}}, \Pi_{\mathrm{FINAL}}\right)$$

consisting of the following components:

1. **Chip family.** $\mathfrak{A} = \{(\mathcal{C}_\tau, \mathcal{I}_\tau)\}_{\tau\in[M]}$, the chips of Definition 2.5, each with its trace height N_τ, its column classes, its prescribed data and its cached-commitment binding (Definition 2.2).
2. **Trace commitment.** Com, a deterministic commitment scheme (Definition 2.11) on witnesses of any type, straight-line extractable (Definition 2.13). It is exposed in the AIR statements (Definition 3.3), and its digests are what the leaf verifiers hash into the challenge accumulator.
3. **Cached-column commitment.** $\mathsf{Com}_{\mathrm{cach}}$, a deterministic commitment scheme on cached submatrices with commitment space $\mathbb{F}^m$. Its output travels in the public statement and is what binds a proof to a program (Definition 2.6).
4. **Hash to the accumulator group.** $h : [M] \times \{0,1\}^* \to G_{\mathcal{H}}$, modeled as a random oracle (Definition 2.15), which maps a typed commitment to the leaf contribution of the challenge accumulator.
5. **Multiset hash.** $\mathcal{H}$, the additively homomorphic extension of h to finite multisets of typed commitments, preimage aware on multisets of cardinality below 2^B (Definition 2.14).
6. **Key derivation.** $\mathsf{DeriveKey} : \mathcal{P} \times G_{\mathcal{H}} \to \mathcal{X}$, modeled as a random oracle (Definition 2.15), which turns the public statement and the accumulated value into the fingerprint key that the Final Verifier checks.
7. **Bus fingerprint.** $\mathcal{F}$ (Definition 2.8), with key space $\mathcal{X}$ and values in $G_{\mathcal{F}}$, statistically sound with error $\varepsilon_{\mathcal{F}}$ and admitting every nonzero multiset.
8. **Argument systems.** Each is a non-interactive argument system in the sense of Section 2.2, for the relation named.

- $\left\{\Pi^{(\tau)}_{\mathrm{AIR}}\right\}_{\tau\in[M]}$: one per type, for $\mathcal{R}^{(\tau)}_{\mathrm{AIR}}$ (Definition 3.3), proving a single AIR instance.
- $\left\{\Pi^{(\tau)}_{\mathrm{COMP}}\right\}_{\tau\in T_{\mathrm{COMP}}}$: one per compressed type, for the compressor relation of $\Pi^{(\tau)}_{\mathrm{AIR}}$ (Definition 3.4), which hard-codes $\mathsf{vk}^{(\tau)}_{\mathrm{AIR}}$.
- $\left\{\Pi^{(\tau)}_{\mathrm{LEAF}}\right\}_{\tau\in[M]}$: one per type, for $\mathcal{R}^{(\tau)}_{\mathrm{LEAF}}$ (Definition 3.6), the leaf verifier, which hard-codes the key $\mathsf{vk}^{(\tau)}_{\mathrm{SEG}}$ of its segment verifier (Definition 3.5).
- Π_{AGG}: for $\mathcal{R}_{\mathrm{AGG}}$ (Definition 3.8), the aggregation node, which hard-codes the M leaf-verifier keys and reads the aggregation key from its own statement.
- Π_{FINAL}: for $\mathcal{R}_{\mathrm{FINAL}}$ (Definition 3.9), the Final Verifier, which hard-codes $\mathsf{vk}_{\mathrm{AGG}}$ and whose verifier the external party runs.

The carriers are those of the ZisK instantiation (Section 3.4), the key hard-coded by Π_{FINAL} is the key of Π_{AGG}, and the parameters satisfy the standing assumptions of Section 2.

Definition 9.2 (Bounded Global Satisfiability). The *target relation* $\mathcal{R}_{\mathsf{ZisK}}$ has external statements $\boldsymbol{p}_{\mathrm{ext}} \in \mathcal{P}_{\mathrm{ext}}$ as statements, and pairs of proof values $\boldsymbol{p}_{\mathrm{pv}} \in \mathcal{P}_{\mathrm{pv}}$ and typed witness collections L as witnesses: with $\boldsymbol{p} \coloneqq (\boldsymbol{p}_{\mathrm{ext}}, \boldsymbol{p}_{\mathrm{pv}})$,

$$\big(\boldsymbol{p}_{\mathrm{ext}}; (\boldsymbol{p}_{\mathrm{pv}}, L)\big) \in \mathcal{R}_{\mathsf{ZisK}} \Leftrightarrow \begin{cases} 1 \le |L| < 2^B, \\ \mathrm{Local}_{\tau_i}(\boldsymbol{p}, w_i) \text{ for every } i \in [|L|], \\ \mathrm{Balanced}_{\boldsymbol{p}}(L). \end{cases}$$

The family $\mathfrak{A}$, the local predicates and B are implicit in the notation.

The relation is invariant under permutations of L; it requires neither that every type occur nor that witnesses be distinct. Its three conjuncts are exactly what the recursion adds to the individual AIR proofs: a bound on the number of instances, one common public statement, and global bus balancing, including the global messages determined by the statement. Everything else that a ZisK proof establishes about the guest execution is a consequence of Local and Balanced for the specific AIRs of ZisK and is the subject of Section 10.

Definition 9.3 (Final Argument System). The *final argument system* of ZisK is the non-interactive argument system Π^* for $\mathcal{R}_{\mathsf{ZisK}}$ whose proofs are final proofs and whose verifier is

$$\mathbf{V}^*(\boldsymbol{p}_{\mathrm{ext}}, \pi_{\mathrm{FIN}}) \coloneqq \mathbf{V}_{\mathrm{FINAL}}(\boldsymbol{p}_{\mathrm{ext}}, \pi_{\mathrm{FIN}}).$$

Its prover is the honest proving pipeline (execute, split, prove every instance, aggregate) and plays no role below.[2]

In the reference architecture [Kad+26] the two systems differ as algorithms: its target relation has boundary machine states as statements, with memory given in full, while its last recursive proof speaks about states whose memory has been replaced by a commitment, so its external verifier must commit to the two boundary memories before it can run that proof's verifier. ZisK needs no such step: the Final relation and $\mathcal{R}_{\mathsf{ZisK}}$ have the same statement, the external statement $\boldsymbol{p}_{\mathrm{ext}}$, so Π^* and Π_{FINAL} are the same algorithm and differ only in the relation against which each is judged.

Definition 9.4 (Correct-Witness Extractability). ZisK is *correct-witness extractable* (CWE) if Π^* is knowledge-sound for $\mathcal{R}_{\mathsf{ZisK}}$ in the sense of Definition 2.12. We write

$$\mathsf{Adv}^{\mathsf{cwe}}_{\mathsf{ZisK}}(\mathcal{A}) \coloneqq \mathsf{Adv}^{\mathsf{ks}}_{\Pi^*}(\mathcal{A})$$

[2]Knowledge soundness depends only on the verification algorithm.

for the *CWE advantage* of a PPT adversary $\mathcal{A}$ and $\mathcal{E}_{\mathsf{ZisK}}$ for the extractor whose existence is asserted, which may run in time $O(\kappa_{\max}) \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ rather than within the bound of Definition 2.12.

9.2. **Correct-Witness Extractability.**

Theorem 9.5 (Correct-Witness Extractability). *Let* ZisK *be a ZisK proof system (Definition 9.1), let* $\kappa_{\max} := 2^B - 1$*, and let* ε^* *and* Λ_{pole} *be as in Lemma 5.8. Under the idealizations of Remark 2.1, for every PPT adversary* $\mathcal{A}$ *that, together with the extractors composed with it below, makes at most* Q *random-oracle queries, with CWE advantage* $\varepsilon := \mathsf{Adv}^{\mathsf{cwe}}_{\mathsf{ZisK}}(\mathcal{A})$*, there exist PPT reduction adversaries* $\mathcal{D}_{\mathrm{FIN}}$*,* $\mathcal{D}_{\mathrm{AGG}}$*,* $\left\{\mathcal{D}^{(\tau)}_{\mathrm{LEAF}}\right\}_{\tau\in[M]}$*,* $\left\{\mathcal{D}^{(\tau)}_{\mathrm{COMP}}\right\}_{\tau\in T_{\mathrm{COMP}}}$ *and* $\mathcal{D}^{\mathrm{pa}}$ *such that*

$$\begin{aligned}\varepsilon \le\ & \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}(\mathcal{D}_{\mathrm{FIN}}) + (\kappa_{\max}-1)\cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}}) \\ & + \kappa_{\max}\cdot \max_{\tau\in[M]} \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{LEAF}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{LEAF}}\right) + \kappa_{\max}\cdot \max_{\tau\in T_{\mathrm{COMP}}} \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{COMP}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{COMP}}\right) \\ & + Q\cdot\varepsilon^* + \mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{pa}}) + \varepsilon_{\mathrm{MT}}(Q) + \frac{Q\cdot\Lambda_{\mathrm{pole}}}{|\mathbb{K}|} + (Q+1)\cdot\varepsilon_{\mathcal{F}}(\Lambda_{\max}).\end{aligned}$$

All reduction adversaries invoke $\mathcal{A}$ *exactly once and run in time at most* $(3\kappa_{\max}+2)\cdot\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$*. In particular, if every term on the right-hand side is negligible in* λ*, then* ZisK *is correct-witness extractable (Definition 9.4).*

Each term of the bound accounts for one way in which an accepting final proof may fail to yield a witness of $\mathcal{R}_{\mathsf{ZisK}}$, and the proof charges each to one step of the descent:

- $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}$: the final proof is accepted, but no witness of $\mathcal{R}_{\mathrm{FINAL}}$ can be extracted from it.
- $(\kappa_{\max}-1)\cdot\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}$: the same for one of the aggregation nodes, of which a tree with κ leaves has at most $\kappa-1$, since each has at least two non-null children.
- $\kappa_{\max}\cdot\max_\tau \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{LEAF}}}$ and $\kappa_{\max}\cdot\max_{\tau\in T_{\mathrm{COMP}}} \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{COMP}}}$: the same for one of the at most κ leaf verifiers and compressors.
- $Q\cdot\varepsilon^*$: some query of the adversary to the transcript of an AIR proof makes a round of the IOP succeed, which each of the Q queries does with probability at most the round-by-round error ε^* (Theorem 4.17).
- $\mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}$: when the key is derived, the accumulator $\boldsymbol{c}$ hides a multiset other than the typed commitments of the leaves.
- $\varepsilon_{\mathrm{MT}}(Q)$: the oracle queries contain a collision or an inversion of the hash, so that some Merkle root has no unique preimage.
- $Q\cdot\Lambda_{\mathrm{pole}}/|\mathbb{K}|$: the derived key is a pole of one of the extracted witnesses, where the bus constraints do not determine the bus contribution.
- $(Q+1)\cdot\varepsilon_{\mathcal{F}}(\Lambda_{\max})$: the fingerprint of an unbalanced collection vanishes at one of the at most $Q+1$ derived keys, so the root check passes although some bus is unbalanced.

The first four terms are charged by the layers of Section 8; the next four by the joint extraction of the AIR witnesses (Lemma 5.8), which also supplies the preimage awareness and the Merkle extraction that the bus argument needs; and the last by Proposition 7.4.

Proof. Let $\mathcal{A}$ be a PPT adversary with CWE advantage ε; we bound ε. We construct $\mathcal{E}_{\mathsf{ZisK}}$ by composing the sub-extractors of Lemma 8.2, Lemma 8.4, Lemma 8.6 and Lemma 8.8 with the joint extractor of the AIR layer of Section 8, each straight-line and each applied to the values extracted by the preceding layer. Let $(\boldsymbol{p}_{\mathrm{ext}}, \pi_{\mathrm{FIN}})$ be $\mathcal{A}$'s output and assume $\mathbf{V}_{\mathrm{FINAL}}(\boldsymbol{p}_{\mathrm{ext}}, \pi_{\mathrm{FIN}}) = 1$, since otherwise $\mathcal{A}$ does not win.

Step 1 (Final layer). Apply $\mathcal{E}_{\mathrm{FIN}}$ to $(\boldsymbol{p}_{\mathrm{ext}}, \pi_{\mathrm{FIN}})$ to obtain $\big(\boldsymbol{p}_{\mathrm{pv}}, \chi, \sigma, \pi\big)$ with $\sigma = (\tau, \kappa, \boldsymbol{c}, s)$ such that, for $\boldsymbol{p} := \big(\boldsymbol{p}_{\mathrm{ext}}, \boldsymbol{p}_{\mathrm{pv}}\big)$, $\mathsf{Typed}(\tau, \kappa)$, $\mathbf{V}^{(\tau)}_{\mathrm{REC}}[\mathsf{vk}_{\mathrm{AGG}}]((\boldsymbol{p}, \chi, \sigma, \mathsf{vk}_{\mathrm{AGG}}), \pi) = 1$, $s + \mathcal{F}_\chi(\mathcal{G}(\boldsymbol{p})) = 0$ and $\chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c})$. This fails with probability at most $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}(\mathcal{D}_{\mathrm{FIN}})$ by Lemma 8.2. In particular $\kappa \in [1, 2^B)$, so $\kappa \le \kappa_{\max}$.[3]

Step 2 (Aggregation layer). Apply $\mathcal{E}_{\mathrm{REC}}$ to $(\boldsymbol{p}, \chi, \sigma, \pi)$; its filter is satisfied by Step 1. Obtain $((\sigma_1, \pi_1), ..., (\sigma_\kappa, \pi_\kappa))$ with $\sigma_i = (\tau_i, 1, \boldsymbol{c}_i, s_i)$, $\tau_i \in [M]$, $\mathbf{V}^{(\tau_i)}_{\mathrm{LEAF}}((\boldsymbol{p}, \chi, \sigma_i, \mathsf{vk}_{\mathrm{AGG}}), \pi_i) = 1$, $\sum_i s_i = s$ and $\sum_i \boldsymbol{c}_i = \boldsymbol{c}$. This fails with probability at most $(\kappa - 1) \cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}})$ by Lemma 8.4, and the extractor runs in $(\kappa - 1) \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.

Step 3 (Leaf-verifier layer, applied κ times). For each $i \in [\kappa]$ apply $\mathcal{E}^{(\tau_i)}_{\mathrm{LEAF}}$ to $((\boldsymbol{p}, \chi, \sigma_i, \mathsf{vk}_{\mathrm{AGG}}), \pi_i)$ to obtain $\big(\mathrm{com}_i, \pi^{\mathrm{SEG}}_i\big)$ with $\mathbf{V}^{(\tau_i)}_{\mathrm{SEG}}\big((\boldsymbol{p}, \chi, s_i, \mathrm{com}_i), \pi^{\mathrm{SEG}}_i\big) = 1$ and $\boldsymbol{c}_i = h(\tau_i, \mathrm{com}_i)$. Each invocation fails with probability at most $\mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau_i)}_{\mathrm{LEAF}}}\Big(\mathcal{D}^{(\tau_i)}_{\mathrm{LEAF}}\Big)$ by Lemma 8.6; the total over κ calls is at most $\kappa \cdot \max_\tau \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{LEAF}}}\Big(\mathcal{D}^{(\tau)}_{\mathrm{LEAF}}\Big)$.

Step 3′ (Compressor layer, applied at most κ times). For each $i \in [\kappa]$ with $\tau_i \in T_{\mathrm{COMP}}$, apply $\mathcal{E}^{(\tau_i)}_{\mathrm{COMP}}$ to $\big((\boldsymbol{p}, \chi, s_i, \mathrm{com}_i), \pi^{\mathrm{SEG}}_i\big)$ to obtain π^{AIR}_i with $\mathbf{V}^{(\tau_i)}_{\mathrm{AIR}}\big((\boldsymbol{p}, \chi, s_i, \mathrm{com}_i), \pi^{\mathrm{AIR}}_i\big) = 1$. Each invocation fails with probability at most $\mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau_i)}_{\mathrm{COMP}}}\Big(\mathcal{D}^{(\tau_i)}_{\mathrm{COMP}}\Big)$ by Lemma 8.8, and the total is at most $\kappa \cdot \max_{\tau \in T_{\mathrm{COMP}}} \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{COMP}}}\Big(\mathcal{D}^{(\tau)}_{\mathrm{COMP}}\Big)$. For $\tau_i \notin T_{\mathrm{COMP}}$ the segment verifier is $\mathbf{V}^{(\tau_i)}_{\mathrm{AIR}}$, and we set $\pi^{\mathrm{AIR}}_i := \pi^{\mathrm{SEG}}_i$.

Step 4 (AIR layer, one joint call). Let $\mathcal{A}_{\mathrm{J}}$ be the algorithm that runs $\mathcal{A}$ and Steps 1 to 3′ and outputs $\boldsymbol{p}$ and $\big(\tau_i, \mathrm{com}_i, s_i, \pi^{\mathrm{AIR}}_i\big)_{i \in [\kappa]}$. When Steps 1 to 3′ succeed, $1 \le \kappa < 2^B$, $\sum_i h(\tau_i, \mathrm{com}_i) = \sum_i \boldsymbol{c}_i = \boldsymbol{c}$, $\chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c})$, and every π^{AIR}_i is accepted under χ, so $\mathcal{A}_{\mathrm{J}}$ passes the filter of the joint game (Definition 5.7). Apply $\mathcal{E}_{\mathrm{J}}$ to obtain $w_1, ..., w_\kappa$ with $((\boldsymbol{p}, \chi, s_i, \mathrm{com}_i); w_i) \in \mathcal{R}^{(\tau_i)}_{\mathrm{AIR}}$, that is, $\mathrm{Local}_{\tau_i}(\boldsymbol{p}, w_i)$, $s_i = \mathrm{BusCont}_\chi\big(\mathcal{I}_{\tau_i}, w_i\big)$ and $\mathsf{Commit}(w_i) = \mathrm{com}_i$, for every i. Let B_{J} be the union of the four events of the proof of Lemma 5.8 for $\mathcal{A}_{\mathrm{J}}$; outside B_{J} the extraction succeeds, and

$$\Pr[B_{\mathrm{J}}] \le Q \cdot \varepsilon^* + \mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{pa}}) + \varepsilon_{\mathrm{MT}}(Q) + \frac{Q \cdot \Lambda_{\mathrm{pole}}}{|\mathbb{K}|}.$$

Step 5 (Assembly). Output $\boldsymbol{p}_{\mathrm{pv}}$ and $L := ((\tau_1, w_1), ..., (\tau_\kappa, w_\kappa))$. Deterministically, given that Steps 1 to 4 succeeded: $|L| = \kappa \in [1, 2^B)$; $\mathrm{Local}_{\tau_i}(\boldsymbol{p}, w_i)$ for every i; $\boldsymbol{c} = \sum_i h(\tau_i, \mathrm{com}_i) = \sum_i h(\tau_i, \mathsf{Commit}(w_i))$; $\chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c})$; and $\sum_i \mathrm{BusCont}_\chi\big(\mathcal{I}_{\tau_i}, w_i\big) + \mathcal{F}_\chi(\mathcal{G}(\boldsymbol{p})) = s + \mathcal{F}_\chi(\mathcal{G}(\boldsymbol{p})) = 0$.

Step 6 (Bus soundness). Let $\mathcal{A}_{\mathrm{bus}}$ be the algorithm that runs $\mathcal{A}$ and Steps 1 to 5 and outputs $\big(\boldsymbol{p}, L, (\mathrm{com}_i)_i\big)$. When Steps 1 to 4 succeed, Step 5 shows that the output satisfies every conjunct of the winning condition of Definition 7.3 except possibly $\neg\mathrm{Balanced}_{\boldsymbol{p}}(L)$, and the game's derivation reproduces the extracted $\boldsymbol{c}$ and χ. Outside B_{J}, moreover, the extractor of $\mathcal{H}$ returns the multiset $\{(\tau_i, \mathrm{com}_i)\}_i$ at the first query to $\mathsf{DeriveKey}$ at $(\boldsymbol{p}, \boldsymbol{c})$, which excludes the accumulator event of Lemma 5.8, and the extractor of Com returns w_i there, since the w_i are the preimages of the roots that the Merkle event of Lemma 5.8 fixes in the query list before that query. Hence, by the second statement of Proposition 7.4,

$$\Pr\big[\text{Steps 1 to 3′ succeed} \wedge \neg B_{\mathrm{J}} \wedge \neg\mathrm{Balanced}_{\boldsymbol{p}}(L)\big] \le (Q + 1) \cdot \varepsilon_{\mathcal{F}}(\Lambda_{\max}).$$

Conclusion. If Steps 1 to 3′ succeed, B_{J} does not occur and L is balanced for $\boldsymbol{p}$, then $\big(\boldsymbol{p}_{\mathrm{ext}}; (\boldsymbol{p}_{\mathrm{pv}}, L)\big) \in \mathcal{R}_{\mathsf{ZisK}}$ by Step 5 and Definition 9.2, so $\mathcal{A}$ does not win. Summing the failure probabilities,

[3] This composition is where the first idealization of Remark 2.1 is invoked: a straight-line extractor for a succinct Π_{FINAL} does not exist in the plain model, and composing it with the extractors below requires relativized arguments.

$$
\begin{aligned}
\varepsilon \le\ & \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{FINAL}}}(\mathcal{D}_{\mathrm{FIN}}) && \text{(Step 1)}\\
& +(\kappa-1)\cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{\mathrm{AGG}}}(\mathcal{D}_{\mathrm{AGG}}) && \text{(Step 2)}\\
& +\kappa\cdot \max_{\tau} \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{LEAF}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{LEAF}}\right) && \text{(Step 3)}\\
& +\kappa\cdot \max_{\tau\in T_{\mathrm{COMP}}} \mathsf{Adv}^{\mathsf{ks}}_{\Pi^{(\tau)}_{\mathrm{COMP}}}\left(\mathcal{D}^{(\tau)}_{\mathrm{COMP}}\right) && \text{(Step 3}')\\
& +\Pr[B_{\mathrm{J}}] && \text{(Step 4)}\\
& +(Q+1)\cdot \varepsilon_{\mathcal{F}}(\Lambda_{\max}) && \text{(Step 6),}
\end{aligned}
$$

and replacing κ by $\kappa_{\max}$, which only increases the right-hand side, gives the stated bound. The terms of preimage awareness and of Merkle extraction are charged once, in Step 4, although both the joint extraction and the bus argument rely on them. All coefficients are polynomially bounded, so the right-hand side is negligible whenever each term is. Unlike the step count of the reference architecture [Kad+26], κ is not a system parameter but part of the extracted witness; the bound with κ is therefore a per-tree refinement and the bound with $\kappa_{\max}$ the unconditional one.

Running time. $\mathcal{E}_{\mathsf{ZisK}}$ never runs $\mathcal{A}$; its cost is the straight-line sub-extractor calls plus deterministic work: one call to $\mathcal{E}_{\Pi_{\mathrm{FINAL}}}$, at most $\kappa-1$ calls to $\mathcal{E}_{\Pi_{\mathrm{AGG}}}$, κ calls to the leaf-verifier extractors, at most κ calls to the compressor extractors and one call to $\mathcal{E}_{\mathrm{J}}$, i.e. at most $3\kappa+1$ calls in total. Under the convention that each call costs at most $\mathsf{Time}(\mathcal{A})+\mathsf{poly}(\lambda)$, $\mathsf{Time}(\mathcal{E}_{\mathsf{ZisK}}) \le (3\kappa_{\max}+1)\cdot \mathsf{Time}(\mathcal{A})+\mathsf{poly}(\lambda)$. Each reduction adversary runs $\mathcal{A}$ once and re-runs the extraction chain above its own layer, hence runs in at most $(3\kappa_{\max}+2)\cdot \mathsf{Time}(\mathcal{A})+\mathsf{poly}(\lambda)$. (Strictly, the convention of Definition 2.12 bounds each extractor by the time of the composed adversary that produced its input, which compounds along the recursion, in the worst case exponentially in the depth of the tree, the usual blow-up of recursive extraction; the linear figure uses the per-call convention and is part of the idealization of Remark 2.1.) In particular $\mathcal{E}_{\mathsf{ZisK}}$ is PPT whenever $\mathcal{A}$ is, since $2^B \le \mathsf{poly}(\lambda)$. □

Remark 9.1 (Query Budget and the Factors $\kappa_{\max}$). *The query bound Q counts the queries of $\mathcal{A}$ together with those of the extractors composed with it, which are polynomially many: Steps 4 and 6 apply to such compositions. Every proof of the tree, moreover, is produced from the same query list, so a single union bound over the Q queries could replace the factors of the aggregation, leaf-verifier and compressor terms, as it already does for the AIR proofs in Step 4. Doing so would merge the layers into one compilation argument; we keep the layered structure of the reference architecture [Kad+26] and its looser bound.*

(WIP) Part III: Semantics

What a ZisK proof claims is the execution relation $\mathcal{R}_{\text{exec}}$ of the whitepaper [Zis26], which formalizes the claim of Section 1.1. Part III states, and partly proves, that a balanced collection of locally satisfying witnesses encodes a witness of this relation; Section 10 recalls what such a witness is and how its words are carried by field elements.

10. Executions and Words

The state of the machine at step t is $S_t = (\text{pc}_t, c_t, \boldsymbol{x}_t, \text{mem}_t)$: the program counter, the output of the previous instruction, the registers $x_1, ..., x_{31}$ and the data memory, a map from the 32-bit byte addresses to bytes. A program P is a finite partial map from instruction addresses to instructions, and one step from S, with the sequence h of hint words that it reads, gives the state $\text{Step}_P(S, h)$, as the whitepaper specifies, operand by operand, for every opcode. A step is *valid* if its program counter is an address of P whose instruction is not `halt`, every memory access it makes lies in one region of the address space, it writes nothing to the input data region, and it reads exactly the words of h from the free-input port.

The relation has the external statement $\boldsymbol{p}_{\text{ext}} = (\text{root}_{\text{ROM}}, \text{out}_0, ..., \text{out}_{31})$ as statement and a whole execution $w_{\text{exec}} = \left(P, \text{in}, (h_t)_{t<T}, (S_t)_{t\leq T}\right)$ as witness, and $(\boldsymbol{p}_{\text{ext}}; w_{\text{exec}}) \in \mathcal{R}_{\text{exec}}$ requires:

- *Program.* P is valid and $\text{RomRoot}(P) = \text{root}_{\text{ROM}}$: its instructions are instructions of the ISA (Section 12.1), and the root of the Rom chip's cached columns for P is the program commitment.
- *Initial state.* $S_0 = (\text{pc}_{\text{BOOT}}, 0, \mathbf{0}, \text{mem}_0)$, where mem_0 holds in from address `0x4000_0008` and is zero outside the input data and ROM data regions.
- *Steps.* For every $t < T$, the step from S_t with hints h_t is valid and $S_{t+1} = \text{Step}_P(S_t, h_t)$.
- *ROM data.* Every write to the ROM data region leaves the memory unchanged, every word of the region is written at most once, and every word that is never written is zero in mem_0.
- *Termination.* $\text{pc}_T = \text{pc}_{\text{END}}$, $c_T = 0$ and $T \leq 2^{36}$.
- *Public outputs.* Every step $t < T$ that executes `pubout` has operands $a_t < 32$ and $b_t = \text{out}_{a_t}$, and for every $i < 32$ exactly one step executes `pubout` with $a_t = i$.

The ROM data clause records what the arithmetization enforces for the read-only region, which groups the accesses to a word but does not order them in time: the contents of the region are fixed from the start by the stores of the boot code, and a read returns the same value whenever it happens. The statement contains neither the input nor the hints; a guest that wants its outputs bound to its input must expose, for instance, a hash of the input among them.

10.1. Transpiler Correctness.

The relation concerns the committed ZisK program. Its connection to the RISC-V binary that the developer compiled is the correctness of the transpiler, an off-chain, deterministic tool that this analysis does not verify.

Definition 10.1 (Transpiler Correctness). The transpiler is *correct* if, for every supported RISC-V binary P_{rv} and its transpilation P, every successful execution of P on an input in and hints h that declares the public outputs $\text{out}_0, ..., \text{out}_{31}$ corresponds to an execution of P_{rv} on in, with the host answering its requests for hints by h, that terminates and produces the same outputs.

We assume that the transpiler is correct; every statement of this paper is about the committed ZisK program, and becomes a statement about the RISC-V binary only through this assumption.

10.2. Words and Limbs.

The ISA computes on 64-bit words, with arithmetic modulo 2^{64}, while the traces and the messages of ZisK are vectors over $\mathbb{F}$, of order $p = 2^{64} - 2^{32} + 1 < 2^{64}$. ZisK writes every word as two 32-bit limbs. The rest of this section states when limbs stand for words, and where the range of every limb is established.

We identify every element of $\mathbb{F}$ with its representative in $[0, p)$.

Definition 10.2 (Words). The *word* of a pair of limbs $(v_0, v_1) \in \mathbb{F}^2$ is $\mathrm{word}(v_0, v_1) := v_0 + 2^{32} v_1 \in \mathbb{N}$, and the *split* of a 64-bit value $v \in [0, 2^{64})$ is $\mathrm{split}(v) := (v \bmod 2^{32}, \lfloor v/2^{32} \rfloor) \in \mathbb{F}^2$. A pair of limbs is *well formed* if $v_0, v_1 < 2^{32}$.

Lemma 10.3 (Words and Limbs). *For every $v \in [0, 2^{64})$, $\mathrm{split}(v)$ is well formed and $\mathrm{word}(\mathrm{split}(v)) = v$; for every well-formed pair, $\mathrm{split}(\mathrm{word}(v_0, v_1)) = (v_0, v_1)$. Hence* word *is a bijection from the well-formed pairs to $[0, 2^{64})$, and two messages whose words are carried in well-formed pairs are equal if and only if the words are.*

Proof. Both limbs of $\mathrm{split}(v)$ are below $2^{32} < p$, so they are their own representatives, and $(v \bmod 2^{32}) + 2^{32} \lfloor v/2^{32} \rfloor = v$. For a well-formed pair, $v_0 + 2^{32} v_1 < 2^{64}$, and division by 2^{32} with remainder recovers v_1 and v_0. The last statement follows, since a message determines its limbs and the limbs, when well formed, determine the word and are determined by it. □

The lemma is what lets the analysis read words off messages: the operands and results of the Operation Bus, the values of the Memory Bus and the outputs in $\boldsymbol{p}_{\mathrm{ext}}$ are words exactly when their limbs are well formed. A pair that is not well formed is not the split of its word, and two pairs may differ in $\mathbb{F}^2$ while having the same word, so every pair that the reconstruction reads must be shown to be well formed.

10.3. Where Limbs Are Well Formed.

ZisK establishes the range of a limb where the value is *produced*, and chips that only move a value do not check it again. In the terms of Section 11, the producers are:

- *The program.* The immediates of an instruction are fields of the committed program, in $[0, 2^{32})$ by the instruction format.
- *The Operation Bus.* The contract of every operation states that the limbs of its result are well formed whenever those of its operands are (Definition 11.7); for a precompile, the words that it writes to memory are well formed by the contract of its memory effect.
- *The input data region.* Its AIR range-checks every value in 16-bit chunks, so every word read from the input, and every hint, is well formed.
- *The global messages.* The public outputs are two 32-bit limbs of each word in $\boldsymbol{p}_{\mathrm{ext}}$ by the instantiation of Section 3.4, and the initial values of the registers and the anchors of the memory chains carry the value zero.

Starting from these producers, the reconstruction of Section 15 shows, step by step, that every operand, result, register and memory word of the execution is well formed: a value is read either from a producer or from a word that an earlier step stored, which the induction has already shown to be well formed.

10.4. Addresses, Steps and Wrap-Around.

Byte addresses (32 bits), program counters (32 bits), memory steps (at most 2^{38}) and segment indices (below $2^{36}/N$) are below p and are carried as single field elements, so an equality between two of them in $\mathbb{F}$ is an equality of integers. The chips bound them by range checks where they are produced, and the soundness of every chip includes that the arithmetic identities it checks over range-checked chunks hold over the integers rather than modulo p: this is how the result of an operation is the one the ISA specifies, modulo 2^{64}.

11. Bus Semantics

Part II ends with a balanced collection: from an accepting final proof it extracts proof values $\boldsymbol{p}_{\text{pv}}$ and a typed witness collection L with $\big(\boldsymbol{p}_{\text{ext}};\big(\boldsymbol{p}_{\text{pv}}, L\big)\big) \in \mathcal{R}_{\mathsf{ZisK}}$, so that every witness is locally satisfying for $\boldsymbol{p} = \big(\boldsymbol{p}_{\text{ext}}, \boldsymbol{p}_{\text{pv}}\big)$ and L is balanced for $\boldsymbol{p}$. Balance states only that the multiplicities of every message add up to zero in $\mathbb{F}$. What a message *means* is a contract between the chips on its bus. This section turns balance and the contracts into guarantees: every message that a chip assumes holds, and on the buses that only count, the messages proved are the messages assumed.

11.1. Pulls and Pushes.

Definition 11.1 (Interactions of a Collection). The *interactions* of a typed witness collection L for a public statement $\boldsymbol{p}$ form the finite list $\mathcal{J}_{\boldsymbol{p}}(L)$ of triples (b, μ, v), of a bus, a multiplicity $\mu \in \mathbb{F}$ and a message v, that contains $(b_\iota, m_\iota(w_i, r), \boldsymbol{e}_\iota(w_i, r))$ for every entry (τ_i, w_i) of L, every interaction $\iota \in \mathcal{I}_{\tau_i}$ and every row $r \in \big[0, N_{\tau_i}\big)$, and $(b, \mathcal{G}(\boldsymbol{p})(b, v), v)$ for every global message (b, v) in the support of $\mathcal{G}(\boldsymbol{p})$. An interaction is a *pull* if $\mu = -1$, and a *push* if $\mu \notin \{0, -1\}$.

The list has at most $\Lambda(L) + g$ entries, and L is balanced for $\boldsymbol{p}$ exactly when, for every bus b and message v, the multiplicities of the entries of $\mathcal{J}_{\boldsymbol{p}}(L)$ that carry (b, v) add up to zero (Definition 2.10). Whether an interaction pulls or pushes depends on the value of its multiplicity, not on how the chip writes it. A chip *assumes* a message with the multiplicity -1 of a selected row, and *proves* it with any other nonzero multiplicity, such as the free multiplicity column of a lookup table.

Lemma 11.2 (Balance with Few Interactions). *Let L be balanced for $\boldsymbol{p}$, with $\Lambda(L) + g < p$.*

1. *For every pull of a message (b, v) in $\mathcal{J}_{\boldsymbol{p}}(L)$, there is a push of (b, v) in $\mathcal{J}_{\boldsymbol{p}}(L)$.*
2. *If every interaction on the bus b has its multiplicity in $\{0, 1, -1\}$, then every message v is pushed on b exactly as many times as it is pulled: the multiset of messages pushed on b is the multiset of messages pulled from it.*

Proof. (1) Suppose that (b, v) is pulled and never pushed. Then every entry that carries (b, v) has multiplicity 0 or -1, and $k \geq 1$ of them have -1, so the multiplicities of (b, v) add up to $-k$. Since $k \leq \big|\mathcal{J}_{\boldsymbol{p}}(L)\big| \leq \Lambda(L) + g < p$, this is not zero in $\mathbb{F}$, which contradicts balance.

(2) The multiplicities of (b, v) add up to $k_+ - k_-$, where k_+ and k_- count the entries with multiplicities 1 and -1. Both are below p, so $k_+ - k_- \equiv 0 \bmod p$ forces $k_+ = k_-$. □

For a collection of fewer than 2^B witnesses, $\Lambda(L) + g \leq \Lambda_{\max}$, so the lemma applies to every collection that Part II extracts as soon as $\Lambda_{\max} < p$. In ZisK, with $B = 20$ and trace heights of at most 2^{22}, this holds whenever no chip has 2^{21} or more interactions per row, far more than any chip has. The second statement resolves Remark 7.2 for the buses whose multiplicities are 0, 1 and -1; the others need only the first.

11.2. Contracts.

A contract is attached to messages rather than to buses, because some chips of ZisK both prove and assume messages on the same bus, of different kinds: Arith proves multiplications and assumes comparisons, both on the Operation Bus. A bus is therefore split into *channels* by a key read from the message, such as the opcode of an operation.

Definition 11.3 (Channels and Contracts). A *channel assignment* gives every bus b a function κ_b on its messages, whose values are the *channels* $(b, \kappa_b(v))$ of the bus, and gives every channel c either a *predicate* $\Phi_c(\boldsymbol{p}, v)$ on the public statement and the messages of c, or none. A channel with a predicate is a *predicate channel*; a channel without one is a *counted channel*. A pull or a push of

a message *meets* its contract if the message satisfies the predicate of its channel, or if its channel is counted.

Definition 11.4 (Sound Chip). Let R_τ be a set of predicate channels and Spec_τ a predicate on public statements and witnesses of type τ. The chip of type τ is *sound*, relying on R_τ and establishing Spec_τ, if for every $\boldsymbol{p}$ and every $w \in \mathcal{W}_\tau$ with $\mathrm{Local}_\tau(\boldsymbol{p}, w)$ whose pulls on the channels of R_τ meet their contracts, every push of w meets its contract and $\mathrm{Spec}_\tau(\boldsymbol{p}, w)$ holds. Its pushes on a channel c are *unconditional* if they meet their contract for every w with $\mathrm{Local}_\tau(\boldsymbol{p}, w)$, whatever its pulls.

Local soundness alone does not make balance useful: a chip that assumes a message and proves the same message on every row balances by itself and is locally sound for any predicate, true or not. What rules this out is an order in which every guarantee rests on requirements established before it.

Definition 11.5 (Ordered Chips). Sound chips are *ordered* by a linear order $<$ on $[M]$ if, for all types σ and τ and every channel $c \in R_\tau$ on which σ pushes, either $\sigma < \tau$ or the pushes of σ on c are unconditional.

In particular no chip relies on a channel on which it pushes conditionally. The global messages come before every chip: they read only the public statement, and a channel assignment is *compatible* with them if every global message that is pushed meets its contract.

Theorem 11.6 (System Soundness). *Let the chips of ZisK be sound and ordered for a channel assignment compatible with the global messages, and let L be balanced for $\boldsymbol{p}$, with $\mathrm{Local}_{\tau_i}(\boldsymbol{p}, w_i)$ for every entry (τ_i, w_i) of L and $\Lambda(L) + g < p$. Then:*

1. *every pull and every push of $\mathcal{J}_{\boldsymbol{p}}(L)$ meets its contract;*
2. *$\mathrm{Spec}_{\tau_i}(\boldsymbol{p}, w_i)$ holds for every entry of L; and*
3. *on every bus whose interactions have multiplicities in $\{0, 1, -1\}$, the multiset of messages pushed is the multiset of messages pulled.*

Proof. We show, by induction along the order, that for every type τ every entry of type τ meets the contracts of its pushes and satisfies Spec_τ. Let the claim hold for every type before τ, and let w be an entry of type τ. Every pull of w on a channel $c \in R_\tau$ carries a message (b, v) that, by Lemma 11.2, is also pushed, on the same channel since the channel is a function of the message. The push comes from a global message, which meets its contract by compatibility; from an entry of a type $\sigma < \tau$, which meets it by induction; or from an entry of a type whose pushes on c are unconditional, which meets it by Local. In every case v satisfies $\Phi_c(\boldsymbol{p}, \cdot)$, so the pull meets its contract too. Since the chip of type τ is sound, every push of w meets its contract and $\mathrm{Spec}_\tau(\boldsymbol{p}, w)$ holds.

Every push of $\mathcal{J}_{\boldsymbol{p}}(L)$ therefore meets its contract, and so does every pull, by the same argument: its message is pushed, and a push and a pull of the same message are on the same channel. This proves (1) and (2); (3) is the second statement of Lemma 11.2. □

The theorem gives each assumed message the meaning that its provers establish, but only up to what the contracts say. Counted channels carry no predicate: their meaning comes from the third statement, which the memory argument (Section 14) and the stitching of segments (Section 13) turn into consequences about the execution.

11.3. The Channels of ZisK.

The channel assignment of ZisK follows the contracts of the whitepaper, in the word encoding of Section 10.2.

Definition 11.7 (Channels of ZisK).

1. *Operation bus.* The key of a message $(\mathrm{op}, a_0, a_1, b_0, b_1, c_0, c_1, \mathrm{flag}, t, x, \mathrm{extra})$ is its opcode op. For every opcode other than `pubout`, the predicate states that, if $a_0, a_1, b_0, b_1 < 2^{32}$, then $c_0, c_1 < 2^{32}$ and $(c, \mathrm{flag}) = \mathrm{op}(a, b)$, for the words a, b and c of the limbs (Definition 10.2); for a precompile, it states moreover that the memory effect of the operation, at step t and with extended argument x, is the one its prover makes on the Memory Bus, with well-formed limbs. For `pubout`, the predicate states that $a < 32$, that $b = \mathrm{out}_a$, the public output of index a in $\boldsymbol{p}_{\mathrm{ext}}$, and that $(c, \mathrm{flag}) = (b, 0)$.
2. *ROM bus.* One channel, whose predicate states that the instruction of the committed program at the address of the message has the fields of the message.
3. *Table buses.* One channel per bus, whose predicate states that the message is a row of the table, or, on a range bus, that its value lies in the range.
4. *Memory bus and continuation buses.* Counted channels.

The global messages of ZisK are compatible with this assignment: the only global messages on a predicate channel are those of the public outputs, which state $b = \mathrm{out}_a$ by construction. Section 12 proves the chips of ZisK sound for this assignment, or states their soundness as an obligation, and gives the order.

12. Chips

This section states what each chip of ZisK establishes about the messages it proves, in the terms of Section 11, from the constraints of its AIR. The chips on which the reconstruction of the execution rests —the Rom, the fixed tables, BinaryAdd, the Main chip and the Memory AIRs— are proved sound below; the soundness of the others is stated, with the same contract, as an obligation. The last section orders the chips and applies Theorem 11.6.

12.1. Valid Programs.

The Main chip executes whatever instructions the program table holds. Its rows follow the step semantics of the ISA only if the table is the ROM table of a *valid* program, as $\mathcal{R}_{\mathrm{exec}}$ requires: one whose instructions have their fields in their domains, take m32, is_external_op and is_precompiled from their opcodes, use register indices in $[1, 31]$, and set set_pc only for opcodes whose flag is always zero. In the ROM table of such a program the addresses are distinct, and the packed flags of every message select exactly one source for each operand and at most one store. No AIR checks validity: it is a property of the committed program, which the external verifier fixes by comparing $\mathrm{root}_{\mathrm{ROM}}$ with the commitment of the program it expects. Every program produced by the transpiler is valid, and the theorem of Section 15 assumes that the program committed in $\mathrm{root}_{\mathrm{ROM}}$ is.

12.2. The Rom and the Fixed Tables.

The Rom chip has a single interaction: it proves, on the ROM Bus, every row of its table columns with a free multiplicity. Its table columns are cached (Definition 2.2), so by Local they open the commitment $\mathrm{root}_{\mathrm{ROM}}$ carried in the public statement.

Lemma 12.1 (Program Table). *Let the Rom witnesses of L have the same cached columns, and let P_L be the program whose messages are their rows other than the all-zero padding rows. Then* $\mathrm{RomRoot}(P_L) = \mathrm{root}_{\mathrm{ROM}}$*, and every push on the ROM Bus is a message of P_L. These pushes are unconditional.*

Proof. The Rom chip has no constraint other than its interaction, which proves a row of its cached columns; Local requires these columns to open the coordinates of $\boldsymbol{p}_{\text{ext}}$ that hold root_{ROM}, which is the statement $\text{RomRoot}(P_L) = \text{root}_{\text{ROM}}$. Every push is a row of the cached columns, hence a message of P_L, whatever the witness; the chip makes no pull, so its pushes are unconditional. □

The hypothesis that all the Rom witnesses have the same cached columns follows from the binding of $\mathsf{Commit}_{\text{cach}}$, since they all open the same root_{ROM}; its failure is a term of the composition in Part IV. With the program P_L, the predicate of the ROM channel states that the message is a message of P_L.

The other chips that prove from fixed columns have the same shape: the range tables, the tables of the binary, binary extension, arithmetic, MemAlign and DMA chips, the Keccak-f table, and the frequent-operation caches, which prove on the Operation Bus. Each proves, with a free multiplicity, a row of its fixed columns.

Lemma 12.2 (Fixed Tables). *Every push of a chip that proves from fixed columns is a row of those columns, unconditionally. If the fixed columns are the table that the chip's bus specifies, the pushes meet their contracts.*

Proof. The fixed columns are part of the verification key of the chip, so every locally satisfying witness has them, and every interaction of the chip proves a row of (some of) them. If they are the specified table, a row of them satisfies the predicate of its channel. □

That the fixed columns of every table are the specified table, and that every row of a frequent-operation cache is a correct operation, are properties of the setup, which this analysis takes as obligations (Table 5).

12.3. **BinaryAdd.**

The BinaryAdd chip proves the 64-bit addition `add` on the Operation Bus, one operation per row. A row holds the limbs a_0, a_1, b_0, b_1 of the operands, four 16-bit chunks $\gamma_0, ..., \gamma_3$ of the result and two carries κ_0, κ_1, with the constraints

$$\kappa_i(1-\kappa_i) = 0, \quad a_0 + b_0 = 2^{32}\kappa_0 + 2^{16}\gamma_1 + \gamma_0, \quad a_1 + b_1 + \kappa_0 = 2^{32}\kappa_1 + 2^{16}\gamma_3 + \gamma_2,$$

the chunks range-checked to $[0, 2^{16})$ on a range bus. It proves $(\texttt{add}, a_0, a_1, b_0, b_1, c_0, c_1, 0, 0, 0, 0)$ with $c_0 = 2^{16}\gamma_1 + \gamma_0$ and $c_1 = 2^{16}\gamma_3 + \gamma_2$, and cancels its padding rows, which prove $0 + 0 = 0$, with a single interaction per instance.

Lemma 12.3 (BinaryAdd). *The BinaryAdd chip is sound, relying on the* 16*-bit range channel: every push it makes meets the contract of* `add`*.*

Proof. Let the pulls of the range channel meet their contract, so that every chunk lies in $[0, 2^{16})$, and let a row have $a_0, a_1, b_0, b_1 < 2^{32}$. Then $c_0, c_1 < 2^{32}$. The first equation holds in $\mathbb{F}$ between two integers below $2^{33} < p$, hence over the integers, and so does the second, with $\kappa_0 \in \{0, 1\}$. Adding the first to 2^{32} times the second, $a + b = c + 2^{64}\kappa_1$ for the words of the limbs, so $c = a + b \bmod 2^{64}$ with flag 0, which is $\texttt{add}(a, b)$. The padding interaction carries the message of $0 + 0 = 0$, which meets the contract whatever its multiplicity. □

The proof is the pattern of every operation chip of ZisK: the result is decomposed into range-checked pieces, the operation is checked as an identity between integers bounded below p, and the identity holds over the integers, hence modulo 2^{64}.

12.4. **The Main Chip.**

A Main witness holds one segment of the execution. Its AIR values give the segment index j, whether it is the last segment, the program counter pc^{in} and the output c^{in} it starts from, and

the program counter $\mathrm{pc}^{\mathrm{out}}$ and the output c^{out} it ends with. Its row $r \in [0, N)$, of step $t = jN + r$, holds the fields of an instruction —its program counter pc_r, opcode, source and store flags, offsets, immediates and jump offsets— and the limbs of the operands a_r and b_r, the result c_r and the flag of the row. We write c_r^{prev} for c^{in} if $r = 0$ and for c_{r-1} otherwise, and next_r for the program counter that the step rule of the ISA gives after row r: $\mathrm{pc}_r + \mathrm{j1}$ if the flag of the row is 1, $c_{r,0} + \mathrm{j1}$ if set_pc, and $\mathrm{pc}_r + \mathrm{j2}$ otherwise.

Lemma 12.4 (Main Rows). *Let w be a Main witness with $\mathrm{Local}_{\mathrm{Main}}(\boldsymbol{p}, w)$ whose pulls on the ROM channel, the operation channels and the range channels meet their contracts. Then for every row r:*

1. Fetch. *The row holds the fields of the message of P_L at pc_r, and pc_r is an address of P_L.*
2. Operands. *a_r is c_r^{prev} if its source is* `C`*, the immediate of the instruction if it is* `IMM`*, and otherwise the value of the access that the row pulls on the Memory Bus: the load of the word at address* a_off *(*`MEM`*), at memory step $1 + 4t$, or, for register* a_off *(*`REG`*), the access at that memory step that the row pushes after pulling the previous access to the register. Likewise b_r, at memory step $1 + 4t + 1$, where an indirect source (*`IND`*) loads* ind_w *bytes at address $a_{r,0} +$* b_off*, with $a_{r,1} = 0$.*
3. Operation. *If the instruction is internal, it is* `flag`*, with result 0 and flag 1, or* `copyb`*,* `fcall_param`*,* `fcall` *or* `fcall_get`*, with result b_r and flag 0. If it is external and the limbs of a_r and b_r are well formed, then the result and the flag of the row are those of the operation on a_r and b_r, the result is well formed, and, for a precompile, its memory effect at step t is the one its prover makes.*
4. Store. *If the instruction stores, the row pulls on the Memory Bus, at memory step $1 + 4t + 2$, the store of $\mathrm{pc}_r + \mathrm{j2}$ if* store_pc *and of c_r otherwise, at the address or register of its destination.*
5. Control. $\mathrm{pc}_0 = \mathrm{pc}^{\mathrm{in}}$, $\mathrm{pc}_{r+1} = \mathrm{next}_r$ *for $r < N - 1$, and* $\mathrm{pc}^{\mathrm{out}} = \mathrm{next}_{N-1}$ *and* $c^{\mathrm{out}} = c_{N-1}$.

To do. Proof: from the constraints of the Main AIR at v0.16.1: the ROM lookup with the packed flags, the source and store selectors, the internal opcodes, the operation lookup with the m32 masking, and the program-counter constraint.

The memory accesses that the lemma names are pulls on a counted channel: the lemma says which accesses a row makes, not what they return. Section 14 shows that they return the memory of the execution, and Section 13 that the segments follow one another.

12.5. **The Memory AIRs.**

The three Memory AIRs are instances of one template, for the RAM, the ROM data and the input data regions. A witness of a region holds a slice of rows $(\mathrm{addr}, t_{\mathrm{mem}}, v, \mathrm{wr}, \mathrm{sel})$ of the region's access log, preceded by the payload B^{in} that it pulls on the continuation bus, which plays the role of the row before its first row.

Lemma 12.5 (Memory Rows). *Let w be a witness of the Memory AIR of a region, with $\mathrm{Local}_{\tau}(\boldsymbol{p}, w)$, whose pulls on the range channels meet their contracts. Then the sequence of its rows, preceded by B^{in}, satisfies the conditions of a log of the region, with the first address of the first segment new; its addresses lie between the base and the end of the region; the accesses it proves on the Memory Bus are those of its selected rows, as aligned 8-byte operations; and it pushes on the continuation bus its last row as the payload of the next segment, unless it is the last segment.*

To do. Proof: from the constraints of the Mem template at v0.16.1, for its RAM, immutable and free-input variants.

12.6. **Obligations.**

The soundness of the other chips is stated as an obligation, in the same form: every push meets the contract of its channel, relying on the table and range channels and, where stated, on operation channels; and every interaction of the chip on the Memory Bus has multiplicity 0, 1 or -1. Table 5 lists them.

Chips	**Obligation**
Binary, BinaryExtension	sound for the operation channels of their opcodes
Arith	sound for the operation channels of its opcodes, relying on the comparisons it assumes from Binary
Add256, ArithEq, ArithEq384, Keccakf, Sha256f, Poseidon2, Blake2br	sound for the operation channels of their opcodes, including the memory effect
Dma, DmaMemCpy, DmaInputCpy	sound for the operation channels of the DMA opcodes, relying on the operations they assume from the DMA workers
Dma64Aligned, DmaUnaligned, DmaPrePost and their variants	sound for the operation channels of the internal DMA operations, including their memory effect
MemAlign, MemAlignByte, MemAlignReadByte, MemAlignWriteByte	every sub-word or unaligned access they prove is the corresponding read or write of the aligned words they access
Fixed tables and frequent-operation caches	their fixed columns are the specified tables, and every cache row is a correct operation

TABLE 5. Obligations on the chips that this analysis does not prove sound.

12.7. **The Order.**

Proposition 12.6 (Order of the Chips). *Under the obligations of Table 5, the chips of ZisK are sound for the channels of Definition 11.7 and ordered by any linear order that places*

1. *the fixed tables and the Rom first;*
2. *then Binary, BinaryAdd, BinaryExtension, the precompiles and the DMA workers;*
3. *then Arith and the DMA front-ends;*
4. *then the MemAlign chips and the Memory AIRs;*
5. *and the Main chip last.*

Proof. Soundness. The Rom and the fixed tables are sound, with unconditional pushes, by Lemma 12.1 and Lemma 12.2, under the obligation on the contents of the fixed tables and the caches; BinaryAdd is sound, relying on a range channel, by Lemma 12.3. The Main chip and the Memory AIRs push only on the memory and continuation buses, whose channels are counted, so every push they make meets its contract: they are sound, establishing Lemma 12.4 and Lemma 12.5 respectively, relying on the ROM, operation and range channels. The other chips are sound by the obligations.

Order. Every chip may rely on the table and range channels, whose only pushers are the fixed tables, first in the order. Beyond these, the chips that rely on a predicate channel are Arith, on the comparison operations, which Binary and the caches prove; the DMA front-ends, on the internal operations, which the DMA workers prove; and the Main chip, on the ROM channel, which the Rom proves, and on the operation channels, which the operation chips and the caches prove. In each case the pushers come earlier in the order, or are the caches, whose pushes are unconditional. □

Corollary 12.7 (Contracts of ZisK). *Let* $(\boldsymbol{p}_{\text{ext}}; (\boldsymbol{p}_{\text{pv}}, L)) \in \mathcal{R}_{\mathsf{ZisK}}$*, let the Rom witnesses of* L *have the same cached columns, with* P_L *valid, and let the obligations of Table 5 hold. Then every pull and every push of* $\mathcal{J}_{\boldsymbol{p}}(L)$ *meets its contract, every Main witness of* L *satisfies Lemma 12.4*

and every Memory witness Lemma 12.5, and the memory and continuation buses carry the same multiset of messages pushed as pulled.

Proof. The witnesses of L are locally satisfying and L is balanced for $\boldsymbol{p}$, with $|L| < 2^B$, so $\Lambda(L) + g \leq \Lambda_{\max} < p$. The global messages are compatible with the channels of Definition 11.7, and the chips are sound and ordered by Proposition 12.6. Theorem 11.6 then gives the contracts, and the specifications Lemma 12.4 and Lemma 12.5 of the Main witnesses and the Memory witnesses. Its third statement applies to the memory and continuation buses, whose multiplicities are 0, 1 and -1: the Memory AIRs and the continuation interactions have boolean selectors; the selectors of the Main chip are its source and store flags, which the validity of P_L makes exclusive; and for the other chips this is part of their obligations. □

13. Segmentation and Stitching

A chip with state is split into segments, each proved on its own, and the continuation buses chain them. These buses carry no contract: their meaning comes from counting, which Corollary 12.7 provides. This section states what counting gives.

13.1. Chains of Segments.

Definition 13.1 (Chain). A finite list of *segments*, each with an index $k < p$, a *start* payload, an *end* payload and a *last* flag, is a *chain from b_0 to b_f* if, for some $n \geq 1$, the indices are $0, ..., n-1$, one each; segment 0 starts with b_0; the segment of index $k+1$ starts with the end of the segment of index k; only the segment of index $n-1$ is last; and it ends with b_f. It is an *open chain from b_0* if it satisfies the same conditions except that it need not end with b_f.

Lemma 13.2 (Chains by Counting). *Let every segment of a list pull the message (k, start) of its index and start, and push $(k+1, \text{end})$ if it is not last, and the terminal message* end *otherwise, which no indexed message equals; let a global message push $(0, b_0)$ and pull the terminal message b_f; and let the messages pushed be the messages pulled, as multisets, with fewer segments than p. Then the segments form a chain from b_0 to b_f. Without the terminal messages and the global pull, they form an open chain from b_0.*

Proof. Let S be the list of segments, $n := |S| < p$, and fix a bijection β from the pushes to the pulls that preserves messages. The terminal messages are pushed by the last segments and pulled once, by the global message, so exactly one segment ℓ is last, and it ends with b_f. Without the terminal messages, n indexed messages are pulled and $1 + (n-k)$ pushed if k segments are last, so again exactly one is.

Let s_0 be the segment whose pull β matches with the global push of $(0, b_0)$, and, for a segment s that is not last, let $\text{succ}(s)$ be the segment whose pull β matches with the push of s. Then s_0 has index 0 and starts with b_0, and $\text{succ}(s)$ has index $k_s + 1$ in $\mathbb{F}$ and starts with the end of s. Since β is a bijection, succ is injective, s_0 is not in its image, and every other segment is in it.

Let $s_{i+1} := \text{succ}(s_i)$ while s_i is not last. The s_i are distinct: for a repetition $s_i = s_j$ with $i < j$ and i minimal, $i = 0$ is excluded since s_0 is not in the image of succ, and otherwise $s_{i-1} = s_{j-1}$ by injectivity. The sequence therefore ends, at ℓ. The other segments R are not last, and each is the successor of a segment of R, so succ permutes R; along a cycle of length c the index increases by 1 at every step, so $c \equiv 0 \bmod p$, which is impossible for $0 < c \leq n < p$. Hence R is empty, the segments are $s_0, ..., s_{n-1} = \ell$, and s_i has index i in $\mathbb{F}$, which is the integer i since $i < p$. □

13.2. Main and Memory Segments.

Lemma 13.3 (Main Segments). *Under the hypotheses of Corollary 12.7, the Main witnesses of L form a chain from $(\mathrm{pc}_{\mathrm{BOOT}}, 0)$ to $(\mathrm{pc}_{\mathrm{END}}, 0)$ for the payloads $(\mathrm{pc}^{\mathrm{in}}, c^{\mathrm{in}})$ and $(\mathrm{pc}^{\mathrm{out}}, c^{\mathrm{out}})$ of Lemma 12.4. If there are m of them, $mN \leq 2^{36}$.*

To do. Proof: Lemma 13.2 on the Main continuation bus, whose messages are $(j, \mathrm{last}, \mathrm{pc}, c_0, c_1)$, with the genesis and terminus anchors of the global messages; the range check of the segment index gives $mN \leq 2^{36}$.

Lemma 13.4 (Memory Segments). *Under the hypotheses of Corollary 12.7, the witnesses of the Memory AIR of a region form an open chain from the genesis payload of the region, if the region is enabled, and there are none otherwise. Their rows, concatenated in the order of the chain, form one log of the region.*

To do. Proof: Lemma 13.2 on the memory continuation bus, whose messages carry the base address of the region, and Lemma 12.5 for each segment; a region whose genesis anchor is switched off by its proof value has no start, hence no segment.

The continuation buses of the DMA workers carry the operation in flight across a segment boundary. That they form chains, so that every bulk operation is processed once and completely, is part of the obligations on the DMA chips (Table 5).

13.3. The Global Step.

Definition 13.5 (Steps of the Main Segments). Let the Main witnesses of L form a chain of m segments. Row r of the segment of index j *executes* step $t = jN + r$, for $t < T := mN$, and we write pc_t, a_t, b_t, c_t and c_t^{prev} for the values of that row.

Lemma 13.6 (Global Control Flow). *Under the hypotheses of Corollary 12.7, every step $t < T$ is executed by exactly one row; $\mathrm{pc}_0 = \mathrm{pc}_{\mathrm{BOOT}}$ and $c_0^{\mathrm{prev}} = 0$; $\mathrm{pc}_{t+1} = \mathrm{next}_t$ and $c_{t+1}^{\mathrm{prev}} = c_t$ for $t + 1 < T$; and $\mathrm{next}_{T-1} = \mathrm{pc}_{\mathrm{END}}$ and $c_{T-1} = 0$.*

Proof. By Lemma 13.3 the segments have the indices $0, ..., m-1$, one each, so $(j, r) \mapsto jN + r$ is a bijection from their rows to $[0, T)$. By Corollary 12.7 every Main witness satisfies Lemma 12.4. Within a segment, its control statement gives $\mathrm{pc}_{t+1} = \mathrm{next}_t$, and $c_{t+1}^{\mathrm{prev}} = c_t$ by the definition of c^{prev}. Across the boundary between the segments j and $j+1$, the segment $j+1$ starts with the end of the segment j, so its first row has $\mathrm{pc} = \mathrm{pc}^{\mathrm{in}} = \mathrm{pc}^{\mathrm{out}} = \mathrm{next}_t$ and $c^{\mathrm{prev}} = c^{\mathrm{in}} = c^{\mathrm{out}} = c_t$ for $t = (j+1)N - 1$. The segment 0 starts with $(\mathrm{pc}_{\mathrm{BOOT}}, 0)$, which gives pc_0 and c_0^{prev}, and the segment $m-1$ ends with $(\mathrm{pc}_{\mathrm{END}}, 0)$, which gives next_{T-1} and c_{T-1}. □

14. Memory

The Main chip, the MemAlign chips and the precompiles access memory by pulling messages on the Memory Bus; the Memory AIRs prove the aligned accesses of each region as a sorted log; and the registers are carried by a linked-step argument of the Main chip on the same bus. The bus carries no contract. This section states how counting on it, together with the logs, makes every load of the execution return its memory.

14.1. Accesses.

Definition 14.1 (Accesses of a Collection). An *access* is a tuple $(\mathrm{op}, \mathrm{addr}, t_{\mathrm{mem}}, \mathrm{width}, v)$ of an operation `LOAD`, `STORE` or `REG`, an address, a memory step, a width in bytes and a word v, read from a memory-bus message whose value limbs are well formed (Lemma 10.3). The *requested* accesses of L are those that the Main chip, the MemAlign chips and the precompiles pull; an access is *aligned* if its width is 8 and its address a multiple of 8.

Lemma 14.2 (Memory Bus). *Under the hypotheses of Corollary 12.7:*

1. *the aligned* `LOAD` *and* `STORE` *accesses requested in a region, together with those that the MemAlign chips request to implement the others, are the submitted accesses of the region's log of Lemma 13.4, as multisets;*
2. *every sub-word or unaligned access is implemented by MemAlign as the corresponding read or write of the aligned words it covers; and*
3. *the* `REG` *accesses are pushed and pulled only by the Main chip and by the global messages that initialize the registers.*

To do. Proof: the third statement of Corollary 12.7 on the Memory Bus; the regions are disjoint and identified by their addresses; (2) is the obligation on MemAlign (Table 5); a message determines its access, since addresses and steps are below p.

14.2. **The Memory of an Execution.**

Definition 14.3 (Memory of a Step). For $t \leq T$, the *memory* mem_t of step t is the function from byte addresses to bytes that holds: in the RAM region, the bytes of the last write at a memory step below $1 + 4t$, or zero if there is none; in the ROM data region, the bytes of the unique write to the word, or zero if there is none; in the input data region, the value of the word in its log; and zero elsewhere.

Lemma 14.4 (Offline Memory Check). *Let a multiset of accesses of a region be the accesses of a log of the region from its genesis payload.*

1. RAM. *The writes to an address have distinct memory steps, and every read returns the value of the last write to its address at a memory step at most its own, or zero if there is none.*
2. ROM data. *Every word is written at most once, and every read returns the value of that write, or zero if there is none.*
3. Input data. *All the reads of a word other than the free-input port return the same value.*

Proof. The addresses of a log do not decrease, so the accesses to an address form a contiguous block, whose first access is the only one with a new address.

(1) Within a block, memory steps do not decrease and increase strictly at every write, so the writes have distinct steps; the writes before a read in the block have steps at most its own, and the writes after it larger steps. By induction along the block, every read returns the value of the last write before it, or zero if there is none: the first access, if a read, returns zero, and every other read returns the value of its predecessor, which is that write, or a read that returns the value of that write by induction.

(2) Within a block, the first access is the only one that can be a write, and every other access is a read that returns the value of its predecessor; so every access returns the value of the first, which is the value of the write if it is one and zero otherwise.

(3) Within a block of an address other than the free-input port, every access but the first returns the value of its predecessor. □

Lemma 14.5 (Registers). *Under the hypotheses of Corollary 12.7, every read of register* $i \in [1, 31]$ *at memory step* t_{mem} *returns the value of the last write to it at a smaller memory step, or zero if there is none.*

To do. Proof: every register access pulls the previous access of its register and pushes its own, with a range-checked increasing memory step; the global messages open every chain with value zero at step 0, and the end-of-segment

reloads carry each register across a segment boundary. With Lemma 14.2 (3), the accesses of each register form one chain ordered by memory step.

Lemma 14.6 (Loads Read the Memory). *Under the hypotheses of Corollary 12.7, if every word stored at a step before t is well formed, then every load of step t —an operand read of the Main chip at memory step $1+4t$ or $1+4t+1$, or a read of a precompile dispatched at step t— returns the bytes of mem_t at its address, except a read of the free-input port, which returns a hint; and every read of a register returns its value after step $t-1$.*

To do. Proof: Lemma 14.2, Lemma 13.4, Lemma 14.4 and Lemma 14.5, with the slot assignment of memory steps: the reads of step t come after every write of an earlier step and before the writes of step t. Since Lemma 14.4 lets a read see a write at its own memory step, this needs that no write at slot 2 of step t (the store of the Main row) is to an address that a precompile dispatched at step t reads at that slot, which the validity of P_L ensures: a precompile instruction stores nothing in memory.

15. Reconstructing the Execution

The results of the previous sections assemble into the decoding of a balanced collection of locally satisfying witnesses into an execution of the committed program. The decoding is deterministic and involves no cryptography: beyond the binding term for the cached columns, charged in Part IV, it adds no error to the bounds of Parts I and II.

Definition 15.1 (Decoded Execution). For a collection L whose Main witnesses form a chain of m segments, the *decoded execution* $w_L = \left(P_L, \mathrm{in}, (h_t)_{t<T}, (S_t)_{t\le T}\right)$ has $T = mN$; the program P_L of Lemma 12.1; as input in, the values of the input data region in its log; as hints h_t, the values that the reads of step t take from the free-input port; and as states $S_t = (\mathrm{pc}_t, c_t^{\mathrm{prev}}, \boldsymbol{x}_t, \mathrm{mem}_t)$, with the program counter and the output of the previous step of Definition 13.5, the registers $\boldsymbol{x}_t$, where $x_t(i)$ is the value of the last write to register i at a memory step below $1+4t$, or zero if there is none, and the memory of Definition 14.3. The state S_T takes $\mathrm{pc}_T := \mathrm{next}_{T-1}$ and $c_T^{\mathrm{prev}} := c_{T-1}$.

Theorem 15.2 (Reconstruction of the Execution). *Let $(\boldsymbol{p}_{\mathrm{ext}}; (\boldsymbol{p}_{\mathrm{pv}}, L)) \in \mathcal{R}_{\mathsf{ZisK}}$, let the Rom witnesses of L have the same cached columns, with P_L valid, and let the obligations of Table 5 hold. Then $(\boldsymbol{p}_{\mathrm{ext}}; w_L) \in \mathcal{R}_{\mathrm{exec}}$.*

To do. Proof: by induction on t, every operand, result, register and memory word of the steps before t is well formed and $S_{t+1} = \mathrm{Step}_{P_L}(S_t, h_t)$, with a valid step: Lemma 12.4 gives the fetch, the operation and the store of the row of step t, Lemma 14.6 that its loads read S_t, and Lemma 13.6 the next program counter and output. The initial state, the termination at $\mathrm{pc}_{\mathrm{END}}$ with $c_T = 0$ and $T \le 2^{36}$ are Lemma 13.6 and Lemma 13.3; the ROM data clause is the log of the ROM data region; the program clause is Lemma 12.1. For the public outputs, every `pubout` meets the contract $b = \mathrm{out}_a$ of the Operation Bus; the only entries that carry an output message are its global push, with multiplicity 1, and the pulls of the Main chip, with multiplicity -1, so balance with fewer than p entries makes it pulled exactly once.

(WIP) Part IV: Composition

16. Main Theorem

The three parts compose into the statement that the paper set out to prove: an accepting final proof certifies an execution of the committed program. Part II extracts, from an accepting final proof, a balanced collection of locally satisfying witnesses; Part I bounds the knowledge-soundness errors that this extraction is charged with; and Part III decodes the collection into an execution, adding no error.

16.1. **Execution Extractability.**

The external verifier checks a proof for a given program: it compares the program commitment of the statement with the commitment of the program it expects. The guarantee is therefore stated for a fixed program.

Definition 16.1 (Execution Extractability). Let P^* be a valid program (Section 12.1) and Π^* the final argument system of ZisK (Definition 9.3). For an extractor $\mathcal{E}$ and a PPT adversary $\mathcal{A}$, let $\mathsf{Adv}^{\mathsf{exec}}_{P^*}(\mathcal{A})$ be the probability that $\mathcal{A}$ outputs $(\boldsymbol{p}_{\mathrm{ext}}, \pi)$ with $\mathbf{V}^*(\boldsymbol{p}_{\mathrm{ext}}, \pi) = 1$ and $\mathrm{root}_{\mathrm{ROM}} = \mathrm{RomRoot}(P^*)$, while $(\boldsymbol{p}_{\mathrm{ext}}; \mathcal{E}(\boldsymbol{p}_{\mathrm{ext}}, \pi)) \notin \mathcal{R}_{\mathrm{exec}}$. ZisK is *execution-extractable* for P^* if there is an extractor, running in time $O(\kappa_{\max}) \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$, for which this advantage is negligible.

The definition is knowledge soundness in the sense of Definition 2.12 for the relation $\mathcal{R}_{\mathrm{exec}}$, restricted to the statements that commit to P^*, in the random-oracle model of Definition 2.15 and with a straight-line extractor.

Theorem 16.2 (Main Theorem). *Let P^* be a valid program, and assume the idealizations of Remark 2.1; that the AIR of every recursive verifier enforces its relation (Definition 6.1); and the obligations of Table 5. Then for every PPT adversary $\mathcal{A}$ that, together with the extractors composed with it, makes at most Q random-oracle queries, there are reduction adversaries $\mathcal{D}^{\mathrm{pa}}$ and $\mathcal{D}^{\mathrm{bind}}$, running in time $O(\kappa_{\max}) \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$, such that*

$$\mathsf{Adv}^{\mathsf{exec}}_{P^*}(\mathcal{A}) \le \varepsilon_{\mathrm{rec}} + Q \cdot \varepsilon^* + \varepsilon_{\mathrm{MT}}(Q) + \frac{Q \cdot \Lambda_{\mathrm{pole}}}{|\mathbb{K}|} + (Q+1) \cdot \varepsilon_{\mathcal{F}}(\Lambda_{\max}) + \mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}(\mathcal{D}^{\mathrm{pa}}) + \mathsf{Adv}^{\mathsf{bind}}_{\mathsf{Com}_{\mathrm{cach}}}(\mathcal{D}^{\mathrm{bind}}),$$

where the recursive arguments contribute

$$\varepsilon_{\mathrm{rec}} := \varepsilon_{\mathrm{ks}}(\Pi_{\mathrm{FINAL}}) + (\kappa_{\max} - 1) \cdot \varepsilon_{\mathrm{ks}}(\Pi_{\mathrm{AGG}}) + \kappa_{\max} \cdot \max_{\tau \in [M]} \varepsilon_{\mathrm{ks}}\Big(\Pi^{(\tau)}_{\mathrm{LEAF}}\Big) + \kappa_{\max} \cdot \max_{\tau \in T_{\mathrm{COMP}}} \varepsilon_{\mathrm{ks}}\Big(\Pi^{(\tau)}_{\mathrm{COMP}}\Big),$$

with $\varepsilon_{\mathrm{ks}}(\Pi_X) := Q \cdot \varepsilon^*_{\Pi_X} + \varepsilon_{\mathrm{MT}}(Q)$ the bound of Theorem 6.2 for the argument Π_X.

Proof. The extractor runs the extractor $\mathcal{E}_{\mathsf{ZisK}}$ of Theorem 9.5 and decodes its output by Definition 15.1. Let $\mathcal{A}$ output $(\boldsymbol{p}_{\mathrm{ext}}, \pi)$, accepted, with $\mathrm{root}_{\mathrm{ROM}} = \mathrm{RomRoot}(P^*)$.

Extraction. By Theorem 9.5, except with the probability that its bound states, $\mathcal{E}_{\mathsf{ZisK}}$ outputs $(\boldsymbol{p}_{\mathrm{pv}}, L)$ with $(\boldsymbol{p}_{\mathrm{ext}}; (\boldsymbol{p}_{\mathrm{pv}}, L)) \in \mathcal{R}_{\mathsf{ZisK}}$. In that bound, every knowledge-soundness advantage of a recursive argument Π_X is at most $\varepsilon_{\mathrm{ks}}(\Pi_X)$ by the second statement of Theorem 6.2, which applies since the AIR of its verifier enforces its relation; this gives $\varepsilon_{\mathrm{rec}}$ and the remaining terms.

Program. Every Rom witness of L is locally satisfying, so its cached columns open $\mathrm{root}_{\mathrm{ROM}} = \mathrm{RomRoot}(P^*)$. If the cached columns of some Rom witness are not the table of P^*, the adversary $\mathcal{D}^{\mathrm{bind}}$ that runs $\mathcal{A}$ and $\mathcal{E}_{\mathsf{ZisK}}$ and outputs these columns together with the table of P^* breaks the binding of $\mathsf{Com}_{\mathrm{cach}}$. Otherwise all the Rom witnesses have the same cached columns, and $P_L = P^*$ is valid.

Decoding. In that case Theorem 15.2, under the obligations of Table 5, gives $(\boldsymbol{p}_{\text{ext}}; w_L) \in \mathcal{R}_{\text{exec}}$, without further error. Summing the failure probabilities gives the bound. The reductions run $\mathcal{A}$ once and the extraction chain of Theorem 9.5, whose time Theorem 9.5 bounds, and the decoding is linear in the size of L. □

The theorem is the composition of statements, two of which are proved in this paper as stated — Theorem 6.2 in Part I and Theorem 9.5 in Part II— and one of which, Theorem 15.2, is stated in Part III together with the lemmas it rests on, some of whose proofs are left to a machine-checked formalization. Section 17 records every assumption and obligation the theorem rests on.

Remark 16.1 (Concrete Reading). *The dominant terms are $Q \cdot \varepsilon^*$; the knowledge-soundness errors of the recursive arguments, each about Q times the query error of Table 4, and multiplied by $\kappa_{\max} - 1$ for the aggregation nodes and by $\kappa_{\max} = 2^{20} - 1$ for the leaf verifiers and the compressors; and the bus term $(Q+1) \cdot \varepsilon_{\mathcal{F}}(\Lambda_{\max})$, which grows with the square of the number of interactions: with traces of height up to 2^{22}, $\Lambda_{\max} \geq 2^{42}$ and $\varepsilon_{\mathcal{F}}(\Lambda_{\max}) \geq \ell \cdot 2^{-109}$, of the same order as the leaf-verifier terms. The remaining terms are negligible for $|\mathbb{K}| \approx 2^{192}$. The factor $\kappa_{\max}$ comes from bounding the extractions layer by layer, and Remark 9.1 explains how a single union bound over the queries would remove it. With $B = 20$, $\kappa_{\max} \approx 10^6$: the extractor runs in time $(3\kappa_{\max} + 1) \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ and the reductions in $(3\kappa_{\max} + 2) \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ (Theorem 9.5), about $3 \cdot 10^6$ times that of the adversary. Together with the idealizations of Remark 2.1, the bound should be read as a statement about the structure of the reduction rather than as a concrete security level.*

17. Ledger and Limitations

Every term of Theorem 16.2, and every hypothesis it rests on, is defined and proved in this paper, stated in it and left to prove, inherited from a standard result, or an explicit assumption. Table 6 records the status of each.

Term or hypothesis	**Status**	**Where**
ε^*, $\varepsilon^*_{\Pi_X}$	proved	Round-by-round errors of Theorem 4.17 and Corollary 4.19, with grinding (Lemma 5.6); concrete levels in Table 4.
$\varepsilon_{\text{MT}}(Q)$	standard, in the random-oracle model	Merkle trees over a random oracle (Definition 5.2, Theorem 5.5).
Joint extraction of the AIR witnesses	proved	Lemma 5.8 and the first statement of Theorem 6.2.
$Q \cdot \Lambda_{\text{pole}}/\lvert\mathbb{K}\rvert$	proved	The poles of the bus columns (Remark 4.1), charged in Lemma 5.8.
$\varepsilon_{\mathcal{F}}(\Lambda_{\max})$	proved	LogUp (Lemma 7.1) and the bus argument (Proposition 7.4).
Enforcement of the recursive relations	assumption on the implementation	Definition 6.1.
Straight-line composition	idealization	Remark 2.1, first item.
$\mathsf{Adv}^{\mathsf{pa}}_{\mathcal{H}}$	idealization	Definition 2.14 and Remark 2.1, second item; LtHash is only known to be collision resistant, and the rigorous alternative is Remark 7.1.
$\mathsf{Adv}^{\mathsf{bind}}_{\mathsf{Com}_{\text{cach}}}$	standard, in the random-oracle model	Binding of a Merkle tree over a random oracle (Definition 2.11).

Term or hypothesis	Status	Where
$\Lambda_{\max} < p$	parameter condition	Lemma 11.2; it holds for ZisK.
Part III	stated, partly proved	Theorem 15.2 and the lemmas of Section 11 to Section 14; the proofs that read the constraints of the AIRs, and that of Theorem 15.2, are not given here.
Chip obligations	obligation	Table 5: the chips not proved sound in Part III, and the contents of the fixed tables and of the frequent-operation caches.
Valid program	assumption on P^*	Section 12.1; every program produced by the transpiler is valid.
Transpiler correctness	assumption, outside the theorem	Definition 10.1; it relates the committed ZisK program to the RISC-V binary.

TABLE 6. Ledger of the terms and hypotheses of Theorem 16.2.

The analysis is about soundness, of the protocol as deployed in ZisK v0.16.1. The following items are conditions on the implementation that the theorem assumes, or are outside it.

- **Requirements on the recursive verifiers.** The relations of Section 3.3 require the recursive verifiers to enforce the typing predicate on every non-null child and on the root, to require at least two non-null children at every aggregation node, to hash an identifier of the type together with the commitment into the challenge contribution, to expose the trace commitment as a public input of the AIR argument, and to feed the public statement, which carries the cached-column commitment, into the key derivation (Section 3.4).
- **Completeness.** That the honest pipeline produces an accepting final proof for every successful execution of length at most 2^{36} is not stated formally. The relations of Section 3.3 admit the honest proofs, and the converse direction of Definition 6.1 is what a proof of completeness would need from the recursive verifiers.
- **Zero knowledge.** ZisK's proofs are not analyzed for zero knowledge, and nothing in this paper depends on it.
- **Wrappers.** A final proof can be wrapped into a pairing-based SNARK for verification on chain. The wrapper is outside the analysis; a wrapped proof inherits Theorem 16.2 only if the wrapper is knowledge-sound, with a straight-line extractor, for the relation of accepting final proofs.
- **Idealizations.** Theorem 16.2 rests on the two idealizations of Remark 2.1; the bound should be read as a statement about the structure of the reduction rather than as a concrete security level (Remark 16.1).
- **Part III.** The decoding of a balanced collection into an execution is stated, with the lemmas it rests on, and only partly proved here: the proofs that read the constraints of the AIRs, and the final induction, are left to a machine-checked formalization, together with the obligations on the chips that are not proved sound (Table 5).
- **Versions.** The analysis follows the constraints of ZisK v0.16.1. Later versions change parts of the design that the analysis depends on, such as how the read-only data is initialized, and the statements of Part III must be checked again for each version.

References

[Ajt96] Miklós Ajtai. Generating Hard Instances of Lattice Problems (Extended Abstract). In *28th Annual ACM Symposium on Theory of Computing*, pages 99–108. ACM Press, May 1996. doi:10.1145/237814.237838.

[Arn+25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed-Solomon Proximity Testing with Super-Fast Verification. In *Advances in Cryptology – EUROCRYPT 2025, Part IV*, volume 15604 *of Lecture Notes in Computer Science*, pages 214–243. Springer, Cham, 2025. doi:10.1007/978-3-031-91134-7_8.

[BCG24] Annalisa Barbara, Alessandro Chiesa, and Ziyi Guan. Relativized Succinct Arguments in the ROM Do Not Exist. Cryptology ePrint Archive, Report 2024/728, 2024. URL: https://eprint.iacr.org/2024/728.

[BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive Oracle Proofs. In *TCC 2016-B: 14th Theory of Cryptography Conference, Part II*, volume 9986 *of Lecture Notes in Computer Science*, pages 31–60. Springer, Berlin, Heidelberg, 2016. doi:10.1007/978-3-662-53644-5_2.

[Ben+18a] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, 2018. URL: https://eprint.iacr.org/2018/046.

[Ben+18b] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *ICALP 2018: 45th International Colloquium on Automata, Languages and Programming*, volume 107 *of Leibniz International Proceedings in Informatics (LIPIcs)*, pages 14:1–14:17. Schloss Dagstuhl, July 2018. doi:10.4230/LIPIcs.ICALP.2018.14.

[Ben+25] Eli Ben-Sasson, Dan Carmon, Ulrich Haböck, Swastik Kopparty, and Shubhangi Saraf. On Proximity Gaps for Reed–Solomon Codes. Cryptology ePrint Archive, Report 2025/2055, 2025. URL: https://eprint.iacr.org/2025/2055.

[Blo+23] Alexander R. Block, Albert Garreta, Jonathan Katz, Justin Thaler, Pratyush Ranjan Tiwari, and Michal Zajac. Fiat-Shamir Security of FRI and Related SNARKs. In *Advances in Cryptology – ASIACRYPT 2023, Part II*, volume 14439 *of Lecture Notes in Computer Science*, pages 3–40. Springer, Singapore, 2023. doi:10.1007/978-981-99-8724-5_1.

[BN06] Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-Key model and a general forking lemma. In *ACM CCS 2006: 13th Conference on Computer and Communications Security*, pages 390–399. ACM Press, 2006. doi:10.1145/1180405.1180453.

[Can+19] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. Fiat-Shamir: from practice to theory. In *51st Annual ACM Symposium on Theory of Computing*, pages 1082–1090. ACM Press, 2019. doi:10.1145/3313276.3316380.

[Cla+03] Dwaine E. Clarke, Srinivas Devadas, Marten van Dijk, Blaise Gassend, and G. Edward Suh. Incremental Multiset Hash Functions and Their Application to Memory Integrity Checking. In Chi-Sung Laih, editor, *Advances in Cryptology – ASIACRYPT 2003*, volume 2894 *of Lecture Notes in Computer Science*, pages 188–207. Springer, Berlin, Heidelberg, November 2003. doi:10.1007/978-3-540-40061-5_12.

[DRS09] Yevgeniy Dodis, Thomas Ristenpart, and Thomas Shrimpton. Salvaging Merkle-Damgård for Practical Applications. In *Advances in Cryptology – EUROCRYPT 2009*, volume 5479 *of Lecture Notes in Computer Science*, pages 371–388. Springer, Berlin, Heidelberg, 2009. doi:10.1007/978-3-642-01001-9_22.

[Eth26] Ethereum Foundation. soundcalc. GitHub repository, 2026. URL: https://github.com/ethereum/soundcalc.

[FS87] Amos Fiat and Adi Shamir. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology – CRYPTO'86*, volume 263 *of Lecture Notes in Computer Science*, pages 186–194. Springer, Berlin, Heidelberg, August 1987. doi:10.1007/3-540-47721-7_12.

[GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A Faster Version of the Poseidon Hash Function. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *AFRICACRYPT 23: 14th International Conference on Cryptology in Africa*, volume 14064 *of Lecture Notes in Computer Science*, pages 177–203. Springer, Cham, July 2023. doi:10.1007/978-3-031-37679-5_8.

[GMW25] Albert Garreta, Nicolas Mohnblatt, and Benedikt Wagner. A Simplified Round-by-round Soundness Proof of FRI. Cryptology ePrint Archive, Report 2025/1993, 2025. URL: https://eprint.iacr.org/2025/1993.

[Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Report 2022/1530, 2022. URL: https://eprint.iacr.org/2022/1530.

[Kad+26] George Kadianakis, Dmitry Khovratovich, Benedikt Wagner, and Arantxa Zapico. A Vanilla zkVM Architecture. Ethereum Foundation Cryptography Research; reference document of the EF zkVM Security Sprint, 2026. URL: https://github.com/khovratovich/zkvm-ef-security-sprint.

[Nat15] National Institute of Standards and Technology. Secure Hash Standard (SHS). FIPS PUB 180-4, 2015. doi:10.6028/NIST.FIPS.180-4.

[O'C+20] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: One Function, Fast Everywhere. 2020. URL: https://github.com/BLAKE3-team/BLAKE3-specs.

[Zis25] ZisK. Secure Challenge Derivation in ZisK. HackMD technical note, 2025. URL: https://hackmd.io/SXKiWASTRu2QYgy7B-NGmg.

[Zis26] ZisK Core Team. ZisK Whitepaper. 2026. URL: https://github.com/0xPolygonHermez/zisk-whitepaper.